Sa., 1. Aug. um 13:24

was muss ich machen um dieses repo laufen lassen zu können? hier die readme. brauchst du sonst noch irgendwelche angaben oder dateien?

Shared functional specialization in transformer-based language models and the human brain

This repository accompanies the manuscript "Shared functional specialization in transformer-based language models and the human brain". In this manuscript we use the internal computations of a transformer model (BERT) to predict fMRI activity while subjects listen to naturalistic spoken narratives.

Kumar, S.*, Sumers, T. R.*, Yamakoshi, T., Goldstein, A., Hasson, U., Norman, K. A., Griffiths, T. L., Hawkins, R. D., & Nastase, S. A. (2024). Shared functional specialization in transformer-based language models and the human brain. *Nature Communications (accepted)*. https://doi.org/10.1101/2022.06.08.495348

The following list describes the various analysis scripts:

Text Processing and Transformer Models

`narratives-transcript-processing.ipynb`: Used to pre-process transcripts from the `Narratives` dataset. Outputs (1) phonemes and associated nuisance variables for regression analyses; (2) TR-aligned tokens for use with Transformer notebooks.

`transformer-representations.ipynb`: the primary notebook used to generate Transformer representations (embeddings, transformations) for regression analyses.

`transformer-transformation-magnitudes.ipynb`: small script used to produce transformation magnitudes from the transformations themselves.

`transformer-utils.py`: functionality for extracting various Transformer representations, including some experimental metrics that were used in the paper.

`extract_linguistic_features.py`: Extract linguistic features (parts-of-speech and dependency tags) using spaCy.

`decode_linguistic_features.py`: Run the decoding analysis, where we decode linguistic features from the headwise transformation representations. All the necessary functions should be in decode_linguistic_features_utils.py.

Data handling
`create_fmri_dataset.py`: Copy data from narratives dataset into lab project folder.

`ROIs.ipynb`: Handling ROI labeling and visualizing.

Analysis code
`banded_ridge_regression.py`: Run encoding model analyses (banded ridge regression) with a given representation as main features.

`calculate_boostrap_pvalue.py`: Calculate bootstrap pvalue (with FDR corrections) for individual parcels to determine significance.

`compute_isc.py`: Compute noise ceilings with intersubject correlation.

`headwise_banded_ridge_regression.py`: Headwise version of the encoding analyses (banded ridge regression, but knock out weights for all heads but one when evaluating a head).

`run_encoding_models_banded.py`: Creates a joblist.txt for banded ridge regression jobs to input as a slurm job array.

`run_encoding_models_headwise_banded.py`: Same as above, but for the headwise encoding analyses.

`run_isc_jobs.py`: Run multiple subjects' ISC analyses in parallel (slurm jobs).

`run_jupyter.sh`: Run a jupyter notebook on the cluster.

`run_pvalue_jobs.py`: Run bootstrap p-value analyses in parallel.

`run_save_jobs.py`: Run jobs in parallel that will save collapsed performance across subjects.

`save_mean_volume.py`: Collapse performance across subjects for a specific representation.

Visualizations
`Head_Dep_Brain_Plots.ipynb`: Plots whole brain results for headwise encoding (and dependency encoding).

`Layerwise_Plot.ipynb`: Plots performance across layers.

`Results_Figures.ipynb`: Notebook to plot most main results in the paper.

`View_Encoding_Results.ipynb`: Plots whole-brain glass brain results.

`bert_rdms.ipynb`: Compare RDMs for embeddings and transformations (as well as autocorrelation).

`layer_preference.ipynb`: Create layer preference and layer

specificity histograms.

`plot_layer_brain.ipynb`: Process and export layer preferences for visualization on cortical surface.

`headwise_specialization.ipynb`: Run PCA on transformation weights and visualization two-dimensional projections.

Slurm jobs
`avg_job.sh`: Slurm job corresponding to save_mean_volumes.py.

`boot_job.sh`: Slurm job corresponding to calculate_bootstrap_pvalue.py.

`dsq-submit.sh`: Slurm job that submits a job array specified in a .txt file.

`joblist.txt`: Job list for submitting slurm job array.

Miscellaneous
`fmri_conda_env.yml`: yml conda env file for fmri analyses.

`transformer_conda_env.yml`: yml conda env file for transcript pre-processing and Transformer analysis notebooks.

Ja. Mit der README allein kann man leider **nicht sicher sagen, wie man das Projekt zum Laufen bekommt**, weil sie nur beschreibt, *welche Skripte es gibt*, aber nicht *wie sie zusammenspielen*.

Es gibt aber ein paar Dinge, die wir schon ableiten können.

Was ich als erstes machen würde

1. Repository klonen

```
Bash
```

```
git clone <repo-url>
cd <repo>
```

---

2. Schauen, welche Conda-Umgebungen existieren

Es gibt bereits zwei Environment-Dateien:

```
fmri_conda_env.yml
transformer_conda_env.yml
```

Diese würde ich zuerst installieren.

```bash
conda env create -f transformer_conda_env.yml
```

und

```bash
conda env create -f fmri_conda_env.yml
```

Danach aktivieren:

```bash
conda activate transformer
```

oder

```bash
conda activate fmri
```

(der Name steht in der yml-Datei.)

---

3. Daten beschaffen

Das ist vermutlich der wichtigste Punkt.

Die README erwähnt mehrfach

> Narratives dataset

und

> fMRI activity

Das Repository enthält die Daten höchstwahrscheinlich **nicht**.

Man braucht vermutlich

- Narratives Dataset
- fMRI-Daten
- ROI-Dateien
- eventuell vorverarbeitete Transkripte

---

4. Transformer-Repräsentationen erzeugen

Dafür scheint dieses Notebook gedacht zu sein:

```
transformer-representations.ipynb
```

---

5. Encoding Model ausführen

Danach

```
banded_ridge_regression.py
```

oder

```
run_encoding_models_banded.py
```

Was ich von dir brauche

Damit ich dir **genau** sagen kann, wie du das Repo startest, wären folgende Dateien hilfreich.

1. Die beiden yml-Dateien ⭐⭐⭐

```
transformer_conda_env.yml
fmri_conda_env.yml
```

Damit sehe ich sofort

- Python-Version
- benötigte Packages
- CUDA-Version
- etc.

---

2. Die Projektstruktur ⭐⭐⭐

Einmal

```
</> Bash

tree -L 2
```

oder in VS Code den Explorer aufklappen und Screenshot schicken.

---

3. Die erste Ebene der Dateien

Also beispielsweise

```
README.md
banded_ridge_regression.py
config.py
...
```

---

4. Falls vorhanden

eine Datei wie

```
config.py
paths.py
settings.py
constants.py
```

Diese enthalten oft Pfade zu den Daten.

---

5. Den Anfang von

```
banded_ridge_regression.py
```

Die ersten etwa 80 Zeilen reichen meistens.

---

6. Den Anfang von

```
transformer-representations.ipynb
```

oder alternativ den ersten Codeblock.

Was ich vermute

Die Pipeline dürfte ungefähr so aussehen:

```
Narratives Dataset
        |
        ▼
Transcript preprocessing
        |
        ▼
```

```
TR-aligned tokens
        |
        ▼
BERT
        |
        ▼
Embeddings
        |
        ▼
Transformation representations
        |
        ▼
Banded Ridge Regression
```

Meine Vermutung

Da du das Projekt vermutlich **für Reproduktion oder Erweiterung** verwenden möchtest (du hattest ja schon Fragen zu den Brain-Encoding-Notebooks und den Heatmaps), würde ich **nicht** einfach versuchen, irgendwelche Skripte auszuführen.

Stattdessen würde ich zuerst gemeinsam nachvollziehen:

1. welche Daten erwartet werden,
2. wo sie liegen müssen,
3. welche Reihenfolge der Skripte vorgesehen ist,
4. welche Ergebnisse jedes Skript erzeugt.

Das spart später oft mehrere Stunden Fehlersuche.

Wenn du mir die beiden `.yml`-Dateien und die Projektstruktur schickst, kann ich dir daraus eine konkrete Schritt-für-Schritt-Anleitung erstellen (inklusive der genauen Befehle und der richtigen Reihenfolge der Skripte).

> wie kann ich mir in der konsole die ordnerstruktur ausgeben lassen?

Das hängt vom Betriebssystem bzw. der Shell ab.

Linux / macOS (oder WSL)

Am einfachsten mit `tree`:

```bash
tree
```

Nur die ersten zwei Ebenen:

```bash
tree -L 2
```

Falls `tree` nicht installiert ist:

- Ubuntu/Debian:

```bash
sudo apt install tree
```

Windows PowerShell

```powershell
tree
```

Oder mit Dateien:

```powershell
tree /F
```

Git Bash

Falls `tree` installiert ist:

```bash
tree -L 2
```

Falls nicht, kannst du stattdessen `find` verwenden:

```bash
find . -maxdepth 2
```

Falls du auf eurem LST-Cluster bist

Da `tree` auf Clustern oft nicht installiert ist, kannst du Folgendes verwenden:

```bash
find . -maxdepth 2
```

oder etwas übersichtlicher:

```bash
find . -maxdepth 2 -type d
```

Das zeigt nur die Ordner.

Wenn du in VS Code im Terminal bist, schick mir einfach die Ausgabe von:

```bash
tree -L 2
```

oder (falls das nicht funktioniert)

```bash
find . -maxdepth 2
```

Dann kann ich direkt erkennen, wie das Repository aufgebaut ist.

PS C:\Users\paulina\brains> tree
Auflistung der Ordnerpfade für Volume Windows
Volumeseriennummer : 000000D1 A42D:696B
C:.
├———.vscode
└———bert-brains
    ├———analysis
    ├———data

```
    │   ├──21st_year
    │   ├──greeneyes-bert-rsms
    │   └──stimuli
    │       ├──black
    │       ├──bronx
    │       ├──forgot
    │       ├──lucy
    │       ├──merlin
    │       ├──milkywayoriginal
    │       ├──milkywaysynonyms
    │       ├──milkywayvodka
    │       ├──notthefallintact
    │       ├──pieman
    │       ├──piemanpni
    │       ├──prettymouth
    │       ├──shapesphysical
    │       ├──shapessocial
    │       ├──sherlock
    │       └──slumlordreach
    ├──data_handling
    ├──slurm
    ├──text_processing_and_transformers
    └──visualizations
```

hier mehr info szum projekt

Perfekt, das hilft schon deutlich weiter. Das Repository sieht **gut strukturiert** aus und entspricht ziemlich genau der README.

Was ich daraus ableiten kann

```
bert-brains/
│
├── data/
│   ├── stimuli/
│   ├── 21st_year/
│   └── greeneyes-bert-rsms/
│
├── text_processing_and_transformers/
```

```
│
├── analysis/
│
├── visualizations/
│
├── data_handling/
│
└── slurm/
```

Das sieht nach einer klassischen Pipeline aus:

```
Stimuli (Transkripte)
        │
        ▼
Text Processing
        │
        ▼
BERT-Repräsentationen
        │
        ▼
Encoding Models
        │
        ▼
Visualisierung
```

Was jetzt interessant ist

Mich interessiert jetzt vor allem:

1. Sind die Daten vollständig?

Führe bitte einmal im Repo aus:

```powershell
tree data /F
```

Dann sehe ich,

- welche Dateien vorhanden sind,
- ob nur Ordner existieren,
- oder ob bereits `.nii`, `.h5`, `.csv`, `.npy`, `.pkl` usw. enthalten sind.

---

2. Welche Python-Umgebungen gibt es?

Bitte zeige einmal

```powershell
dir *.yml
```

oder

```powershell
Get-ChildItem -Recurse -Filter *.yml
```

---

3. Welche README-Dateien gibt es?

Oft haben Unterordner eigene Dokumentationen.

```powershell
Get-ChildItem -Recurse README*
```

---

4. Ganz wichtig: Gibt es ein `requirements.txt`?

```powershell
Get-ChildItem -Recurse requirements*
```

Ich vermute momentan

Das Repository ist **kein "Programm", das man einfach startet**.

Vielmehr handelt es sich um den vollständigen Code zur Reproduktion der Paper-Ergebnisse.

Das heißt, man führt nicht `main.py` aus, sondern beispielsweise:

1. `transformer-representations.ipynb`
2. `banded_ridge_regression.py`
3. `calculate_bootstrap_pvalue.py`

4. `Results_Figures.ipynb`

je nachdem, welchen Teil der Analyse man reproduzieren möchte.

Die wichtigste Frage

Was möchtest du eigentlich machen?

Zum Beispiel:

- **A)** Nur das Paper reproduzieren.
- **B)** Die BERT-Repräsentationen erzeugen.
- **C)** Die fMRI-Encoding-Modelle ausführen.
- **D)** Nur die Visualisierungen ansehen.
- **E)** Das Projekt als Grundlage für deine eigene Forschung nutzen.

Je nachdem ist der Einstieg ein ganz anderer.

```
C:\USERS\PAULINA\BRAINS\BERT-BRAINS\DATA
│   greeneyes.txt
│
├───21st_year
│       21st_year.csv
│       21st_year.xlsx
│       21st_year_edited.csv
│       21st_year_edited_two.csv
│       ling_features.csv
│       tr_tokens.csv
│       tr_tokens_new.csv
│
├───greeneyes-bert-rsms
│       layer_0_rsm
│       layer_10_rsm
│       layer_11_rsm
│       layer_12_rsm
│       layer_1_rsm
│       layer_2_rsm
│       layer_3_rsm
│       layer_4_rsm
│       layer_5_rsm
```

```
│       layer_6_rsm
│       layer_7_rsm
│       layer_8_rsm
│       layer_9_rsm
│
└───stimuli
    ├───black
    │       align.csv
    │
    ├───bronx
    │       align.csv
    │
    ├───forgot
    │       align.csv
    │
    ├───lucy
    │       align.csv
    │
    ├───merlin
    │       align.csv
    │
    ├───milkywayoriginal
    │       align.csv
    │
    ├───milkywaysynonyms
    │       align.csv
    │
    ├───milkywayvodka
    │       align.csv
    │
    ├───notthefallintact
    │       align.csv
    │
    ├───pieman
    │       align.csv
    │       tr_tokens.csv
    │
    ├───piemanpni
    │       align.csv
```

```
│
├──prettymouth
│      align.csv
│
├──shapesphysical
│      align.csv
│
├──shapessocial
│      align.csv
│
├──sherlock
│      align.csv
│
└──slumlordreach
       align.csv
```

Super, das sieht schon viel besser aus. Tatsächlich scheint das Repository **bereits mit Beispieldaten ausgeliefert** zu werden. Das ist ein gutes Zeichen.

Was ich daraus schließe

Du hast bereits:

- ✅ Stimuli (`align.csv`)
- ✅ TR-Token-Dateien
- ✅ Linguistische Features
- ✅ Beispiel-RSMs (Representational Similarity Matrices)

Was ich **noch nicht sehe**, sind die eigentlichen **fMRI-Daten** (z. B. `.nii.gz` , `.h5` , `.npy` , `.pkl`). Vermutlich sind diese aus Platz- oder Lizenzgründen nicht im Repository enthalten. Das bedeutet:

- Die **Textverarbeitung** und BERT-Analysen sollten wahrscheinlich funktionieren.
- Die **Encoding-Analysen** (`banded_ridge_regression.py`) werden vermutlich zusätzliche fMRI-Daten benötigen.

Jetzt würde ich als Nächstes Folgendes prüfen

1. Welche Umgebungen gibt es?

Bitte zeige einmal:

```powershell
```

```
Get-ChildItem -Recurse -Filter *.yml
```

2. Welche Jupyter-Notebooks gibt es?

```powershell
Get-ChildItem -Recurse -Filter *.ipynb
```

3. Welche Python-Dateien gibt es?

```powershell
Get-ChildItem -Recurse -Filter *.py
```

4. Ganz wichtig: Gibt es irgendwo Konfigurationsdateien?

```powershell
Get-ChildItem -Recurse config*
```

oder

```powershell
Get-ChildItem -Recurse settings*
```

Was ich an deiner Stelle zuerst machen würde

Da du dich (aus unseren bisherigen Gesprächen) vor allem für die Transformer- und Attention-Analysen interessierst, würde ich **nicht** direkt versuchen, die komplette fMRI-Pipeline zu starten.

Ich würde zuerst das Notebook

```
text_processing_and_transformers/
```

```
transformer-representations.ipynb
```

zum Laufen bringen.

Wenn das funktioniert, weißt du, dass

- die Python-Umgebung stimmt,
- Hugging Face/BERT geladen werden kann,
- die Daten richtig eingebunden sind.

Danach kann man sich Schritt für Schritt an die Encoding-Modelle herantasten.

Ich kann dir auch helfen, das Repository systematisch zum Laufen zu bringen

Wenn du mir noch diese beiden Dinge schickst:

1. `transformer_conda_env.yml`
2. `fmri_conda_env.yml`

und zusätzlich den Inhalt von

```
Nur Text

text_processing_and_transformers/
```

(die Dateiliste reicht),

dann kann ich dir eine konkrete Anleitung geben:

- welche Umgebung du zuerst anlegst,
- welche Pakete ggf. fehlen,
- welches Notebook du als Erstes öffnest,
- und in welcher Reihenfolge du die Pipeline sinnvoll ausführst.

Verzeichnis: C:\Users\paulina\brains\bert-brains

Mode	LastWriteTime	Length	Name
-a----	31.07.2026 17:23	2555	fmri_conda_env.yml
-a----	31.07.2026 17:23	5922	transformer_conda_env.yml

Perfekt. Als Nächstes brauche ich den **Inhalt** der beiden Dateien, nicht nur ihre Namen.

Du kannst sie z. B. so anzeigen:

```powershell
Get-Content transformer_conda_env.yml
```

und

```powershell
Get-Content fmri_conda_env.yml
```

Oder öffne sie einfach in VS Code und kopiere den Inhalt hier hinein.

Noch eine Vermutung

Da das Repository von 2024 ist, könnten die Environment-Dateien etwas älter sein. Es kann sein, dass einzelne Paketversionen heute nicht mehr installierbar sind. In dem Fall kann ich dir helfen, die Dateien anzupassen.

Zusätzlich wäre noch hilfreich

Zeig bitte einmal den Inhalt von

```Nur Text
text_processing_and_transformers/
```

mit

```powershell
dir text_processing_and_transformers
```

oder

```powershell
```

```
tree text_processing_and_transformers
```

Denn dann können wir direkt entscheiden, mit welchem Notebook oder Skript wir starten.

Mein Vorschlag

Ich würde **nicht** blind die Conda-Umgebung installieren, sondern zuerst die `transformer_conda_env.yml` prüfen. Oft enthalten solche Dateien unnötige oder veraltete Pakete. So können wir mögliche Probleme vermeiden, bevor du Zeit in eine Installation investierst.

PS C:\Users\paulina\brains\bert-brains> Get-Content transformer_conda_env.yml

```
name: bert-brains
channels:
  - pytorch
  - conda-forge
  - defaults
dependencies:
  - anyio=3.3.0=py39h6e9494a_0
  - appnope=0.1.2=py39hecd8cb5_1001
  - argon2-cffi=20.1.0=py39h9ed2024_1
  - async_generator=1.10=pyhd3eb1b0_0
  - attrs=21.2.0=pyhd3eb1b0_0
  - babel=2.9.1=pyh44b312d_0
  - backcall=0.2.0=pyhd3eb1b0_0
  - blas=1.0=mkl
  - bleach=3.3.1=pyhd3eb1b0_0
  - bottleneck=1.3.2=py39he3068b8_1
  - brotli=1.0.9=hb1e8313_2
  - brotlipy=0.7.0=py39h89e85a6_1001
  - bzip2=1.0.8=h1de35cc_0
  - ca-certificates=2021.7.5=hecd8cb5_1
  - catalogue=1.0.0=pyhd3eb1b0_0
  - certifi=2021.5.30=py39hecd8cb5_0
  - cffi=1.14.6=py39h2125817_0
  - chardet=4.0.0=py39h6e9494a_1
  - charset-normalizer=2.0.0=pyhd8ed1ab_0
  - cryptography=3.4.7=py39ha2c9959_0
  - cycler=0.10.0=py39hecd8cb5_0
  - cymem=2.0.5=py39h23ab428_0
```

```
- cython-blis=0.7.4=py39h9ed2024_1
- decorator=5.0.9=pyhd3eb1b0_0
- defusedxml=0.7.1=pyhd3eb1b0_0
- entrypoints=0.3=py39hecd8cb5_0
- ffmpeg=4.3=h0a44026_0
- fonttools=4.25.0=pyhd3eb1b0_0
- freetype=2.10.4=ha233b18_0
- gettext=0.21.0=h7535e17_0
- gmp=6.2.1=h23ab428_2
- gnutls=3.6.15=hed9c0bf_0
- icu=58.2=h0a44026_3
- idna=3.1=pyhd3deb0d_0
- importlib-metadata=3.10.0=py39hecd8cb5_0
- importlib_metadata=3.10.0=hd3eb1b0_0
- intel-openmp=2021.3.0=hecd8cb5_3375
- ipykernel=5.3.4=py39h01d92e1_0
- ipython=7.22.0=py39h01d92e1_0
- ipython_genutils=0.2.0=pyhd3eb1b0_1
- ipywidgets=7.6.3=pyhd3eb1b0_1
- jedi=0.17.2=py39hecd8cb5_1
- jinja2=3.0.1=pyhd3eb1b0_0
- jpeg=9b=he5867d9_2
- json5=0.9.5=pyh9f0ad1d_0
- jsonschema=3.0.2=py39hecd8cb5_0
- jupyter=1.0.0=py39hecd8cb5_7
- jupyter_client=6.1.12=pyhd3eb1b0_0
- jupyter_console=6.4.0=pyhd3eb1b0_0
- jupyter_core=4.7.1=py39hecd8cb5_0
- jupyter_server=1.10.2=pyhd8ed1ab_0
- jupyterlab=3.1.1=pyhd8ed1ab_0
- jupyterlab_pygments=0.1.2=py_0
- jupyterlab_server=2.6.2=pyhd8ed1ab_0
- jupyterlab_widgets=1.0.0=pyhd3eb1b0_1
- kiwisolver=1.3.1=py39h23ab428_0
- lame=3.100=h1de35cc_0
- lcms2=2.12=hf1fd2bf_0
- libcxx=10.0.0=1
- libffi=3.3=hb1e8313_2
- libgfortran=3.0.1=h93005f0_2
```

- libiconv=1.16=h1de35cc_0
- libidn2=2.3.2=h9ed2024_0
- libpng=1.6.37=ha441bb4_0
- libsodium=1.0.18=h1de35cc_0
- libtasn1=4.16.0=h9ed2024_0
- libtiff=4.2.0=h87d7836_0
- libunistring=0.9.10=h9ed2024_0
- libuv=1.40.0=haf1e3a3_0
- libwebp-base=1.2.0=h9ed2024_0
- libxml2=2.9.12=hcdb78fc_0
- llvm-openmp=10.0.0=h28b9765_0
- lz4-c=1.9.3=h23ab428_0
- markupsafe=2.0.1=py39h9ed2024_0
- matplotlib=3.4.2=py39hecd8cb5_0
- matplotlib-base=3.4.2=py39h8b3ea08_0
- mistune=0.8.4=py39h9ed2024_1000
- mkl=2021.3.0=hecd8cb5_517
- mkl-service=2.4.0=py39h9ed2024_0
- mkl_fft=1.3.0=py39h4a7008c_2
- mkl_random=1.2.2=py39hb2f4e1b_0
- munkres=1.1.4=py_0
- murmurhash=1.0.5=py39h23ab428_0
- nbclassic=0.3.1=pyhd8ed1ab_1
- nbclient=0.5.3=pyhd3eb1b0_0
- nbconvert=6.1.0=py39hecd8cb5_0
- nbformat=5.1.3=pyhd3eb1b0_0
- ncurses=6.2=h0a44026_1
- nest-asyncio=1.5.1=pyhd3eb1b0_0
- nettle=3.7.3=h230ac6f_1
- ninja=1.10.2=hf7b0b51_1
- notebook=6.4.0=py39hecd8cb5_0
- numexpr=2.7.3=py39h5873af2_1
- numpy=1.20.3=py39h4b4dc7a_0
- numpy-base=1.20.3=py39he0bd621_0
- olefile=0.46=py_0
- openh264=2.1.0=hd9629dc_0
- openjpeg=2.3.0=hb95cd4c_1
- openssl=1.1.1k=h9ed2024_0
- packaging=21.0=pyhd3eb1b0_0

- pandas=1.3.0=py39h23ab428_0
- pandocfilters=1.4.3=py39hecd8cb5_1
- parso=0.7.0=py_0
- pexpect=4.8.0=pyhd3eb1b0_3
- pickleshare=0.7.5=pyhd3eb1b0_1003
- pillow=8.3.1=py39ha4cf6ea_0
- pip=21.2.2=py39hecd8cb5_0
- plac=1.1.0=py39hecd8cb5_0
- preshed=3.0.5=py39h23ab428_4
- prometheus_client=0.11.0=pyhd3eb1b0_0
- prompt-toolkit=3.0.17=pyh06a4308_0
- prompt_toolkit=3.0.17=hd3eb1b0_0
- ptyprocess=0.7.0=pyhd3eb1b0_2
- pycparser=2.20=py_2
- pygments=2.9.0=pyhd3eb1b0_0
- pyopenssl=20.0.1=pyhd8ed1ab_0
- pyparsing=2.4.7=pyhd3eb1b0_0
- pyqt=5.9.2=py39h23ab428_6
- pyrsistent=0.18.0=py39h9ed2024_0
- pysocks=1.7.1=py39h6e9494a_3
- python=3.9.6=h88f2d9e_0
- python-dateutil=2.8.2=pyhd3eb1b0_0
- python_abi=3.9=2_cp39
- pytorch=1.9.0=py3.9_0
- pytz=2021.1=pyhd8ed1ab_0
- pyzmq=20.0.0=py39h23ab428_1
- qt=5.9.7=h468cd18_1
- qtconsole=5.1.0=pyhd3eb1b0_0
- qtpy=1.9.0=py_0
- readline=8.1=h9ed2024_0
- requests=2.26.0=pyhd8ed1ab_0
- requests-unixsocket=0.2.0=py_0
- scipy=1.6.2=py39hd5f7400_1
- seaborn=0.11.1=pyhd3eb1b0_0
- send2trash=1.5.0=pyhd3eb1b0_1
- setuptools=52.0.0=py39hecd8cb5_0
- sip=4.19.13=py39h23ab428_0
- six=1.16.0=pyhd3eb1b0_0
- sniffio=1.2.0=py39h6e9494a_1

```
- spacy=2.3.5=py39hf7b0b51_0
- sqlite=3.36.0=hce871da_0
- srsly=1.0.5=py39h23ab428_0
- terminado=0.9.4=py39hecd8cb5_0
- testpath=0.5.0=pyhd3eb1b0_0
- thinc=7.4.5=py39h1341a74_0
- tk=8.6.10=hb0a8c7a_0
- torchaudio=0.9.0=py39
- torchvision=0.10.0=py39_cpu
- tornado=6.1=py39h9ed2024_0
- traitlets=5.0.5=pyhd3eb1b0_0
- typing_extensions=3.10.0.0=pyh06a4308_0
- tzdata=2021a=h52ac0ba_0
- urllib3=1.26.6=pyhd8ed1ab_0
- wasabi=0.8.2=pyhd3eb1b0_0
- wcwidth=0.2.5=py_0
- webencodings=0.5.1=py39hecd8cb5_1
- websocket-client=0.57.0=py39h6e9494a_4
- wheel=0.36.2=pyhd3eb1b0_0
- widgetsnbextension=3.5.1=py39hecd8cb5_0
- xz=5.2.5=h1de35cc_0
- zeromq=4.3.4=h23ab428_0
- zipp=3.5.0=pyhd3eb1b0_0
- zlib=1.2.11=h1de35cc_3
- zstd=1.4.9=h322a384_0
- pip:
  - click==8.0.1
  - en-core-web-lg==2.2.0
  - en-core-web-sm==2.2.0
  - filelock==3.0.12
  - huggingface-hub==0.0.12
  - joblib==1.0.1
  - pyyaml==5.4.1
  - regex==2021.8.3
  - sacremoses==0.0.45
  - sentencepiece==0.1.96
  - tokenizers==0.10.3
  - tqdm==4.62.0
  - transformers==4.9.1
```

```
prefix: /Users/sumers/opt/anaconda3/envs/bert-brains


prefix: /Users/sumers/opt/anaconda3/envs/bert-brains
PS C:\Users\paulina\brains\bert-brains> Get-Content
fmri_conda_env.yml
name: fmri
channels:
  - conda-forge
  - defaults
dependencies:
  - _libgcc_mutex=0.1=main
  - binutils_impl_linux-64=2.31.1=h6176602_1
  - binutils_linux-64=2.31.1=h6176602_9
  - blas=2.14=openblas
  - ca-certificates=2020.7.22=0
  - certifi=2020.6.20=py37_0
  - cycler=0.10.0=py37_0
  - cython=0.29.21=py37he6710b0_0
  - decorator=4.4.2=py_0
  - freetype=2.10.2=h5ab3b9f_0
  - gcc_impl_linux-64=7.3.0=habb00fd_1
  - gcc_linux-64=7.3.0=h553295d_9
  - gxx_impl_linux-64=7.3.0=hdf63c60_1
  - gxx_linux-64=7.3.0=h553295d_9
  - h5py=2.10.0=py37hd6299e0_1
  - hdf5=1.10.6=hb1b8bf9_0
  - joblib=0.16.0=py_0
  - jpeg=9b=h024ee3a_2
  - kiwisolver=1.2.0=py37hfd86e86_0
  - lcms2=2.11=h396b838_0
  - ld_impl_linux-64=2.33.1=h53a641e_7
  - libblas=3.8.0=14_openblas
  - libcblas=3.8.0=14_openblas
  - libedit=3.1.20191231=h14c3975_1
  - libffi=3.3=he6710b0_2
  - libgcc-ng=9.1.0=hdf63c60_0
  - libgfortran-ng=7.3.0=hdf63c60_0
  - libgpuarray=0.7.6=h14c3975_0
```

- liblapack=3.8.0=14_openblas
- liblapacke=3.8.0=14_openblas
- libopenblas=0.3.7=h5ec1e0e_6
- libpng=1.6.37=hbc83047_0
- libstdcxx-ng=9.1.0=hdf63c60_0
- libtiff=4.1.0=h2733197_1
- llvm-openmp=8.0.1=hc9558a2_0
- lz4-c=1.9.2=he6710b0_1
- mako=1.1.3=py_0
- markupsafe=1.1.1=py37h14c3975_1
- matplotlib-base=3.3.1=py37h817c723_0
- mpi=1.0=mpich
- mpi4py=3.0.3=py37hf046da1_1
- mpich=3.3.2=hc856adb_0
- ncurses=6.2=he6710b0_1
- networkx=2.5=py_0
- nibabel=3.1.1=py_0
- nitime=0.8.1=py37hc1659b7_1
- numpy=1.19.1=py37h30dfecb_0
- numpy-base=1.19.1=py37h75fe3a5_0
- olefile=0.46=py37_0
- openmp=8.0.1=0
- openssl=1.1.1g=h7b6447c_0
- packaging=20.4=py_0
- pandas=1.1.1=py37he6710b0_0
- patsy=0.5.1=py37_0
- pillow=7.2.0=py37hb39fc2d_0
- pip=20.2.2=py37_0
- psutil=5.7.2=py37h7b6447c_0
- pybind11=2.5.0=py37hfd86e86_0
- pydicom=2.0.0=pyh9f0ad1d_0
- pygpu=0.7.6=py37heb32a55_0
- pyparsing=2.4.7=py_0
- python=3.7.9=h7579374_0
- python-dateutil=2.8.1=py_0
- pytz=2020.1=py_0
- readline=8.0=h7b6447c_0
- scikit-learn=0.21.3=py37hcdab131_0
- scipy=1.5.2=py37habc2bb6_0

```
  - setuptools=49.6.0=py37_0
  - six=1.15.0=py_0
  - sqlite=3.33.0=h62c20be_0
  - statsmodels=0.11.1=py37h7b6447c_0
  - theano=1.0.4=py37hfd86e86_0
  - tk=8.6.10=hbc83047_0
  - tornado=6.0.4=py37h7b6447c_1
  - wheel=0.35.1=py_0
  - xz=5.2.5=h7b6447c_0
  - zlib=1.2.11=h7b6447c_3
  - zstd=1.4.5=h9ceee32_0
  - pip:
    - brainiak==0.10
prefix: /usr/people/sreejank/anaconda3/envs/fmri
```

Perfekt – jetzt kann man schon viel konkreter sagen, wie das Projekt aufgebaut ist.

Erste Beobachtung: Die Environments sind nicht für Windows erstellt

Man sieht das an mehreren Stellen:

Im `fmri_conda_env.yml` stehen z. B.

```yaml
binutils_linux-64
gcc_linux-64
gxx_linux-64
_libgcc_mutex
```

und der Prefix

```
/usr/people/sreejank/anaconda3/envs/fmri
```

→ Das ist eindeutig ein **Linux-Cluster**.

Im `transformer_conda_env.yml` steht dagegen

```
prefix: /Users/sumers/opt/anaconda3/envs/bert-brains
```

→ Das wurde auf **macOS** exportiert.

Das heißt: **Diese Dateien wirst du unter Windows nicht 1:1 installieren können.**

Gute Nachricht

Das Transformer-Environment ist trotzdem problemlos auf Windows nachzubauen.

Es enthält hauptsächlich

- Python 3.9
- PyTorch 1.9
- transformers 4.9
- spaCy 2.3
- Jupyter

Das sind alles plattformunabhängige Pakete.

Das fMRI-Environment dagegen ...

... würde ich **gar nicht erst lokal installieren**.

Dort stecken

- MPI
- BrainIAK
- Theano
- Linux-Compiler
- OpenMP

Das wurde offensichtlich für einen HPC-Cluster gebaut.

Ich vermute stark, dass die Autoren alle eigentlichen Encoding-Analysen per **Slurm** gerechnet haben (passt auch zu deinem Repo mit dem `slurm/`-Ordner).

Ich würde deshalb so vorgehen

Phase 1 (lokal auf Windows)

Lokal würde ich nur den Transformer-Teil benutzen:

```
text_processing_and_transformers/
```

also

- BERT laden
- Attention analysieren
- Hidden States erzeugen
- Transformationen berechnen

Das sollte problemlos funktionieren.

Phase 2 (Cluster)

Falls du später wirklich

```
analysis/
```

mit

```
banded_ridge_regression.py
```

laufen lassen möchtest,

würde ich das direkt auf dem LST-Cluster machen.

Dafür ist das Environment offensichtlich gedacht.

Was ich jetzt als Nächstes anschauen würde

Mich interessiert jetzt **eine einzige Datei**:

```
text_processing_and_transformers/
```

Bitte einmal

```powershell
dir text_processing_and_transformers
```

Denn ich möchte sehen,

- welche Notebooks existieren,
- welches Notebook der Einstieg ist,
- ob irgendwo `transformer_utils.py` liegt.

Ich habe außerdem noch einen Verdacht

Das Projekt ist mittlerweile etwa **vier Jahre alt** (Python 3.9, Transformers 4.9). Wahrscheinlich funktioniert es heute **nicht mehr unverändert** mit den neuesten Hugging Face-Versionen.

Ich würde deshalb **nicht** versuchen, exakt die alte Umgebung nachzubauen. Stattdessen würde ich eine aktuelle Umgebung mit Python 3.10 oder 3.11 erstellen und nur die tatsächlich benötigten Pakete installieren. Falls beim Ausführen eines Notebooks API-Änderungen auftreten, können wir diese gezielt anpassen. Das ist meist deutlich einfacher und robuster als ein altes Environment mit veralteten Versionen zum Laufen zu zwingen.

Eine Frage noch

Möchtest du das Repository **auf deinem Windows-PC** nutzen oder **auf dem LST-Cluster**? Davon hängt ab, ob wir eine moderne lokale Umgebung erstellen oder die ursprüngliche Cluster-Umgebung möglichst originalgetreu nachbauen.

Sa., 1. Aug. um 14:46

aber mein cluster läuft göaube ich auf linux

Ja – **genau deshalb ist der Cluster wahrscheinlich der bessere Ort**, um dieses Repository auszuführen.

Die `fmri_conda_env.yml` ist eindeutig für Linux/HPC geschrieben. Das erkennt man an Paketen wie

- `gcc_linux-64`
- `binutils_linux-64`
- `mpi`

- `mpich`
- `brainiak`

und an den Slurm-Skripten im Repository (`slurm/`).

Ich würde folgendermaßen vorgehen

Option 1: Alles auf dem Cluster (meine Empfehlung)

Wenn du Zugang zum LST-Cluster hast, dann:

1. Repository auf den Cluster klonen.
2. Conda-Umgebung dort erstellen.
3. Jupyter über den Cluster starten (es gibt sogar ein `run_jupyter.sh` im Repo).
4. Die Notebooks direkt auf dem Cluster ausführen.

Das passt genau zum vorgesehenen Workflow der Autoren.

Option 2: Gemischt

- **Lokal (Windows):** Entwicklung, Lesen des Codes, kleine Tests.
- **Cluster (Linux):** Rechenintensive Analysen und alles, was fMRI oder Slurm betrifft.

Das machen viele Forschungsgruppen genau so.

Bevor du die Umgebung erstellst

Ich würde trotzdem **nicht einfach**

```bash
conda env create -f transformer_conda_env.yml
```

ausführen.

Der Grund ist, dass die Datei mit

```yaml
prefix: /Users/sumers/opt/anaconda3/envs/bert-brains
```

endet. Dieser `prefix` stammt vom Rechner eines Autors und kann zu Problemen führen.

Ich würde ihn vorher entfernen oder stattdessen den Namen der Umgebung verwenden.

Eine Sache würde ich noch prüfen

Kannst du auf dem Cluster einmal Folgendes ausführen?

```bash
conda --version
```

und

```bash
python --version
```

sowie

```bash
conda info
```

Dann sehen wir:

- ob Conda bereits installiert ist,
- welche Version verwendet wird,
- ob es Besonderheiten auf dem LST-Cluster gibt.

Danach können wir die Umgebung sauber erstellen und prüfen, ob alle Pakete installiert werden oder ob einzelne Versionen angepasst werden müssen.

```
(base) [phussein@submit ~]$ cd brains/
(base) [phussein@submit brains]$ ls
bert-brains  brains.sub
(base) [phussein@submit brains]$ cd bert-brains/
(base) [phussein@submit bert-brains]$ ls
LICENSE    analysis  data_handling       fmri_conda_env.yml
text_processing_and_transformers  visualizations
README.md  data      figure_1_github.png  slurm
transformer_conda_env.yml
(base) [phussein@submit bert-brains]$
```

ist schon alles auf dem cluster

Perfekt, das ist sogar die ideale Ausgangslage. 👍

Da das Repository bereits auf dem Cluster liegt und der Cluster Linux verwendet, würde ich **alles dort ausführen**. Die Autoren haben das Projekt offensichtlich genau dafür entwickelt (Slurm, MPI, Linux-Umgebung).

Schritt 1: Neue Conda-Umgebung anlegen

Wechsle ins Repository:

```bash
cd ~/brains/bert-brains
```

Dann versuche zuerst:

```bash
conda env create -f transformer_conda_env.yml
```

Falls eine Fehlermeldung wegen des `prefix:` am Ende der Datei erscheint, entferne einfach die letzte Zeile:

```yaml
prefix: /Users/sumers/opt/anaconda3/envs/bert-brains
```

Das kannst du z. B. mit `nano` machen:

```bash
nano transformer_conda_env.yml
```

Speichern: **Ctrl+O**, Enter, dann **Ctrl+X**.

Anschließend erneut:

```bash
conda env create -f transformer_conda_env.yml
```

Schritt 2: Aktivieren

```bash
conda activate bert-brains
```

(Die Umgebung heißt laut YAML `bert-brains`.)

Schritt 3: Prüfen

```bash
python --version
```

sollte ungefähr

```
Python 3.9.x
```

anzeigen.

Dann:

```bash
python -c "import transformers; print(transformers.__version__)"
```

und

```bash
python -c "import torch; print(torch.__version__)"
```

Falls die Installation fehlschlägt

Das würde mich **nicht überraschen**, weil die Datei mehrere Jahre alt ist. Conda findet manche alten

Builds eventuell nicht mehr.

Dann gehen wir anders vor und erstellen eine moderne Umgebung mit den tatsächlich benötigten Paketen.

Danach

Wenn die Umgebung steht, würde ich **nicht sofort** die komplette Pipeline starten.

Ich würde zuerst eines der Notebooks im Ordner

```
Nur Text

text_processing_and_transformers/
```

zum Laufen bringen. Das bestätigt, dass:

- die Umgebung funktioniert,
- BERT geladen werden kann,
- die Daten korrekt eingebunden sind.

Kannst du mir noch den Inhalt dieses Ordners zeigen?

```
Bash

ls text_processing_and_transformers
```

Dann können wir direkt entscheiden, welches Notebook oder Skript sich am besten als Einstieg eignet.

vielleicht lieber als job?

base) [phussein@submit bert-brains]$ ls
LICENSE analysis data_handling fmri_conda_env.yml
text_processing_and_transformers visualizations
README.md data figure_1_github.png slurm
transformer_conda_env.yml
(base) [phussein@submit bert-brains]$ conda env create -f
transformer_conda_env.yml
Killed

(base) [phussein@submit bert-brains]$

Ja, **das würde ich sogar bevorzugen**.

Das `Killed` kommt wahrscheinlich daher, dass der **Login-Node (`submit`)** Prozesse beendet, die zu viel RAM oder CPU verbrauchen. Das Lösen der Conda-Abhängigkeiten kann mehrere GB Speicher benötigen.

Möglichkeit 1 (empfohlen): Interaktive Session

Wenn euer Cluster interaktive Jobs unterstützt (z. B. mit `srun`), fordere zuerst Rechenressourcen an:

```bash
srun --pty --mem=8G --cpus-per-task=4 --time=01:00:00 bash
```

oder – je nach Cluster – mit `salloc`:

```bash
salloc --mem=8G --cpus-per-task=4 --time=01:00:00
```

Dann erst:

```bash
conda env create -f transformer_conda_env.yml
```

Möglichkeit 2: Als Slurm-Job

Du kannst auch ein kleines Installationsskript schreiben, z. B. `create_env.sh`:

```bash
#!/bin/bash
#SBATCH --job-name=create_env
#SBATCH --output=create_env.out
#SBATCH --error=create_env.err
#SBATCH --time=02:00:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
```

```
source ~/.bashrc
conda env create -f transformer_conda_env.yml
```

Dann:

```bash
sbatch create_env.sh
```

Bevor wir das tun, würde ich aber noch etwas prüfen

Mich würde interessieren, welche Paketverwaltung auf eurem Cluster empfohlen wird. Kannst du einmal Folgendes ausführen?

```bash
conda info
```

und

```bash
which conda
```

Außerdem:

```bash
conda env list
```

Viele HPC-Cluster empfehlen inzwischen **mamba** oder **micromamba**, weil das Auflösen der Abhängigkeiten viel schneller ist und deutlich weniger Speicher benötigt. Wenn `mamba` bereits verfügbar ist, wäre

```bash
mamba env create -f transformer_conda_env.yml
```

wahrscheinlich die beste Lösung.

(base) [phussein@submit bert-brains]$ conda info

```
     active environment : base
    active env location : /nethome/phussein/miniconda3
            shell level : 1
       user config file : /nethome/phussein/.condarc
 populated config files : /nethome/phussein/
miniconda3/.condarc
          conda version : 26.1.1
    conda-build version : not installed
         python version : 3.13.9.final.0
                 solver : libmamba (default)
       virtual packages : __archspec=1=nehalem
                          __conda=26.1.1=0
                          __glibc=2.34=0
                          __linux=5.14.0=0
                          __unix=0=0
       base environment : /nethome/phussein/miniconda3
(writable)
      conda av data dir : /nethome/phussein/miniconda3/etc/
conda
  conda av metadata url : None
           channel URLs : https://repo.anaconda.com/pkgs/main/
linux-64
                          https://repo.anaconda.com/pkgs/main/noarch
                          https://repo.anaconda.com/pkgs/r/linux-64
                          https://repo.anaconda.com/pkgs/r/noarch
          package cache : /nethome/phussein/miniconda3/pkgs
                          /nethome/phussein/.conda/pkgs
       envs directories : /nethome/phussein/miniconda3/envs
                          /nethome/phussein/.conda/envs
    temporary directory : /tmp
                platform : linux-64
            user-agent : conda/26.1.1 requests/2.32.5
CPython/3.13.9 Linux/5.14.0-611.45.1.el9_7.x86_64 almalinux/9.7
glibc/2.34 solver/libmamba conda-libmamba-solver/25.4.0
libmambapy/2.3.2 aau/0.7.3 c/. s/. e/.
                 UID:GID : 12140:12140
              netrc file : None
            offline mode : False
```

```
(base) [phussein@submit bert-brains]$ which conda
conda ()
{
    \local cmd="${1-__missing__}";
    case "$cmd" in
        activate | deactivate)
            __conda_activate "$@"
        ;;
        install | update | upgrade | remove | uninstall)
            __conda_exe "$@" || \return;
            __conda_activate reactivate
        ;;
        *)
            __conda_exe "$@"
        ;;
    esac
}
```

Das ist tatsächlich eine gute Ausgangslage.

Das Wichtigste ist:

- ✅ Du hast **Miniconda** in deinem Home-Verzeichnis.
- ✅ Der **libmamba-Solver** ist bereits aktiv (schneller als der klassische Solver).
- ✅ Du bist auf einem Linux-Cluster.
- ❌ Trotzdem wurde der Prozess mit `Killed` beendet.

Das spricht **nicht** für ein Conda-Problem, sondern dafür, dass der **Login-Knoten den Prozess wegen zu hoher Speichernutzung beendet hat**.

Als Nächstes würde ich prüfen, ob interaktive Jobs möglich sind

Kannst du einmal ausführen:

```bash
sinfo
```

und

```bash
squeue -u $USER
```

Dann sehen wir, welche Partitionen verfügbar sind und ob du interaktive Slurm-Jobs starten kannst.

Falls ihr Slurm verwendet (wovon ich ausgehe)

Dann würde ich gar nicht auf `submit` installieren, sondern z. B.:

```bash
srun --pty --mem=8G --cpus-per-task=4 --time=02:00:00 bash
```

und **erst innerhalb dieser interaktiven Shell**:

```bash
conda env create -f transformer_conda_env.yml
```

Das vermeidet genau solche `Killed`-Fehler.

Noch eine Sache fällt mir auf

Dein Conda ist **sehr neu** (26.1.1), die Environment-Datei stammt aber aus 2021. Es kann sein, dass einzelne Paketversionen inzwischen nicht mehr verfügbar sind oder Konflikte verursachen.

Deshalb würde ich die Installation zunächst etwas robuster machen:

```bash
conda env create -f transformer_conda_env.yml --prune
```

Falls sie dann wegen einzelner Pakete scheitert (und nicht wegen `Killed`), können wir die YAML gezielt modernisieren.

Ich vermute aber etwas anderes

Ich habe den Verdacht, dass ihr am LST-Cluster ohnehin **GPU- oder CPU-Compute-Nodes** für solche Arbeiten nutzen sollt und der Login-Knoten absichtlich speicherintensive Prozesse beendet.

Wenn du mir die Ausgabe von `sinfo` schickst, kann ich dir sagen, welchen `srun`- oder `sbatch`-Befehl du für euren Cluster am besten verwenden solltest.

wir verwenden condor

Ah, okay — dann ist der Ansatz mit Slurm falsch. Danke für die Korrektur. Wenn ihr **HTCondor** verwendet, passt auch dein ursprüngliches `brains.sub` ins Bild.

Das `Killed` bei

```bash
conda env create -f transformer_conda_env.yml
```

ist dann sehr wahrscheinlich passiert, weil du es auf dem **Submit-Node** (`submit`) ausgeführt hast und dort Ressourcenlimits gelten. Bei Condor sollte man rechenintensive Dinge über einen Job auf einen Worker-Node schicken.

Vorgehen mit Condor

Du kannst die Conda-Erstellung als Condor-Job laufen lassen.

Erstelle z. B. `create_env.sh`:

```bash
#!/bin/bash

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh

cd /nethome/phussein/brains/bert-brains

conda env create -f transformer_conda_env.yml
```

Dann eine Submit-Datei `create_env.sub`:

```
executable = create_env.sh
output = create_env.out
error = create_env.err
log = create_env.log

request_cpus = 4
request_memory = 8GB
+MaxRuntime = 7200

queue
```

Starten:

```bash
condor_submit create_env.sub
```

Status:

```bash
condor_q
```

Logs anschauen:

```bash
cat create_env.out
cat create_env.err
```

Aber: eine wichtige Sache vorher

Bevor du die alte Environment-Datei installierst, würde ich sie leicht anpassen.

Die Zeile am Ende:

```yaml
prefix: /Users/sumers/opt/anaconda3/envs/bert-brains
```

muss raus.

Also:

```bash
nano transformer_conda_env.yml
```

letzte Zeile löschen.

Noch besser: Teste zuerst, ob mamba verfügbar ist

Du hast bereits:

```
solver : libmamba
```

aber nicht zwingend das `mamba` Kommando.

Teste:

```bash
mamba --version
```

Falls vorhanden:

```bash
mamba env create -f transformer_conda_env.yml
```

Das ist oft deutlich schneller und speichersparender.

Eine andere Möglichkeit

Da du den Cluster wahrscheinlich für deine LST-Projekte nutzt: Es kann sein, dass bei euch Conda-Umgebungen **nicht auf Worker-Nodes erstellt werden dürfen**, weil die Home-Verzeichnisse über NFS gemountet sind und parallele Paketinstallation problematisch ist.

Dann wäre ein typischer Workflow:

1. Conda-Umgebung einmal mit einem größeren interaktiven/privilegierten Node erstellen.
2. Danach normale Condor-Jobs mit dieser Umgebung ausführen.

Kannst du einmal zeigen:

```bash
cat ~/brains/brains.sub
```

und

```bash
condor_status | head
```

Dann sehe ich, wie eure Condor-Jobs konfiguriert sind und welche Ressourcen die Worker-Nodes haben.

lieber so: #!/bin/bash

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh

cd /nethome/phussein/brains

conda env create -f transformer_conda_env.yml

Fast — aber ein kleiner Fehler ist drin: Du wechselst mit

```bash
cd /nethome/phussein/brains
```

in den **übergeordneten Ordner**. Dort liegt zwar `bert-brains`, aber die Datei

```
transformer_conda_env.yml
```

liegt in:

```
/nethome/phussein/brains/bert-brains/transformer_conda_env.yml
```

Du brauchst also:

```bash
#!/bin/bash

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh
```

```
cd /nethome/phussein/brains/bert-brains

conda env create -f transformer_conda_env.yml
```

Das ist die Version, die ich verwenden würde.

Noch zwei kleine Verbesserungen für Condor:

1. Fehler direkt sichtbar machen

```bash
#!/bin/bash
set -e

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh

cd /nethome/phussein/brains/bert-brains

conda env create -f transformer_conda_env.yml
```

`set -e` sorgt dafür, dass das Skript bei einem Fehler sofort abbricht.

---

2. Prüfen, ob das Environment schon teilweise erstellt wurde

Falls der erste Versuch auf dem Submit-Node abgebrochen wurde, kann es sein, dass Conda Reste hinterlassen hat:

```bash
conda env list
```

Falls dort schon `bert-brains` auftaucht, würde ich vor einem neuen Versuch entfernen:

```bash
conda env remove -n bert-brains
```

Danach kannst du die Condor-Submit-Datei dazu bauen.

Eine Sache noch: Ich würde **nicht** erwarten, dass diese Installation in wenigen Minuten fertig ist. Das Environment enthält u. a.:

- PyTorch 1.9
- spaCy 2.3
- Jupyter
- Transformers 4.9

und viele alte Pakete. Gib dem Job ruhig z. B. 30–60 Minuten Laufzeit.

Wenn du mir noch dein `brains.sub` zeigst, kann ich es direkt passend für diesen Installationsjob umschreiben.

```
##################################################
####################
# HTCondor submit description file
# Everything with a leading # is a comment

initialdir              = /nethome/phussein/brains
executable              = create_env.sh
output                  = /scratch/phussein/logs/brains/create_env.sh.
$(ClusterId).$(Year)_$(Month)_$(Day)_$(SUBMIT_TIME).out
error                   = /scratch/phussein/logs/brains/create_env.sh.
$(ClusterId).$(Year)_$(Month)_$(Day)_$(SUBMIT_TIME).err
log                     = /scratch/phussein/logs/brains/create_env.sh.
$(ClusterId).$(Year)_$(Month)_$(Day)_$(SUBMIT_TIME).log
request_CPUs            = 1
request_memory          = 64G
request_GPUs            = 1
requirements            = (GPUs_Capability
>=8.0)&&(GPUs_GlobalMemoryMb > 60000)
#environment = WANDB_API_KEY=46736878....
getenv = True
queue 1
##################################################
####################
```

Danke, das hilft. Für **diesen speziellen Job (Conda-Environment erstellen)** würde ich deine `brains.sub` etwas ändern.

Deine aktuelle Datei ist für einen **GPU-Job** gedacht:

```
Nur Text

request_GPUs            = 1
requirements            = (GPUs_Capability >=8.0)&&(GPUs_GlobalMemoryMb > 60000)
```

Das brauchst du für Conda **überhaupt nicht**. Du blockierst damit unnötig eine H100/A100-Klasse GPU, obwohl nur Pakete installiert werden.

Ich würde eine separate Datei machen, z. B. `create_env.sub`:

```
Nur Text

######################################################################
# HTCondor submit file for creating conda environment

initialdir              = /nethome/phussein/brains/bert-brains
executable              = create_env.sh

output                  = /scratch/phussein/logs/brains/create_env.$(ClusterId).out
error                   = /scratch/phussein/logs/brains/create_env.$(ClusterId).err
log                     = /scratch/phussein/logs/brains/create_env.$(ClusterId).log

request_CPUs            = 4
request_memory          = 16G

getenv                  = True

queue 1
######################################################################
```

Und dein `create_env.sh`:

```
Bash

#!/bin/bash
set -e

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh

cd /nethome/phussein/brains/bert-brains

conda env create -f transformer_conda_env.yml
```

Dann:

```bash
condor_submit create_env.sub
```

Status:

```bash
condor_q
```

Noch ein wichtiger Punkt

Ich sehe gerade:

```
initialdir = /nethome/phussein/brains
```

in deinem bisherigen Submit-File.

Dann wird das Skript aus diesem Verzeichnis gestartet. Das ist okay, weil du im Skript sowieso `cd` machst. Trotzdem ist es sauberer, `initialdir` direkt auf das Repo zu setzen.

Falls der Job wieder "Killed" wird

Dann würde ich als Erstes prüfen:

```bash
cat /scratch/phussein/logs/brains/create_env.*.err
```

und

```bash
cat /scratch/phussein/logs/brains/create_env.*.out
```

Eine weitere Sache: Ich würde **vor dem Abschicken** noch einmal die YAML-Datei auf den `prefix` prüfen. Der Eintrag

```yaml
prefix: /Users/sumers/opt/anaconda3/envs/bert-brains
```

muss raus, sonst versucht Conda eventuell, eine macOS-Umgebung an einem falschen Ort anzulegen.

Du kannst prüfen:

```bash
tail transformer_conda_env.yml
```

und falls nötig:

```bash
sed -i '/^prefix:/d' transformer_conda_env.yml
```

Danach sollte der Condor-Job deutlich sauberer laufen.

> warum haben wir das transformer env erstellt und nicht das andere fmri conda??

Gute Frage — und tatsächlich ist das eine wichtige Unterscheidung. Wir haben **bewusst zuerst `transformer_conda_env.yml` genommen**, weil ich aus unserem bisherigen Ziel geschlossen habe, dass du zuerst den **Transformer-Teil** des Projekts zum Laufen bringen möchtest. Aber wenn dein eigentliches Ziel die **fMRI-Encoding-Modelle** aus dem Paper sind, wäre das andere Environment relevant.

Die beiden Environments sind für **verschiedene Teile der Pipeline**:

`transformer_conda_env.yml` → BERT / Repräsentationen

Das ist für:

```
text_processing_and_transformers/
```

also z. B.:

- `transformer-representations.ipynb`
- BERT laden
- Hidden States extrahieren
- Embeddings berechnen
- Transformation representations erzeugen

Typische Pakete:

- `transformers`
- `torch`
- `spacy`
- `jupyter`

Pipeline:

```
Text/Transkript
      ↓
BERT
      ↓
Embeddings + Layer/Head representations
```

`fmri_conda_env.yml` → Encoding Models / Gehirndaten

Das ist für:

```
analysis/
```

also:

- `banded_ridge_regression.py`
- `compute_isc.py`
- `calculate_bootstrap_pvalue.py`
- fMRI-Auswertung

Typische Pakete:

- `brainiak`
- `nibabel`
- `scikit-learn`
- `mpi4py`
- `nitime`

Pipeline:

```
BERT-Repräsentationen
        +
fMRI-Daten
        ↓
Banded Ridge Regression
        ↓
Vorhersage Gehirnaktivität
```

Für die vollständige Reproduktion des Papers brauchst du also beide

Die tatsächliche Reihenfolge ist:

```
1. transformer environment
        ↓
2. BERT representations erzeugen
        ↓
3. fmri environment
        ↓
4. Encoding models laufen lassen
```

Warum ich nicht mit `fmri_conda_env.yml` gestartet habe

Weil deine Datenstruktur zunächst so aussah, als wären schon viele Textdaten vorhanden:

```
data/
 ├──21st_year/
 │    ├──tr_tokens.csv
 │    ├──ling_features.csv
 │
```

```
└──stimuli/
    └──align.csv
```

Das sind genau die Inputs für den Transformer-Teil.

Außerdem hattest du vorher erwähnt, dass dich besonders die **Transformer-Features, Attention und Brain-Encoding-Zusammenhänge** interessieren. Deshalb erschien der Transformer-Teil als sinnvoller Einstieg.

Aber: Wenn dein Ziel C aus deiner früheren Liste war:

> "Die fMRI-Encoding-Modelle ausführen"

dann brauchen wir tatsächlich als nächsten Schritt:

```bash
conda env create -f fmri_conda_env.yml
```

und nicht nur `transformer_conda_env.yml`.

Ich würde jetzt so vorgehen:

1. `transformer` Environment erstellen (weil wir damit die BERT-Features erzeugen können).
2. Danach `fmri` Environment erstellen.
3. Prüfen, welche Daten für die fMRI-Analyse fehlen.

Eine Sache würde ich aber vorher noch prüfen: Im Ordner `data/` sehe ich bisher **keine fMRI-Dateien**. Kannst du einmal zeigen:

```bash
cd data
find . -type f | head -50
```

Dann sehen wir, ob die fMRI-Daten überhaupt im Repo liegen oder ob nur die Text-/Stimulusdaten vorhanden sind.

wir sollen uns ja für dieses paper/code repo ein projekt mit den folgenden anforderungen überlegen:

Final Papers (for the 7CP version)

Note: We will discuss this in the first meeting. Requirements may be changed based on popular demand.

Term papers will be about a small independent project.

Possible topics:

- Make a computational simulation of some aspect of brain function
- Replicate a research paper, varying some aspect that wasn't investigated in the original paper
- Investigate whether some ML model (e.g. of language or vision) is or isn't brain-like in some respect

The report is expected to contain a brief literature review, motivation of your study, a description of what you did, and report of your results.

The report should have at most 8 pages of main report, plus unlimited appendix, in the NeurIPS style format. The main report should be self-contained, but you can use the appendix to report prompts, further analyses, or other material.

If you use AI tools in your project in any form, you need to explicitly declare and document this in an appendix.

The report should be uploaded via CMS. The due date is October 31, 2026, 23:59.

Everyone is expected to report on their project idea in the project ideas session, and to participate in discussion to give feedback to other students' ideas. Students may prepare a short slide deck on their idea. This will not be graded; the project ideas session is intended to help improve and finetune project ideas.

Ah, das ändert die Perspektive deutlich. Dann geht es **nicht darum, einfach das gesamte Paper zu reproduzieren**, sondern ihr sollt auf Basis des Repos ein **kleines eigenes Forschungsprojekt** machen. In dem Fall ist es sogar sinnvoll, nicht sofort die komplette Pipeline mit fMRI-Daten aufzusetzen.

Das Repo ist eher eine **Ausgangsbasis**.

Die drei möglichen Richtungen aus der Aufgabenstellung passen sehr gut zu diesem Paper:

Was das Paper eigentlich liefert

Das Paper untersucht:

> Sind interne Repräsentationen von BERT ähnlich spezialisiert wie Hirnareale beim Sprachverstehen?

Die Grundidee:

```
Narrative stimulus
        ↓
BERT
        ↓
Layer / Head representations
        ↓
Encoding model
        ↓
fMRI prediction
```

Dann schaut man:

- Welche Layer sagen welche Hirnregionen vorher?
- Welche Attention Heads sind spezialisiert?
- Gibt es funktionale Spezialisierung wie im Gehirn?

Mögliche Projektideen auf Basis dieses Repos

Ich würde eher ein **kleines Variationsprojekt** machen, nicht versuchen, das Paper komplett nachzubauen.

Ein paar Ideen:

Idee 1 (sehr passend): Vergleich verschiedener Sprachmodelle

Fragestellung:

> Does functional specialization emerge similarly in newer language models?

Das wäre direkt Kategorie:

> Investigate whether some ML model is brain-like in some respect

Du könntest BERT aus dem Paper vergleichen mit:

- BERT-base (Original)
- GPT-2
- Llama-3
- Gemma
- Qwen

Untersuchung:

- Welche Layer korrelieren mit welchen linguistischen Features?
- Welche Layer zeigen ähnliche Repräsentationsmuster?

Beispiel:

```
BERT layer 1-12
        vs
Llama layers
        ↓
Representational Similarity Analysis
```

Vorteil:

- Du brauchst nicht zwingend neue fMRI-Daten.
- Du kannst vorhandene Stimuli verwenden.
- Sehr machbar in 8 Seiten.

Idee 2 (mein Favorit): Attention Heads vs. Linguistische Funktionen

Das passt extrem gut zu deinem bisherigen Interesse an Attention.

Fragestellung:

> Are individual attention heads specialized for linguistic functions in modern LLMs?

Das Originalpaper macht etwas Ähnliches mit BERT-Heads.

Du könntest:

1. BERT/GPT-2/Llama nehmen
2. Attention Heads extrahieren
3. testen:
    - Syntax?
    - Coreference?
    - Long-range dependencies?
    - Semantic similarity?

Beispiel:

```
Head 3.7

        "The dog that chased the cat was tired"

attends strongly:
dog → chased
```

Dann vergleichen:

- alte Modelle vs neue Modelle

Idee 3: Brain-like scaling

Sehr interessante Variation:

> Does increasing model size make representations more brain-like?

Vergleich:

```
BERT-base
   |
GPT-2 small
   |
GPT-2 medium
   |
```

```
Llama 8B
```

Messung:

Representational similarity:

```
model representations
          |
          |
          ↓
similarity to brain/fMRI/RSA
```

Das wäre sehr aktuell.

Idee 4: Layerwise specialization

Das Originalpaper zeigt:

- frühe Layer → einfache linguistische Features
- spätere Layer → komplexere Semantik

Du könntest prüfen:

> Does instruction tuning change layer specialization?

Vergleich:

```
Llama-3 base
        vs
Llama-3 instruct
```

Frage:

Verändert Instruction Tuning die "brain-like" Organisation?

Sehr spannend.

Idee 5: Negation / Compositionality (würde zu deinen

bisherigen Projekten passen)

Du könntest untersuchen:

> Are transformer representations of negation reflected in brain-like encoding patterns?

Beispiel:

Stimuli:

```
The dog is alive.
The dog is not alive.
```

Dann:

- BERT representations
- LLM representations
- Similarity analysis

Meine Einschätzung für dich

Basierend darauf, was du bisher gemacht hast (Attention, EAP, Copy circuits, ToM, fMRI Paper), würde ich nicht versuchen, das komplette Encoding Model nachzubauen.

Ich würde eher eine dieser drei Richtungen wählen:

⭐ Option A: "How brain-like are modern LLM representations?"

Vergleich:
BERT vs Llama/Gemma/Qwen

Sehr gut für 8 Seiten.

⭐ Option B: "Functional specialization of attention heads across models"

Passt sehr gut zu deinem bisherigen Attention-Wissen.

⭐ Option C: "Does instruction tuning change brain-like representations?"

Aktuell und eigenständig.

Für das Repo bedeutet das:

Du brauchst wahrscheinlich nur:

```
text_processing_and_transformers/
```

und eventuell Teile aus:

```
visualizations/
```

Nicht unbedingt:

```
fmri_conda_env.yml
analysis/
```

außer du möchtest wirklich die fMRI-Encoding-Modelle reproduzieren.

Eine Frage wäre jetzt entscheidend:

Habt ihr im Kurs Zugriff auf die fMRI-Daten bzw. dürfen/sollen die verwendet werden?
Denn davon hängt ab, ob wir ein Projekt mit echter Gehirnvorhersage machen oder eine "brain-like representation"-Analyse ohne neue fMRI-Daten planen.

mein favorit ist, die selbe pipeline mit einem neueren modell durchzulafen. motivation: BERT repräsentationen haben bekannt schwächen wie negation etc, also geht mit den embeddings information verloren. daher neues modell verwenden um rauszufinden, ob dessen embeddings bessere predictors für brain activity sind.

Bonus: functional specialization für neues modell

Das ist tatsächlich eine **sehr gute Projektidee** und passt sehr sauber zu den Anforderungen. Sie ist auch näher an einem echten Forschungsbeitrag als eine reine Reproduktion.

Ich würde die Idee nur etwas präzisieren: Nicht unbedingt behaupten, dass "BERT-Embeddings Information verlieren" (das ist schwer direkt zu zeigen), sondern eher:

> BERT wurde auf Masked Language Modeling trainiert und zeigt bekannte Limitationen bei bestimmten linguistischen und kompositionalen Fähigkeiten (z. B. Negation, syntaktische Generalisierung). Moderne autoregressive LLMs oder instruction-tuned Modelle zeigen in vielen Benchmarks verbesserte Sprachverständnisfähigkeiten. Es ist daher offen, ob diese verbesserten Fähigkeiten auch in einer stärkeren Übereinstimmung zwischen Modellrepräsentationen und menschlicher Sprachverarbeitung resultieren.

Das ist eine sehr gute Motivation.

Projektidee (geschärft)

Arbeitstitel

From BERT to Modern LLMs: Do Improved Language Models Better Predict Human Brain Activity?

oder etwas spezifischer:

Are Modern Transformer Representations More Brain-like Than BERT?

Forschungsfrage

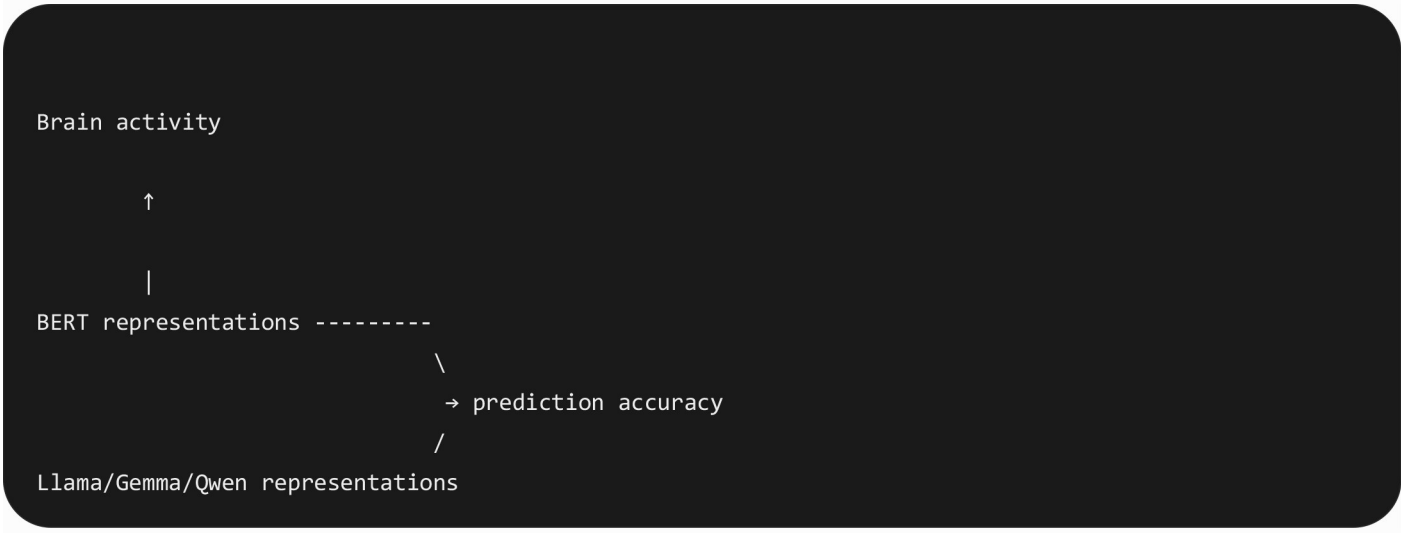
Die zentrale Frage:

> Do representations from newer language models provide better predictions of human brain activity than BERT representations?

Hypothesen

H1: Better predictive performance

Modernere Modelle sollten höhere Encoding-Performance erreichen:



Messung:

- correlation between predicted and actual fMRI
- R^2
- noise-normalized accuracy

H2 (Bonus): Functional specialization

Das Originalpaper zeigt:

BERT:

```
early layers:
    syntax / word-level

middle layers:
    linguistic features

late layers:
    semantics
```

Frage:

> Does a newer model show similar or different specialization?

Analyse:

Layer-wise:

```
Layer 1
Layer 5
Layer 10
Layer 20
...
```

und Head-wise:

```
Attention head
      ↓
brain region prediction
```

Methodik

Du könntest die Pipeline vom Paper übernehmen:

Schritt 1: Stimuli

Gleiches Narratives Dataset:

```
spoken narratives
       |
       ↓
TR-aligned tokens
```

Schritt 2: Representations extrahieren

Original:

```
BERT-base
12 layers
768 dim
```

Dein Modell:

z.B.

Option A (realistisch):

Meta Platforms Llama-3.1-8B

oder

Option B (einfacher):

Google DeepMind Gemma-2

oder

Alibaba Cloud Qwen

Warum eher kleinere Modelle:

- fMRI-Encoding braucht viele Features
- Memory

Schritt 3: Encoding Model

Original:

Banded Ridge Regression:

```
model representations
          |
          ↓
ridge regression
          |
          ↓
predicted fMRI
```

Vergleich:

Model	Brain prediction
BERT	baseline
New LLM	?

Bonus: Functional specialization

Das wäre sehr cool:

Nicht nur:

"Wer gewinnt?"

sondern:

"Warum?"

Zum Beispiel:

Layer preference

Original:

```
BERT layer
   |
   ↓
brain parcel
```

Deine Frage:

```
Llama layer
   |
   ↓
brain parcel
```

Vergleich:

- Verschiebt sich Spezialisierung nach hinten?

- Werden spätere Layer wichtiger?

Warum diese Idee stark ist

Sie erfüllt alle drei möglichen Themen:

✅ Replicate paper, vary something not investigated:

> Replace BERT with a modern LLM

✅ Investigate whether ML model is brain-like:

> Compare representational similarity to brain activity

✅ Computational simulation of brain function:

> Modeling neural encoding

Risiko / Machbarkeit

Der einzige kritische Punkt ist Rechenaufwand.

Das Paper nutzt:

- BERT
- 12 layers
- relativ kleine Features

Ein Llama-3 8B Modell wäre deutlich größer.

Ich würde deshalb eher planen:

Minimal viable project:

- BERT baseline
- Gemma-2-2B oder Qwen2.5-3B
- gleiche Stimuli
- gleiche Encoding Pipeline

Bonus:

- Layerwise analysis

Was ich für die Projektpräsentation vorschlagen würde

Eine Folie:

Motivation

> Transformer models have become increasingly capable at language understanding, but it remains unclear whether improved linguistic abilities correspond to more brain-like internal representations.

Research question

> Do modern LLM representations predict human brain activity better than BERT representations?

Approach

```
Narratives dataset
        ↓
BERT vs Modern LLM
        ↓
Layer representations
        ↓
Banded ridge encoding
        ↓
Compare brain prediction
```

Ich würde diese Idee definitiv weiterverfolgen. Ein nächster sinnvoller Schritt wäre jetzt zu prüfen, **wie stark die bestehende Pipeline an BERT gekoppelt ist** (also ob man nur das Modell austauschen muss oder ob die Feature-Extraktion angepasst werden muss). Dafür wäre die Datei `transformer-representations.ipynb` bzw. `transformer-utils.py` der wichtigste nächste Blick.

Lieber Michael,

Ich habe im Seminar das Paper Shared functional specialization

in transformer-based language models and the human brain (Kumar et al. 2024) vorgestellt.
Die im Paper verwendeten Modelle sind BERT und GPT-2. Es ist bekannt, dass die latenten Repräsentationen dieser Modelle oft nicht mit der Semantik übereinstimmen, dass beispielsweise Negation nicht korrekt verarbeitet wird.
Dies motiviert meine Forschungsfrage, ob Repräsentationen (embeddings und transformations) aus neueren Modellen (z.B. LLama?) besser geeignet sind, um die Gehirnaktivität vorherzusagen.
Da die functional specialization neuerer Modelle etwas schlechter erforscht ist, kann ich mir vorstellen, dass dieser Aspekt wenn überhaupt einen kleineren Teil des Projekts ausmacht.

Ich bin über Feedback zu meinem Vorschlag dankbar. Falls Sie wollen, können wir uns auch persönlich besprechen.
Freundliche Grüße
Paulina

Ich würde die Mail etwas glätten und die Forschungsfrage etwas vorsichtiger formulieren. Vor allem würde ich "Negation wird nicht korrekt verarbeitet" nicht zu absolut schreiben, sondern als Motivation über bekannte Limitationen. Außerdem würde ich "LLama" zu "Llama" korrigieren.

E-Mail

E-Mail verbinden

Empfänger:in

Vorschlag für das Projekt im Seminar

Lieber Michael,

ich habe im Seminar das Paper *Shared functional specialization in transformer-based language models and the human brain* (Kumar et al., 2024) vorgestellt.

Die im Paper untersuchten Modelle sind BERT und GPT-2. Gleichzeitig zeigen Transformer-Modelle in bestimmten sprachlichen Bereichen bekannte Limitationen, beispielsweise bei der Verarbeitung von Negation oder anderen kompositionalen Bedeutungsaspekten. Dies motiviert meine Forschungsfrage, ob Repräsentationen (Embeddings und Transformationsrepräsentationen) neuerer Sprachmodelle, beispielsweise aus der Llama-Familie, besser geeignet sind, um menschliche Gehirnaktivität vorherzusagen.

Als Erweiterung könnte ich mir außerdem vorstellen, die funktionale Spezialisierung neuerer Modelle zu untersuchen. Da dieser Aspekt bei aktuellen Modellen bisher weniger erforscht ist, würde dieser Teil vermutlich einen kleineren Anteil des Projekts ausmachen und eher als zusätzliche Analyse dienen.

Ich würde mich sehr über Feedback zu diesem Vorschlag freuen. Falls Sie möchten, können wir die Idee auch gerne persönlich besprechen.

Freundliche Grüße
Paulina

Ich würde inhaltlich noch eine kleine strategische Änderung empfehlen: **Nicht "besser geeignet" als feste Annahme formulieren**, sondern als offene Forschungsfrage. Dein Projekt ist wissenschaftlich stärker, wenn auch das Ergebnis möglich ist, dass ein neueres Modell *nicht* besser brain-like ist. Die Formulierung oben lässt diese Möglichkeit offen.

(base) [phussein@login brains]$ cat create_env.102580.err

PackagesNotFoundError: The following packages are not available from current channels:

- anyio==3.3.0=py39h6e9494a_0

Current channels:

- https://conda.anaconda.org/pytorch
- https://conda.anaconda.org/conda-forge
- https://repo.anaconda.com/pkgs/main
- https://repo.anaconda.com/pkgs/r

To search for alternate channels that may provide the conda package you're
looking for, navigate to

https://anaconda.org

and use the search bar at the top of the page.

(base) [phussein@login brains]$ cat create_env.102580.out
2 channel Terms of Service accepted
Retrieving notices: done
Channels:
 - pytorch
 - conda-forge
 - defaults
Platform: linux-64
Collecting package metadata (repodata.json): done
Solving environment: failed
(base) [phussein@login brains]$

Das ist ein typischer Fehler bei alten Conda-Environment-Dateien. Die gute Nachricht: Dein Condor-Setup funktioniert jetzt, der Job läuft durch. Das Problem ist nur die **veraltete YAML-Datei**.

Der Grund:

Die Datei wurde mit einer **alten Conda-Version (ca. 2021)** exportiert und enthält exakte Build-Hashes:

```yaml
anyio=3.3.0=py39h6e9494a_0
```

Der letzte Teil:

```
py39h6e9494a_0
```

ist ein spezifischer Build für den damaligen Rechner. Dieser Build existiert in den aktuellen Conda-Channels nicht mehr.

Lösung: YAML "entfrieren"

Wir entfernen die Build-Nummern, behalten aber die Versionsnummern.

Aus:

```yaml
```

```
- anyio=3.3.0=py39h6e9494a_0
```

wird:

```yaml
- anyio=3.3.0
```

Das muss für alle Conda-Pakete passieren.

Einfachste Variante:

Erstelle eine Kopie:

```bash
cp transformer_conda_env.yml transformer_conda_env_clean.yml
```

Dann:

```bash
sed -i -E 's/([a-zA-Z0-9_-]+)=([0-9]+\.[0-9]+(\.[0-9]+)?)(=.*)?$/\1=\2/' transformer_conda_env_clean.ym
```

Danach prüfen:

```bash
grep "anyio" transformer_conda_env_clean.yml
```

sollte jetzt zeigen:

```
- anyio=3.3.0
```

Danach neu versuchen

Falls die alte Umgebung teilweise angelegt wurde:

```bash
```

```
conda env remove -n bert-brains
```

Dann:

```bash
conda env create -f transformer_conda_env_clean.yml
```

Aber: Ich würde noch eine zweite Änderung machen

Die Umgebung ist extrem alt:

- Python 3.9.6
- PyTorch 1.9
- Transformers 4.9.1
- spaCy 2.3.5

Das kann trotzdem funktionieren, aber gerade für dein eigentliches Projekt (neues Modell wie Llama/Gemma/Qwen) wirst du vermutlich sowieso modernere Transformers brauchen.

Deshalb würde ich fast eher eine **neue schlanke Umgebung** erstellen:

```yaml
name: bert-brains-modern

channels:
  - pytorch
  - nvidia
  - conda-forge

dependencies:
  - python=3.10
  - pytorch
  - torchvision
  - torchaudio
  - numpy
  - pandas
  - scipy
  - scikit-learn
  - jupyter
  - matplotlib
  - seaborn
  - pip
```

Das wäre für dein geplantes Projekt (BERT vs. Llama) wahrscheinlich sogar besser.

Eine Frage vor dem nächsten Schritt:
Willst du **erst die originale Paper-Pipeline exakt reproduzieren** (also BERT/GPT-2 wie im Paper) oder direkt eine **angepasste Umgebung für dein Projekt BERT vs. modernes LLM** bauen?

Davon hängt ab, ob wir die alte YAML reparieren oder eine neue Umgebung machen.

erst mal exakt reproduzieren

Okay, dann würde ich tatsächlich **die originale Pipeline möglichst nah am Paper reproduzieren**. Danach können wir eine zweite, modernere Umgebung für dein eigenes Experiment bauen.

Für die Reproduktion sollten wir also bei `transformer_conda_env.yml` bleiben — aber die YAML-Datei muss an heutige Conda angepasst werden.

Der Fehler zeigt nur, dass die **Build-Informationen veraltet** sind, nicht dass die Pakete grundsätzlich fehlen.

Schritt 1: Kopie der YAML erstellen

Bleib im Repo:

```bash
cd ~/brains/bert-brains
```

Dann:

```bash
cp transformer_conda_env.yml transformer_conda_env_clean.yml
```

Schritt 2: Build-Hashes entfernen

Die alte Datei enthält Zeilen wie:

```yaml
- anyio=3.3.0=py39h6e9494a_0
- numpy=1.20.3=py39h4b4dc7a_0
- pytorch=1.9.0=py3.9_0
```

Wir wollen:

```yaml
- anyio=3.3.0
- numpy=1.20.3
- pytorch=1.9.0
```

Auf Linux:

```bash
sed -i -E 's/^(\s*-\s*[A-Za-z0-9_-]+=[0-9]+\.[0-9]+(\.[0-9]+)?)(=.*)?$/\1/' transformer_conda_env_clean
```

Danach prüfen:

```bash
grep anyio transformer_conda_env_clean.yml
```

Schritt 3: Prefix entfernen

Am Ende steht:

```yaml
prefix: /Users/sumers/opt/anaconda3/envs/bert-brains
```

Das muss weg:

```bash
sed -i '/^prefix:/d' transformer_conda_env_clean.yml
```

Kontrolle:

```bash
tail transformer_conda_env_clean.yml
```

Schritt 4: Alte fehlgeschlagene Umgebung entfernen

Prüfen:

```bash
conda env list
```

Falls `bert-brains` existiert:

```bash
conda env remove -n bert-brains
```

Schritt 5: Erneut als Condor-Job starten

Dein `create_env.sh` dann ändern auf:

```bash
#!/bin/bash
set -e

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh

cd /nethome/phussein/brains/bert-brains

conda env create -f transformer_conda_env_clean.yml
```

Dann wieder:

```bash
condor_submit create_env.sub
```

Was ich danach erwarten würde

Es kann sein, dass als nächstes Fehler kommen wie:

- `PackagesNotFoundError` für alte Pakete
- Konflikte zwischen `spacy 2.3.5` und aktuellem Python
- Probleme mit `pytorch=1.9`

Das wäre normal. Dann reparieren wir Schritt für Schritt.

Noch eine Anmerkung: Für die **exakte Reproduktion** brauchen wir danach vermutlich auch noch das zweite Environment:

```
Nur Text

fmri_conda_env.yml
```

weil die komplette Paper-Pipeline aus zwei Teilen besteht:

1. `transformer_conda_env`
   → BERT/GPT-2 Repräsentationen erzeugen
2. `fmri_conda_env`
   → Encoding Modelle / Gehirnvorhersage

Aber ich würde jetzt zuerst das Transformer-Environment sauber bekommen.

(base) [phussein@submit bert-brains]$ grep anyio transformer_conda_env_clean.yml
  - anyio=3.3.0
(base) [phussein@submit bert-brains]$ sed -i '/^prefix:/d' transformer_conda_env_clean.yml
(base) [phussein@submit bert-brains]$ tail transformer_conda_env_clean.yml
    - filelock==3.0.12
    - huggingface-hub==0.0.12
    - joblib==1.0.1
    - pyyaml==5.4.1
    - regex==2021.8.3
    - sacremoses==0.0.45
    - sentencepiece==0.1.96
    - tokenizers==0.10.3
    - tqdm==4.62.0
    - transformers==4.9.1
(base) [phussein@submit bert-brains]$ conda env list
Killed
(base) [phussein@submit bert-brains]$ conda env remove -n bert-brains
2 channel Terms of Service accepted

EnvironmentLocationNotFound: Not a conda environment: /nethome/phussein/miniconda3/envs/bert-brains

(base) [phussein@submit bert-brains]$

Sehr gut, die YAML ist jetzt sauberer. 👍

Der neue interessante Punkt ist:

```bash
conda env list
Killed
```

Das ist **wieder derselbe Login-Node-Effekt**. Selbst `conda env list` wurde vom System gekillt, obwohl der Befehl eigentlich kaum Ressourcen braucht. Das deutet darauf hin, dass entweder:

1. der Login-Node gerade extrem restriktive Limits hat, **oder**
2. Conda versucht beim Start viel zu initialisieren und wird dort beendet.

Dass

```bash
conda env remove -n bert-brains
```

funktioniert und sagt:

```
EnvironmentLocationNotFound: Not a conda environment: /nethome/phussein/miniconda3/envs/bert-brains
```

ist sogar gut: Es gibt **keine halb fertige Umgebung**, die wir löschen müssten.

Jetzt würde ich den Condor-Job erneut starten

Ändere dein `create_env.sh` auf:

```bash
#!/bin/bash
set -e
```

```
source /nethome/phussein/miniconda3/etc/profile.d/conda.sh

cd /nethome/phussein/brains/bert-brains

conda env create -f transformer_conda_env_clean.yml
```

Dann:

```bash
condor_submit create_env.sub
```

Aber vorher noch eine kleine Änderung am Submit-File

Für Conda-Installation würde ich die Ressourcen etwas erhöhen:

```
request_CPUs = 4
request_memory = 32G
```

16 GB könnten knapp werden, wenn Conda den Solver laufen lässt.

Keine GPU:

entfernen:

```
request_GPUs
requirements
```

Nach dem Start

Schau nicht mit `conda env list` auf dem Login-Node, sondern im Condor-Log.

Also:

```bash
condor_q
```

Wenn fertig:

```bash
cat /scratch/phussein/logs/brains/create_env.*.out
cat /scratch/phussein/logs/brains/create_env.*.err
```

Falls es jetzt wieder bei `Solving environment` scheitert, ist der nächste wahrscheinlich auftretende Fehler nicht mehr der Build-Hash, sondern ein echter Versionskonflikt.

Ein Kandidat ist:

```yaml
- tokenizers==0.10.3
- transformers==4.9.1
```

weil diese Kombination alte Rust-Builds benötigt.

Aber erst mal schauen, was Conda auf dem Worker sagt.

(base) [phussein@login brains]$ cat create_env.102593.err

PackagesNotFoundError: The following packages are not available from current channels:

- appnope=0.1.2

Current channels:

- https://conda.anaconda.org/pytorch
- https://conda.anaconda.org/conda-forge
- https://repo.anaconda.com/pkgs/main
- https://repo.anaconda.com/pkgs/r

To search for alternate channels that may provide the conda package you're
looking for, navigate to

    https://anaconda.org

and use the search bar at the top of the page.

(base) [phussein@login brains]$ cat create_env.102593.out
2 channel Terms of Service accepted
Retrieving notices: done
Channels:
 - pytorch
 - conda-forge
 - defaults
Platform: linux-64
Collecting package metadata (repodata.json): done
Solving environment: failed
(base) [phussein@login brains]$

Das ist jetzt ein **anderer, aber erwartbarer Fehler**. Wir sind einen Schritt weiter: Conda kann die YAML lesen, aber einige Pakete sind nicht mehr verfügbar.

Der konkrete Fall:

```
Nur Text

PackagesNotFoundError:

- appnope=0.1.2
```

`appnope` ist ein macOS-spezifisches Paket. Es war in der originalen Umgebung des Autors enthalten:

```
YAML

- appnope=0.1.2=py39hecd8cb5_1001
```

Das zeigt auch, dass die `transformer_conda_env.yml` ursprünglich **von einem Mac exportiert wurde** (`/Users/sumers/...` im Prefix). Für Linux brauchen wir diese Pakete nicht.

Wir müssen also die YAML noch etwas "linuxfähig" machen.

---

1. Alle macOS-only Pakete entfernen

In `transformer_conda_env_clean.yml` entfernen:

```yaml
- appnope=...
- libcxx=...
- libgfortran=...
- llvm-openmp=...
```

Insbesondere `appnope` ist der aktuelle Blocker.

Du kannst das automatisiert machen:

```bash
sed -i '/appnope/d' transformer_conda_env_clean.yml
```

---

2. Wahrscheinlich kommen noch weitere Mac-Pakete

Ich würde direkt nach weiteren offensichtlichen Mac-Paketen suchen:

```bash
grep -E "appnope|libcxx|llvm-openmp|osx|macos" transformer_conda_env_clean.yml
```

Alles, was eindeutig `osx` oder macOS betrifft, raus.

---

3. Erneut starten

Danach:

```bash
condor_submit create_env.sub
```

Was vermutlich als nächstes passiert

Die YAML ist jetzt schon deutlich näher an Linux. Es kann aber noch weitere alte Pakete geben, z. B.:

- `pyqt=5.9.2`
- `spacy=2.3.5`
- `pytorch=1.9.0`
- `tokenizers=0.10.3`

Falls Conda wieder mit `PackagesNotFoundError` kommt, entfernen wir weiter die **Build-/OS-Abhängigkeiten**.

Eine kleine Anmerkung: Das bestätigt übrigens, dass der Cluster sauber läuft. Der erste Fehler war nicht wegen Condor oder Ressourcen, sondern die Environment-Datei war schlicht ein **Mac-Export aus 2021**. Wir portieren sie gerade auf Linux.

(base) [phussein@submit bert-brains]$ grep -E "appnope|libcxx|llvm-openmp|osx|macos" transformer_conda_env_clean.yml
  - appnope=0.1.2
  - libcxx=10.0.0
  - llvm-openmp=10.0.0

Ja, genau diese drei sind die nächsten Kandidaten. Sie sind ebenfalls **macOS-spezifisch** und sollten aus der Linux-Umgebung raus.

- `appnope` → macOS-App-Nap-Unterstützung, nicht benötigt
- `libcxx` → LLVM C++ Standardbibliothek von macOS
- `llvm-openmp` → macOS/OpenMP-Build

Entferne sie:

```bash
sed -i '/appnope/d' transformer_conda_env_clean.yml
sed -i '/libcxx/d' transformer_conda_env_clean.yml
sed -i '/llvm-openmp/d' transformer_conda_env_clean.yml
```

Danach prüfen:

```bash
grep -E "appnope|libcxx|llvm-openmp|osx|macos" transformer_conda_env_clean.yml
```

Die Ausgabe sollte leer sein.

Dann wieder als Condor-Job starten:

```bash
condor_submit create_env.sub
```

Ich würde außerdem noch eine weitere Stelle prüfen, weil ich schon sehe, dass die Datei ursprünglich von einem Mac kommt:

```bash
grep -E "hdf|darwin|osx|macos" transformer_conda_env_clean.yml
```

Falls da nichts kommt, sind wir vermutlich beim nächsten echten Linux-Kompatibilitätsproblem.

Der Ablauf ist jetzt:

1. Build-Hashes entfernt ✅
2. falscher Mac-Prefix entfernt ✅
3. Mac-Pakete entfernt 🔄
4. Conda-Solver laufen lassen
5. verbleibende Paketkonflikte lösen

Das ist bei alten Conda-YAMLs von Forschungsrepos ziemlich normal.

rep: transformer_conda_env_clean.yml: No such file or directory
(base) [phussein@submit brains]$ cd bert-brains/
(base) [phussein@submit bert-brains]$ grep -E "appnope|libcxx|llvm-openmp|osx|macos" transformer_conda_env_clean.yml
  - appnope=0.1.2
  - libcxx=10.0.0
  - llvm-openmp=10.0.0
(base) [phussein@submit bert-brains]$ sed -i '/appnope/d' transformer_conda_env_clean.yml
sed -i '/libcxx/d' transformer_conda_env_clean.yml
sed -i '/llvm-openmp/d' transformer_conda_env_clean.yml
(base) [phussein@submit bert-brains]$ grep -E "appnope|libcxx|

llvm-openmp|osx|macos" transformer_conda_env_clean.yml
(base) [phussein@submit bert-brains]$ grep -E "hdf|darwin|osx|macos" transformer_conda_env_clean.yml
(base) [phussein@submit bert-brains]$

Perfekt, genau so sollte es aussehen. ✅

Damit haben wir jetzt:

- ✅ Build-Hashes entfernt
- ✅ `prefix:` entfernt
- ✅ macOS-spezifische Pakete entfernt
- ✅ keine weiteren offensichtlichen macOS-Reste gefunden

Die YAML ist jetzt deutlich besser für Linux geeignet.

Jetzt würde ich den Condor-Job nochmal starten.

Vorher würde ich aber kurz sicherstellen, dass dein Submit-File wirklich die **neue YAML** verwendet. Dein `create_env.sh` sollte aktuell ungefähr so aussehen:

```bash
#!/bin/bash
set -e

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh

cd /nethome/phussein/brains/bert-brains

conda env create -f transformer_conda_env_clean.yml
```

Dann:

```bash
condor_submit create_env.sub
```

Status:

```bash
condor_q
```

Falls der nächste Fehler kommt, erwarte ich eher einen dieser beiden Fälle:

Fall 1: Weitere alte Pakete fehlen

z.B.

```
PackagesNotFoundError:
- xyz=...
```

Dann entfernen wir wieder nur die alten Build-/Systemabhängigkeiten.

Fall 2: Dependency-Konflikt

z.B.

```
UnsatisfiableError
```

Dann müssen wir einzelne Versionen lockern.

Eine Sache noch: Nach erfolgreicher Installation wird die Umgebung vermutlich heißen:

```bash
bert-brains
```

Du kannst danach testen:

```bash
conda activate bert-brains
python --version
```

und:

```bash
python -c "import transformers; print(transformers.__version__)"
```

Sollte dann ungefähr sein:

```
Python 3.9.x
4.9.1
```

Das wäre die originale Paper-Umgebung.

Schick einfach den nächsten `create_env.*.err`, falls Conda wieder meckert.

Sa., 1. Aug. um 21:08

(base) [phussein@login brains]$ cat create_env.102601.out
2 channel Terms of Service accepted
Retrieving notices: done
Channels:
 - pytorch
 - conda-forge
 - defaults
Platform: linux-64
Collecting package metadata (repodata.json): done
Solving environment: failed
(base) [phussein@login brains]$ cat create_env.102601.err

PackagesNotFoundError: The following packages are not available from current channels:

  - jpeg==9b=he5867d9_2

Current channels:

  - https://conda.anaconda.org/pytorch
  - https://conda.anaconda.org/conda-forge
  - https://repo.anaconda.com/pkgs/main
  - https://repo.anaconda.com/pkgs/r

To search for alternate channels that may provide the conda package you're
looking for, navigate to

https://anaconda.org

and use the search bar at the top of the page.

(base) [phussein@login brains]$

Das ist ein gutes Zeichen. Der Fehler hat sich verschoben und zeigt genau, wo das Problem liegt.

Diesmal ist es **kein Linux/macOS-Problem**, sondern die Version von `jpeg`:

```
Nur Text

jpeg==9b=he5867d9_2
```

`9b` passt **nicht** zu unserem bisherigen `sed`-Ausdruck, weil dieser nur Versionen wie `1.2.3` berücksichtigt hat. Deshalb ist der Build-Hash bei Paketen mit Buchstaben in der Versionsnummer (z. B. `9b`, `2021a`, `1.1.1k`) erhalten geblieben.

Ich würde jetzt nicht einzelne Pakete nach und nach reparieren

Es werden sonst wahrscheinlich noch weitere folgen, z. B.:

- `openssl=1.1.1k=...`
- `jpeg=9b=...`
- `xz=5.2.5=...`
- `tzdata=2021a=...`

Stattdessen würde ich **alle Build-Strings auf einmal entfernen**.

1. Neue Kopie anlegen

```
Bash

cp transformer_conda_env_clean.yml transformer_conda_env_linux.yml
```

2. Alle Build-Strings entfernen

Führe diesen Befehl aus:

```
Bash

sed -E -i 's/^([[:space:]]*-[[:space:]]*[^=]+=[^=]+)=.*/\1/' transformer_conda_env_linux.yml
```

Der Unterschied zu vorher ist:

- Er entfernt **alles nach dem zweiten** `=`, unabhängig davon, ob die Versionsnummer `3.3.0`, `9b`, `1.1.1k` oder `2021a` ist.

3. Kontrollieren

Zum Beispiel:

```bash
grep "^  - jpeg" transformer_conda_env_linux.yml
grep "^  - openssl" transformer_conda_env_linux.yml
grep "^  - tzdata" transformer_conda_env_linux.yml
```

Du solltest dann etwa sehen:

```yaml
- jpeg=9b
- openssl=1.1.1k
- tzdata=2021a
```

ohne den dritten Teil (`=he5867d9_2` usw.).

4. Den Condor-Job auf die neue Datei umstellen

In `create_env.sh`:

```bash
conda env create -f transformer_conda_env_linux.yml
```

Warum das sinnvoller ist

Im Moment "finden" wir die alten Build-Hashes einen nach dem anderen. Das kann noch 20–30 Iterationen dauern.

Wenn wir **alle Build-Strings** entfernen, bleibt die gewünschte Paketversion erhalten, aber Conda darf einen **aktuellen Linux-Build** auswählen. Das ist genau das, was wir für eine möglichst originalgetreue Reproduktion wollen.

Ich vermute, dass wir danach entweder:

1. die Umgebung erfolgreich erstellen können, oder
2. zum ersten echten **Versionskonflikt (`UnsatisfiableError`)** kommen. Dieser wäre dann der nächste sinnvolle Punkt zum Lösen.

_clean.yml
(base) [phussein@submit bert-brains]$ grep "^ - jpeg" transformer_conda_env_clean.yml
grep "^ - openssl" transformer_conda_env_clean.yml
grep "^ - tzdata" transformer_conda_env_clean.yml
  - jpeg=9b
  - openssl=1.1.1k
  - tzdata=2021a

Perfekt – das zeigt, dass der `sed`-Befehl bereits genau das gemacht hat, was wir wollten.

Du hast jetzt z. B.:

```yaml
- jpeg=9b
- openssl=1.1.1k
- tzdata=2021a
```

und **keine Build-Hashes mehr** (`=he5867d9_2` etc.).

Das macht mich allerdings stutzig, denn der letzte Fehler war:

```
PackagesNotFoundError:
  - jpeg==9b=he5867d9_2
```

Wenn deine aktuelle `transformer_conda_env_clean.yml` bereits `jpeg=9b` enthält, dann **hat der Condor-Job vermutlich noch die alte Datei verwendet**.

Bitte prüfe dein `create_env.sh`

Führe aus:

```bash
cat create_env.sh
```

Ich vermute, dort steht noch:

```bash
conda env create -f transformer_conda_env.yml
```

statt

```bash
conda env create -f transformer_conda_env_clean.yml
```

Das würde exakt erklären, warum Conda weiterhin die alten Build-Strings sieht.

Falls `create_env.sh` bereits korrekt ist

Dann kann es sein, dass der Job gestartet wurde, **bevor** du die Datei bereinigt hast. In dem Fall einfach den Job erneut einreichen:

```bash
condor_submit create_env.sub
```

und anschließend wieder:

```bash
cat create_env.<ClusterID>.err
```

Noch eine Anmerkung

Wir kommen jetzt langsam an den Punkt, an dem ich **nicht mehr die komplette Environment-Datei verwenden würde**, falls weitere Probleme auftreten.

Der Grund ist, dass `conda env export` standardmäßig **Hunderte transitive Abhängigkeiten** exportiert. Für eine Reproduktion braucht man die meistens gar nicht. Oft reicht eine deutlich kürzere Datei mit etwa 15–25 direkt benötigten Paketen, und Conda löst den Rest selbst.

Ich würde aber erst noch diesen nächsten Versuch abwarten. Wenn danach ein `UnsatisfiableError` oder weitere `PackagesNotFoundError` für viele alte Pakete kommen, würde ich stattdessen eine bereinigte, reproduzierbare Linux-Environment-Datei erstellen, die funktional

dem Original entspricht, aber auf einem aktuellen Linux-System installierbar ist. Das ist auch ein üblicher Weg bei älteren Forschungs-Repositories.

ich habe deine änderungen gerade im _clean gemacht. also alles ganz frisch

Dann ist das tatsächlich merkwürdig. Wenn die Änderungen **vor dem Absenden des Jobs** in `transformer_conda_env_clean.yml` waren, dürfte Conda den alten `jpeg==9b=...`-Build gar nicht mehr sehen.

Es gibt zwei naheliegende Möglichkeiten:

1. **`create_env.sh` verwendet noch die falsche Datei** (`transformer_conda_env.yml` statt `_clean.yml`).
2. Der Job wurde noch mit der alten Version gestartet.

Am schnellsten finden wir das heraus.

Bitte zeig mir den Inhalt von:

```bash
cat create_env.sh
```

und zusätzlich:

```bash
ls -l transformer_conda_env*.yml
```

Damit können wir prüfen:

- welche YAML-Datei der Job wirklich verwendet,
- ob die `_clean.yml` tatsächlich die aktuelle Datei ist.

Falls `create_env.sh` bereits auf `transformer_conda_env_clean.yml` zeigt, würde ich als Nächstes direkt in der Datei nach `jpeg` suchen:

```bash
grep -n "jpeg" transformer_conda_env_clean.yml
```

Wenn dort wirklich nur

```yaml
- jpeg=9b
```

steht, dann **sollte Conda diesen Fehler nicht mehr produzieren**. In dem Fall würden wir uns ansehen, ob der Job eventuell eine andere Datei liest oder ob es ein Caching-Problem gibt. Das bekommen wir schnell eingegrenzt.

(base) [phussein@submit bert-brains]$ grep -n "jpeg" transformer_conda_env_clean.yml
49: - jpeg=9b
101: - openjpeg=2.3.0

Das sieht gut aus. Die Datei enthält **nicht mehr** den Build-Hash.

Dann ist die entscheidende Frage jetzt wirklich:

Welche Datei verwendet `create_env.sh`?

Bitte zeige einmal:

```bash
cat create_env.sh
```

Ich vermute stark, dass dort noch steht:

```bash
conda env create -f transformer_conda_env.yml
```

anstatt

```bash
conda env create -f transformer_conda_env_clean.yml
```

Falls `create_env.sh` bereits `_clean.yml` verwendet

Dann wäre der nächste Schritt, zu prüfen, ob Condor wirklich die aktuelle Datei sieht. Dafür kannst du am Anfang von `create_env.sh` vor `conda env create` Folgendes einfügen:

```bash
echo "=== YAML being used ==="
grep -n "jpeg" transformer_conda_env_clean.yml
grep -n "openssl" transformer_conda_env_clean.yml
grep -n "tzdata" transformer_conda_env_clean.yml
echo "======================="
```

Dann erscheint im `.out`-Log genau, welche Inhalte die Datei **zum Zeitpunkt des Jobs** hatte.

Eine andere Vermutung

Mir ist gerade noch etwas aufgefallen: Du hast den Job mit einer neuen Cluster-ID gestartet (`102601`), aber der Fehlertext enthält noch:

```
jpeg==9b=he5867d9_2
```

Conda zeigt bei `PackagesNotFoundError` manchmal die **intern normalisierte Spezifikation** an. Das bedeutet **nicht zwingend**, dass dieser Build noch in deiner YAML steht. Es könnte auch sein, dass Conda intern aus `jpeg=9b` einen Build auswählt, der auf Linux nicht existiert.

Deshalb möchte ich jetzt unbedingt den Inhalt von `create_env.sh` sehen. Danach können wir entscheiden, ob das Problem wirklich die YAML ist oder ob wir auf einen anderen Conda-Solver bzw. eine minimal bereinigte Environment-Datei umsteigen sollten.

So., 2. Aug. um 11:34

(base) [phussein@login brains]$ cat create_env.102654.out
2 channel Terms of Service accepted
Retrieving notices: done
Channels:
 - pytorch
 - conda-forge
 - defaults
Platform: linux-64

Collecting package metadata (repodata.json): done
Solving environment: failed
(base) [phussein@login brains]$ cat create_env.102654.err

PackagesNotFoundError: The following packages are not available from current channels:

  - libgfortran=3.0.1

Current channels:

  - https://conda.anaconda.org/pytorch
  - https://conda.anaconda.org/conda-forge
  - https://repo.anaconda.com/pkgs/main
  - https://repo.anaconda.com/pkgs/r

To search for alternate channels that may provide the conda package you're
looking for, navigate to

    https://anaconda.org

and use the search bar at the top of the page.

(base) [phussein@login brains]$

Das ist tatsächlich ein **Fortschritt**. 👍

Der `jpeg`-Fehler ist verschwunden, also verwendet Conda jetzt deine bereinigte Datei. Jetzt stolpern wir über den nächsten veralteten Eintrag.

`libgfortran=3.0.1` ist ebenfalls ein Überbleibsel der ursprünglichen **macOS-Umgebung**. Auf Linux gibt es stattdessen Pakete wie `libgfortran-ng` oder `libgfortran5`; `libgfortran=3.0.1` existiert dort nicht.

Entferne einfach die Zeile:

```
Bash
```

```
sed -i '/libgfortran/d' transformer_conda_env_clean.yml
```

Zur Kontrolle:

```bash
grep libgfortran transformer_conda_env_clean.yml
```

Es sollte nichts mehr ausgegeben werden.

Ich vermute allerdings, dass noch weitere alte Systembibliotheken folgen werden

Die YAML stammt von 2021 und enthält viele Bibliotheken, die Conda heute selbst auswählt. Statt sie einzeln zu entfernen, könnten wir auch alle offensichtlichen Systembibliotheken streichen, z. B.:

- `libgfortran`
- `libiconv`
- `libpng`
- `libtiff`
- `libxml2`
- `xz`
- `zlib`
- `ncurses`
- `readline`
- `sqlite`
- `tk`
- `bzip2`

Diese werden bei der Installation automatisch passend für Linux aufgelöst.

Mein Vorschlag

Ich würde **noch 1–2 Fehlermeldungen abwarten**. Wenn danach wieder nur eine alte Systembibliothek fehlt, lohnt es sich nicht mehr, sie einzeln zu entfernen. Dann bereinigen wir die YAML systematisch.

Das hat zwei Vorteile:

- deutlich weniger Iterationen,
- am Ende eine saubere Linux-Umgebung, die trotzdem funktional der Originalumgebung

entspricht.

Ich kann dir auch eine **bereinigte Version der `transformer_conda_env.yml`** erstellen, die alle wissenschaftlich relevanten Pakete (Python, PyTorch, Transformers, spaCy usw.) enthält, aber die veralteten Betriebssystembibliotheken entfernt. Das ist der Weg, den viele Reproduktionsstudien bei älteren GitHub-Repositories gehen und spart viel Zeit.

(base) [phussein@submit bert-brains]$ cat transformer_conda_env_clean.yml

```
name: bert-brains
channels:
  - pytorch
  - conda-forge
  - defaults
dependencies:
  - anyio=3.3.0
  - argon2-cffi=20.1.0
  - async_generator=1.10
  - attrs=21.2.0
  - babel=2.9.1
  - backcall=0.2.0
  - blas=1.0
  - bleach=3.3.1
  - bottleneck=1.3.2
  - brotli=1.0.9
  - brotlipy=0.7.0
  - bzip2=1.0.8
  - ca-certificates=2021.7.5
  - catalogue=1.0.0
  - certifi=2021.5.30
  - cffi=1.14.6
  - chardet=4.0.0
  - charset-normalizer=2.0.0
  - cryptography=3.4.7
  - cycler=0.10.0
  - cymem=2.0.5
  - cython-blis=0.7.4
  - decorator=5.0.9
  - defusedxml=0.7.1
  - entrypoints=0.3
```

- ffmpeg=4.3
- fonttools=4.25.0
- freetype=2.10.4
- gettext=0.21.0
- gmp=6.2.1
- gnutls=3.6.15
- icu=58.2
- idna=3.1
- importlib-metadata=3.10.0
- importlib_metadata=3.10.0
- intel-openmp=2021.3.0
- ipykernel=5.3.4
- ipython=7.22.0
- ipython_genutils=0.2.0
- ipywidgets=7.6.3
- jedi=0.17.2
- jinja2=3.0.1
- jpeg=9b
- json5=0.9.5
- jsonschema=3.0.2
- jupyter=1.0.0
- jupyter_client=6.1.12
- jupyter_console=6.4.0
- jupyter_core=4.7.1
- jupyter_server=1.10.2
- jupyterlab=3.1.1
- jupyterlab_pygments=0.1.2
- jupyterlab_server=2.6.2
- jupyterlab_widgets=1.0.0
- kiwisolver=1.3.1
- lame=3.100
- lcms2=2.12
- libffi=3.3
- libiconv=1.16
- libidn2=2.3.2
- libpng=1.6.37
- libsodium=1.0.18
- libtasn1=4.16.0
- libtiff=4.2.0

- libunistring=0.9.10
- libuv=1.40.0
- libwebp-base=1.2.0
- libxml2=2.9.12
- lz4-c=1.9.3
- markupsafe=2.0.1
- matplotlib=3.4.2
- matplotlib-base=3.4.2
- mistune=0.8.4
- mkl=2021.3.0
- mkl-service=2.4.0
- mkl_fft=1.3.0
- mkl_random=1.2.2
- munkres=1.1.4
- murmurhash=1.0.5
- nbclassic=0.3.1
- nbclient=0.5.3
- nbconvert=6.1.0
- nbformat=5.1.3
- ncurses=6.2
- nest-asyncio=1.5.1
- nettle=3.7.3
- ninja=1.10.2
- notebook=6.4.0
- numexpr=2.7.3
- numpy=1.20.3
- numpy-base=1.20.3
- olefile=0.46
- openh264=2.1.0
- openjpeg=2.3.0
- openssl=1.1.1k
- packaging=21.0
- pandas=1.3.0
- pandocfilters=1.4.3
- parso=0.7.0
- pexpect=4.8.0
- pickleshare=0.7.5
- pillow=8.3.1
- pip=21.2.2

- plac=1.1.0
- preshed=3.0.5
- prometheus_client=0.11.0
- prompt-toolkit=3.0.17
- prompt_toolkit=3.0.17
- ptyprocess=0.7.0
- pycparser=2.20
- pygments=2.9.0
- pyopenssl=20.0.1
- pyparsing=2.4.7
- pyqt=5.9.2
- pyrsistent=0.18.0
- pysocks=1.7.1
- python=3.9.6
- python-dateutil=2.8.2
- python_abi=3.9
- pytorch=1.9.0
- pytz=2021.1
- pyzmq=20.0.0
- qt=5.9.7
- qtconsole=5.1.0
- qtpy=1.9.0
- readline=8.1
- requests=2.26.0
- requests-unixsocket=0.2.0
- scipy=1.6.2
- seaborn=0.11.1
- send2trash=1.5.0
- setuptools=52.0.0
- sip=4.19.13
- six=1.16.0
- sniffio=1.2.0
- spacy=2.3.5
- sqlite=3.36.0
- srsly=1.0.5
- terminado=0.9.4
- testpath=0.5.0
- thinc=7.4.5
- tk=8.6.10

```
  - torchaudio=0.9.0
  - torchvision=0.10.0
  - tornado=6.1
  - traitlets=5.0.5
  - typing_extensions=3.10.0.0
  - tzdata=2021a
  - urllib3=1.26.6
  - wasabi=0.8.2
  - wcwidth=0.2.5
  - webencodings=0.5.1
  - websocket-client=0.57.0
  - wheel=0.36.2
  - widgetsnbextension=3.5.1
  - xz=5.2.5
  - zeromq=4.3.4
  - zipp=3.5.0
  - zlib=1.2.11
  - zstd=1.4.9
  - pip:
    - click==8.0.1
    - en-core-web-lg==2.2.0
    - en-core-web-sm==2.2.0
    - filelock==3.0.12
    - huggingface-hub==0.0.12
    - joblib==1.0.1
    - pyyaml==5.4.1
    - regex==2021.8.3
    - sacremoses==0.0.45
    - sentencepiece==0.1.96
    - tokenizers==0.10.3
    - tqdm==4.62.0
    - transformers==4.9.1
```

Danke, das hilft sehr. Ehrlich gesagt würde ich **ab diesem Punkt nicht mehr versuchen, diese `conda env export`-Datei 1:1 zu reparieren**.

Der Grund ist, dass das **keine handgeschriebene Environment-Datei** ist, sondern die Ausgabe von `conda env export`. Solche Dateien enthalten **alle transitiven Abhängigkeiten**, also auch Dinge wie

- `libiconv`
- `libxml2`
- `jpeg`
- `zlib`
- `readline`
- `tk`
- `ncurses`
- `mkl`
- `intel-openmp`
- …

Diese Bibliotheken werden **nicht vom Nutzer ausgewählt**, sondern automatisch von Conda passend zum Betriebssystem installiert.

Deshalb passiert gerade Folgendes:

- ❌ `appnope`
- ❌ `libgfortran`
- ❌ `jpeg`
- → als Nächstes wahrscheinlich `intel-openmp`, `mkl`, `pyqt`, `qt`, …

Das kann noch 20–30 Runden so weitergehen.

Ich würde stattdessen eine reproduzierbare Linux-Version bauen

Wir behalten **alle wissenschaftlich relevanten Pakete und Versionen**, entfernen aber die automatisch aufgelösten Systembibliotheken.

Das sähe ungefähr so aus:

```yaml
name: bert-brains

channels:
  - pytorch
  - conda-forge
  - defaults

dependencies:
  - python=3.9
```

```
  - pytorch=1.9
  - torchvision=0.10
  - torchaudio=0.9

  - numpy=1.20
  - scipy=1.6
  - pandas=1.3

  - matplotlib=3.4
  - seaborn=0.11
```

Conda kümmert sich dann selbst um:

- libpng
- jpeg
- zlib
- libxml2
- ncurses
- readline
- qt
- ...

Warum ich glaube, dass das der richtige Weg ist

Das Paper stammt von 2024, die Umgebung aber effektiv von **2021**. Die Autoren haben damals vermutlich einfach

```bash
conda env export > transformer_conda_env.yml
```

gemacht.

Das ist für die **eigene Sicherung** gedacht, aber **nicht** für die langfristige Reproduzierbarkeit auf anderen Systemen. Viele Forschungsrepos haben genau dieses Problem.

Mein Vorschlag

Ich würde dir eine **bereinigte** `transformer_conda_env_linux.yml` erstellen, die:

- reproduzierbar auf Linux installiert werden kann,
- möglichst nah an der Originalumgebung bleibt,
- die gleichen Versionen für Python, PyTorch, spaCy und Transformers verwendet.

Das dauert vielleicht 10 Minuten statt weiterer 20 Fehlermeldungen. Ich halte das für den sinnvolleren Weg.

ok

Perfekt. Ich würde das sogar etwas systematischer machen als oben. Das Ziel ist:

- **so nah wie möglich am Original**
- **keine unnötigen Systembibliotheken**
- **gleiche Python-, PyTorch- und Transformers-Versionen**
- **exakt dieselben Ergebnisse reproduzieren können**

Vorschlag für `transformer_conda_env_linux.yml`

```yaml
name: bert-brains

channels:
  - pytorch
  - conda-forge
  - defaults

dependencies:
  - python=3.9
  - pip

  # Deep Learning
  - pytorch=1.9
  - torchvision=0.10
  - torchaudio=0.9

  # Scientific Python
  - numpy=1.20
  - scipy=1.6
  - pandas=1.3
```

Danach

```bash
conda env create -f transformer_conda_env_linux.yml
```

Danach noch

Die spaCy-Modelle wurden früher als Pip-Pakete verteilt. Das klappt heute meistens **nicht mehr**.

Stattdessen nach dem Aktivieren:

```bash
python -m spacy download en_core_web_sm
python -m spacy download en_core_web_lg
```

Falls Version 2.3 die Modelle nicht mehr herunterladen kann, laden wir sie direkt von GitHub herunter. Das ist ein bekanntes Problem bei älteren spaCy-Versionen.

Noch etwas Wichtiges

Ich würde **vorher prüfen**, welche Pakete das Repository überhaupt benutzt. Es bringt nichts, z. B. `PyQt` oder `qtconsole` mitzuschleppen, wenn sie nie importiert werden.

Ich kann das ziemlich schnell machen, indem wir alle `import`-Statements im Repository auswerten. Dann wissen wir genau:

- welche Pakete wirklich benötigt werden,
- welche nur zufällig in der exportierten Umgebung gelandet sind.

Das hat zwei Vorteile:

1. Die Umgebung wird kleiner und robuster.
2. Wenn du später **Llama 3**, **Gemma** oder andere Modelle testen möchtest, hast du bereits eine saubere Basis.

Ich würde diesen Weg bevorzugen, gerade weil du das Projekt für deine Seminararbeit wahrscheinlich noch erweitern wirst.

So., 2. Aug. um 21:55

ate_env.102593.log create_env.102654.err
create_env.102681.out create_env.102812.log
(base) [phussein@login brains]$ cat create_env.102812.
create_env.102812.err create_env.102812.log
create_env.102812.out
(base) [phussein@login brains]$ cat create_env.102812.out
2 channel Terms of Service accepted

```
Retrieving notices: done
Channels:
 - pytorch
 - conda-forge
 - defaults
Platform: linux-64
Collecting package metadata (repodata.json): done
Solving environment: done

 done
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
Installing pip dependencies: / Ran pip subprocess with
arguments:
['/nethome/phussein/miniconda3/envs/bert-brains/bin/python',
'-m', 'pip', 'install', '-U', '-r', '/nethome/phussein/brains/bert-
brains/condaenv.7emp42lb.requirements.txt', '--exists-action=b']
Pip subprocess output:
Collecting transformers==4.9.1 (from -r /nethome/phussein/
brains/bert-brains/condaenv.7emp42lb.requirements.txt (line 1))
  Downloading transformers-4.9.1-py3-none-any.whl.metadata
(49 kB)
Collecting tokenizers==0.10.3 (from -r /nethome/phussein/
brains/bert-brains/condaenv.7emp42lb.requirements.txt (line 2))
  Downloading tokenizers-0.10.3-cp39-cp39-
manylinux_2_5_x86_64.manylinux1_x86_64.manylinux_2_12_x86_
64.manylinux2010_x86_64.whl.metadata (5.8 kB)
Collecting huggingface-hub==0.0.12 (from -r /nethome/
phussein/brains/bert-brains/
condaenv.7emp42lb.requirements.txt (line 3))
  Downloading huggingface_hub-0.0.12-py3-none-
any.whl.metadata (5.6 kB)
Collecting sentencepiece==0.1.96 (from -r /nethome/phussein/
brains/bert-brains/condaenv.7emp42lb.requirements.txt (line 4))
  Downloading sentencepiece-0.1.96-cp39-cp39-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (10
kB)
Collecting sacremoses==0.0.45 (from -r /nethome/phussein/
```

```
brains/bert-brains/condaenv.7emp42lb.requirements.txt (line 5))
  Downloading sacremoses-0.0.45-py3-none-any.whl.metadata
(476 bytes)
Collecting regex==2021.8.3 (from -r /nethome/phussein/brains/
bert-brains/condaenv.7emp42lb.requirements.txt (line 6))
  Downloading regex-2021.8.3-cp39-cp39-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (39
kB)
Collecting tqdm==4.62.0 (from -r /nethome/phussein/brains/
bert-brains/condaenv.7emp42lb.requirements.txt (line 7))
  Downloading tqdm-4.62.0-py2.py3-none-any.whl.metadata (57
kB)
Collecting click==8.0.1 (from -r /nethome/phussein/brains/bert-
brains/condaenv.7emp42lb.requirements.txt (line 8))
  Downloading click-8.0.1-py3-none-any.whl.metadata (3.2 kB)
Collecting filelock==3.0.12 (from -r /nethome/phussein/brains/
bert-brains/condaenv.7emp42lb.requirements.txt (line 9))
  Downloading filelock-3.0.12-py3-none-any.whl.metadata (4.3
kB)
Collecting joblib==1.0.1 (from -r /nethome/phussein/brains/
bert-brains/condaenv.7emp42lb.requirements.txt (line 10))
  Downloading joblib-1.0.1-py3-none-any.whl.metadata (6.8 kB)
Collecting pyyaml==5.4.1 (from -r /nethome/phussein/brains/
bert-brains/condaenv.7emp42lb.requirements.txt (line 11))
  Downloading PyYAML-5.4.1-cp39-cp39-
manylinux1_x86_64.whl.metadata (2.1 kB)
Requirement already satisfied: numpy>=1.17 in /nethome/
phussein/miniconda3/envs/bert-brains/lib/python3.9/site-
packages (from transformers==4.9.1->-r /nethome/phussein/
brains/bert-brains/condaenv.7emp42lb.requirements.txt (line 1))
(1.20.3)
Requirement already satisfied: packaging in /nethome/phussein/
miniconda3/envs/bert-brains/lib/python3.9/site-packages (from
transformers==4.9.1->-r /nethome/phussein/brains/bert-brains/
condaenv.7emp42lb.requirements.txt (line 1)) (26.2)
Requirement already satisfied: requests in /nethome/phussein/
miniconda3/envs/bert-brains/lib/python3.9/site-packages (from
transformers==4.9.1->-r /nethome/phussein/brains/bert-brains/
condaenv.7emp42lb.requirements.txt (line 1)) (2.32.5)
```

```
Requirement already satisfied: typing-extensions in /nethome/
phussein/miniconda3/envs/bert-brains/lib/python3.9/site-
packages (from huggingface-hub==0.0.12->-r /nethome/
phussein/brains/bert-brains/
condaenv.7emp42lb.requirements.txt (line 3)) (4.14.1)
Requirement already satisfied: six in /nethome/phussein/
miniconda3/envs/bert-brains/lib/python3.9/site-packages (from
sacremoses==0.0.45->-r /nethome/phussein/brains/bert-brains/
condaenv.7emp42lb.requirements.txt (line 5)) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /
nethome/phussein/miniconda3/envs/bert-brains/lib/python3.9/
site-packages (from requests->transformers==4.9.1->-r /
nethome/phussein/brains/bert-brains/
condaenv.7emp42lb.requirements.txt (line 1)) (3.4.3)
Requirement already satisfied: idna<4,>=2.5 in /nethome/
phussein/miniconda3/envs/bert-brains/lib/python3.9/site-
packages (from requests->transformers==4.9.1->-r /nethome/
phussein/brains/bert-brains/
condaenv.7emp42lb.requirements.txt (line 1)) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /nethome/
phussein/miniconda3/envs/bert-brains/lib/python3.9/site-
packages (from requests->transformers==4.9.1->-r /nethome/
phussein/brains/bert-brains/
condaenv.7emp42lb.requirements.txt (line 1)) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /nethome/
phussein/miniconda3/envs/bert-brains/lib/python3.9/site-
packages (from requests->transformers==4.9.1->-r /nethome/
phussein/brains/bert-brains/
condaenv.7emp42lb.requirements.txt (line 1)) (2025.8.3)
Downloading transformers-4.9.1-py3-none-any.whl (2.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.6/2.6 MB 8.8
MB/s  0:00:00
Downloading tokenizers-0.10.3-cp39-cp39-
manylinux_2_5_x86_64.manylinux1_x86_64.manylinux_2_12_x86_
64.manylinux2010_x86_64.whl (3.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.3/3.3 MB 10.1
MB/s  0:00:00
Downloading huggingface_hub-0.0.12-py3-none-any.whl (37 kB)
Downloading sentencepiece-0.1.96-cp39-cp39-
```

```
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 5.4
MB/s  0:00:00
Downloading sacremoses-0.0.45-py3-none-any.whl (895 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 895.2/895.2 kB
4.3 MB/s  0:00:00
Downloading regex-2021.8.3-cp39-cp39-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (732 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 732.7/732.7 kB
3.6 MB/s  0:00:00
Downloading tqdm-4.62.0-py2.py3-none-any.whl (76 kB)
Downloading click-8.0.1-py3-none-any.whl (97 kB)
Downloading filelock-3.0.12-py3-none-any.whl (7.6 kB)
Downloading joblib-1.0.1-py3-none-any.whl (303 kB)
Downloading PyYAML-5.4.1-cp39-cp39-manylinux1_x86_64.whl
(630 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 630.1/630.1 kB
2.8 MB/s  0:00:00
Installing collected packages: tokenizers, sentencepiece, regex,
filelock, tqdm, pyyaml, joblib, click, sacremoses, huggingface-
hub, transformers
  Attempting uninstall: tqdm
    Found existing installation: tqdm 4.67.1
    Uninstalling tqdm-4.67.1:
      Successfully uninstalled tqdm-4.67.1
  Attempting uninstall: joblib
    Found existing installation: joblib 1.5.1
    Uninstalling joblib-1.5.1:
      Successfully uninstalled joblib-1.5.1

Successfully installed click-8.0.1 filelock-3.0.12 huggingface-
hub-0.0.12 joblib-1.0.1 pyyaml-5.4.1 regex-2021.8.3
sacremoses-0.0.45 sentencepiece-0.1.96 tokenizers-0.10.3
tqdm-4.62.0 transformers-4.9.1

done
#
# To activate this environment, use
#
```

```
#     $ conda activate bert-brains
#
# To deactivate an active environment, use
#
#     $ conda deactivate


Channel "defaults" has the following notices:
  [info] -- Tue Jun  9 00:00:00 2026
  PyTorch 2.12 with CUDA support is now available to install with
your current channel (Anaconda Main). Learn more: https://
anaconda.org/main/pytorch?utm_source=channel_notices

(base) [phussein@login brains]$ cat create_env.102812.err


==> WARNING: A newer version of conda exists. <==
    current version: 26.1.1
    latest version: 26.5.3

Please update conda by running

    $ conda update -n base -c conda-forge conda


(base) [phussein@login brains]$
```

🎉 **Super! Die Umgebung wurde erfolgreich erstellt.**

Das ist ein wichtiger Meilenstein – jetzt können wir anfangen, den Code tatsächlich auszuführen.

1. Umgebung aktivieren

‹/› Bash

```
conda activate bert-brains
```

Prüfe kurz:

‹/› Bash

```
python --version
```

Sollte etwa ausgeben:

```
Nur Text
Python 3.9.x
```

---

2. Funktioniert PyTorch?

```
Bash
python -c "import torch; print(torch.__version__)"
```

Erwartet:

```
Nur Text
1.9.0
```

---

3. Funktioniert Transformers?

```
Bash
python -c "import transformers; print(transformers.__version__)"
```

Erwartet:

```
Nur Text
4.9.1
```

---

4. Funktioniert spaCy?

```
Bash
```

```
python -c "import spacy; print(spacy.__version__)"
```

Erwartet:

```
Nur Text
2.3.5
```

---

5. Sind die Sprachmodelle installiert?

Das würde ich direkt testen:

```
Bash
python -c "import spacy; spacy.load('en_core_web_sm'); print('OK')"
```

und

```
Bash
python -c "import spacy; spacy.load('en_core_web_lg'); print('OK')"
```

Es kann gut sein, dass hier ein Fehler kommt, weil wir die Modelle bewusst aus der Environment-Datei entfernt haben. Falls ja, installieren wir sie anschließend separat.

Danach: Repository testen

Ich würde **nicht sofort die komplette Pipeline** starten, sondern Schritt für Schritt.

Als Erstes schauen wir uns die Transformer-Repräsentationen an, weil das genau der Teil ist, den du später für Llama/Gemma austauschen möchtest.

Dazu interessiert mich insbesondere:

```
Nur Text
text_processing_and_transformers/
    transformer-representations.ipynb
```

Wir wollen prüfen:

1. Läuft das Notebook?
2. Welche Daten werden eingelesen?
3. Wo wird BERT geladen?
4. Wo werden Embeddings und Layer-Repräsentationen gespeichert?

Ich glaube, dein Seminarprojekt wird letztlich ungefähr so aussehen:

```
Transkript
      |
      ▼
BERT (Original)
      |
      ▼
Embeddings / Transformations
      |
      ▼
Encoding Model
      |
      ▼
fMRI Prediction
```

und anschließend

```
Transkript
      |
      ▼
Llama / Gemma / ModernBERT
      |
      ▼
Embeddings / Transformations
      |
      ▼
Encoding Model
      |
      ▼
Vergleich der Vorhersagegüte
```

Dadurch musst du **nur den ersten Block austauschen**. Die gesamte fMRI-Pipeline bleibt unverändert – genau das macht dein Projekt methodisch sauber.

Nächster Schritt

Ich würde jetzt gemeinsam mit dir `transformer-representations.ipynb` analysieren. Das ist sehr wahrscheinlich die zentrale Datei für dein Vorhaben, weil dort die Modellrepräsentationen erzeugt werden. Sobald wir verstanden haben, wie BERT dort eingebunden ist, können wir später relativ gezielt ein neueres Modell einsetzen.

so?

#!/bin/bash

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh
#conda activate /scratch/phussein/envs/test

conda activate bert-brains

export HF_HOME=/scratch/phussein/hf_cache
export TRANSFORMERS_CACHE=$HF_HOME
export HF_TOKEN="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"

#python ...py

Ja, fast. 👍

Ich würde das Skript zunächst zum **Testen der Umgebung** nutzen, bevor du das eigentliche Notebook oder Python-Skript startest.

Zum Beispiel:

Bash

```bash
#!/bin/bash

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh

conda activate bert-brains

echo "Python:"
python --version

echo "PyTorch:"
python -c "import torch; print(torch.__version__)"
```

```
echo "Transformers:"
python -c "import transformers; print(transformers.__version__)"

echo "spaCy:"
python -c "import spacy; print(spacy.__version__)"

echo "Done."
```

Wenn das erfolgreich durchläuft, weißt du, dass die Umgebung korrekt funktioniert.

Danach

Der nächste Schritt wäre **nicht** `python ...py`, sondern zunächst herauszufinden, welches Skript bzw. Notebook die Repräsentationen erzeugt.

Laut README ist das:

```
text_processing_and_transformers/
    transformer-representations.ipynb
```

Da es ein Notebook ist, gibt es zwei Möglichkeiten:

1. **Auf dem Cluster mit Jupyter** (wahrscheinlich so, wie es die Autoren gemacht haben; es gibt sogar ein `run_jupyter.sh`).
2. **Den relevanten Notebook-Code in ein Python-Skript umwandeln**, was für reproduzierbare Experimente oft angenehmer ist.

Ich würde jetzt zuerst den Code verstehen

Zeig mir einmal den Inhalt von

```
Bash
ls text_processing_and_transformers
```

und insbesondere die ersten Zellen des Notebooks:

```
Bash
jupyter nbconvert \
```

```
  --to script \
  text_processing_and_transformers/transformer-representations.ipynb
```

Dadurch entsteht eine `.py`-Datei, die wir gemeinsam lesen können. So finden wir schnell heraus:

- wo BERT geladen wird,
- welche Daten eingelesen werden,
- wo die Embeddings gespeichert werden.

Das ist genau die Stelle, die du später für Llama oder ein anderes Modell austauschen möchtest.

Mo., 3. Aug. um 11:37

[phussein@login brains]$ cat brains.sh.102885.2026_08_02_1785700643.out
Python:
Python 3.9.23
PyTorch:
1.9.1.post3
Transformers:
4.9.1
spaCy:
2.3.5
Done.
[phussein@login brains]$ cat brains.sh.102885.2026_08_02_1785700643.err

Perfekt! 🎉

Die Umgebung funktioniert.

Die einzige kleine Abweichung ist:

```
PyTorch 1.9.1.post3
```

statt

```
1.9.0
```

Das ist völlig unproblematisch. `1.9.1.post3` ist ein Bugfix-Release derselben Hauptversion und sollte für die Reproduktion keine Rolle spielen.

Damit haben wir jetzt

✅ Python 3.9
✅ PyTorch 1.9.x
✅ Transformers 4.9.1
✅ spaCy 2.3.5
✅ Keine Importfehler

Das war der schwierigste Teil. 😊

Als Nächstes würde ich den Code verstehen

Nicht sofort alles laufen lassen.

Mich interessiert zunächst

```
text_processing_and_transformers/
```

Kannst du einmal ausführen:

```bash
ls text_processing_and_transformers
```

Und besonders wichtig:

```bash
jupyter nbconvert \
    --to script \
    text_processing_and_transformers/transformer-representations.ipynb
```

Dadurch wird aus dem Notebook eine Python-Datei:

```
transformer-representations.py
```

Die können wir dann gemeinsam lesen.

Warum ich genau dort anfangen möchte

Für dein Projekt musst du **nur eine einzige Stelle** ändern:

```
Transcript
      |
      ▼
BERT  ← diese Stelle
      |
      ▼
Representations
      |
      ▼
Encoding Model
      |
      ▼
fMRI prediction
```

Wenn wir verstehen,

- wie BERT geladen wird,
- welche Hidden States extrahiert werden,
- wie sie gespeichert werden,

dann ist der Austausch gegen

- Llama 3
- Gemma 3
- Qwen
- ModernBERT

vergleichsweise einfach.

Ich habe sogar schon eine Idee für die Evaluation

Das Paper verwendet hauptsächlich

- Embeddings
- Layer representations ("transformations")
- Head representations

Ich würde dein Projekt wahrscheinlich so aufbauen:

1. **Exakte Reproduktion mit BERT** (zeigt, dass die Pipeline funktioniert)
2. **Nur das Modell austauschen** (z. B. ModernBERT oder Llama)
3. **Vergleich der Encoding Performance**
4. **Optional:** Functional Specialization des neuen Modells

Das ist methodisch sehr sauber, weil sich zwischen den Experimenten **nur eine Variable** ändert: das Sprachmodell. Ich halte das für ein starkes Seminarprojekt.

(base) [phussein@submit text_processing_and_transformers]$ ls
decode_linguistic_features.py narratives-transcript-processing.ipynb transformer_utils.py
decode_linguistic_features_utils.py transformer-representations.ipynb
extract_linguistic_features.py transformer-transformation-magnitudes.ipynb
(base) [phussein@submit text_processing_and_transformers]$

Perfekt. Jetzt würde ich **nicht sofort das Notebook ausführen**, sondern zuerst herausfinden, was es genau macht. Das spart später viel Zeit.

Da du auf dem Cluster arbeitest, hast du zwei Möglichkeiten.

Möglichkeit 1 (empfohlen): Notebook in Python umwandeln

Im Ordner `text_processing_and_transformers`:

```bash
jupyter nbconvert --to script transformer-representations.ipynb
```

Danach solltest du sehen:

```
transformer-representations.py
```

Den kannst du dir dann ansehen:

```bash
head -100 transformer-representations.py
```

oder

```bash
less transformer-representations.py
```

Möglichkeit 2

Falls `nbconvert` nicht installiert ist, kannst du auch direkt Jupyter starten oder das Notebook lokal öffnen. Aber ich finde die `.py`-Version für das Verständnis deutlich angenehmer.

Worauf wir achten

Mich interessieren insbesondere diese Stellen:

- Welche BERT-Variante wird geladen?

  ```python
  BertModel.from_pretrained(...)
  ```

  oder

  ```python
  AutoModel.from_pretrained(...)
  ```

- Welcher Tokenizer?
- Werden
  - hidden states,
  - embeddings,
  - attention heads

  extrahiert?
- In welches Verzeichnis werden die Repräsentationen gespeichert?

Das ist wichtig für dein Projekt

Wenn wir den Code verstehen, wird der Umbau wahrscheinlich ungefähr so aussehen:

```python
# Original
model = BertModel.from_pretrained("bert-base-uncased")
```

↓

```python
# Neues Experiment
model = AutoModel.from_pretrained("answerdotai/ModernBERT-base")
```

oder

```python
model = AutoModel.from_pretrained(
    "meta-llama/Llama-3.2-3B"
)
```

Der Rest der Pipeline könnte nahezu unverändert bleiben.

Ich habe noch eine Anmerkung zu deiner Projektidee

Ich würde **nicht mit Llama beginnen**, sondern mit einem **Encoder-Modell** wie **ModernBERT**.

Der Grund:

- BERT → Encoder
- ModernBERT → moderner Encoder

Beide liefern für jedes Token direkt kontextualisierte Repräsentationen mit sehr ähnlicher API und Architektur. Das macht den Austausch in der bestehenden Pipeline deutlich einfacher.

Ein Decoder-only-Modell wie Llama funktioniert zwar grundsätzlich auch, aber die Repräsentationen unterscheiden sich architektonisch stärker. Falls ihr im Seminar nicht explizit Decoder-Modelle vergleichen wollt, ist ModernBERT für eine erste Reproduktion wahrscheinlich die risikoärmere Wahl. Danach könntest du Llama immer noch als Erweiterung hinzufügen.

MODEL = 'bert-base-uncased'
# BERT: bert-base-uncased, bert-large-uncased
# GPT: gpt2, gpt2-xl, gpt-neo-2.7B

foo = TransformerRSM(STIMULUS, model_name=MODEL, verbose=True)

Das ist sogar noch besser als erwartet. 👍

Die Pipeline ist bereits **modellagnostisch** aufgebaut.

Du hast nämlich:

```python
MODEL = 'bert-base-uncased'

foo = TransformerRSM(
    STIMULUS,
    model_name=MODEL,
    verbose=True
)
```

Das bedeutet, dass das Notebook **nicht direkt** `BertModel` **lädt**, sondern über eine eigene Klasse `TransformerRSM`.

Der entscheidende Code liegt also wahrscheinlich in

```
text_processing_and_transformers/
    transformer_utils.py
```

Dort wird vermutlich anhand von

```python
model_name
```

entschieden, welches Hugging Face-Modell geladen wird.

Das ist eine sehr gute Nachricht

Denn dann könnte dein Experiment im Idealfall nur aus

```python
MODEL = "bert-base-uncased"
```

↓

```python
MODEL = "roberta-base"
```

oder

```python
MODEL = "answerdotai/ModernBERT-base"
```

oder

```python
MODEL = "gpt2-xl"
```

bestehen.

Wenn `TransformerRSM` sauber implementiert ist, braucht man fast nichts ändern.

Als Nächstes

Mich interessiert jetzt `transformer_utils.py`.

Kannst du einmal suchen nach

```bash
grep -n "from_pretrained" transformer_utils.py
```

und außerdem

```bash
grep -n "AutoModel" transformer_utils.py
grep -n "BertModel" transformer_utils.py
grep -n "GPT2Model" transformer_utils.py
```

Meine Vermutung

Ich vermute, dort steht etwas wie

```python
if "bert" in model_name:
    ...
elif "gpt" in model_name:
    ...
```

oder

```python
AutoModel.from_pretrained(model_name)
```

Falls letzteres verwendet wird, wird dein Projekt noch einfacher.

Das wird gerade immer interessanter

Ich dachte zunächst, du müsstest große Teile der Pipeline umbauen.

Jetzt sieht es eher so aus, als hätte der Erstautor die Software bewusst generisch geschrieben.

Das würde bedeuten:

```
BERT
GPT-2
GPT-Neo
```

laufen schon über dieselbe Pipeline.

Dann wäre dein Projekt im Wesentlichen:

> **Kann ein moderneres Sprachmodell dieselbe Pipeline durchlaufen und bessere Gehirnvorhersagen liefern?**

Das ist methodisch genau das, was man für eine Replikation mit einer gezielten Erweiterung möchte.

```
(base) [phussein@submit text_processing_and_transformers]$
grep -n "from_pretrained" transformer_utils.py
19:        self.tokenizer =
AutoTokenizer.from_pretrained(self.model_name)
22:        self.transformer =
AutoModel.from_pretrained(self.model_name,
```

Das ist **die beste Nachricht des Tages.** 😄

Das bedeutet, dass die Autoren **bereits die generischen Hugging Face-Klassen** verwenden:

```python
self.tokenizer = AutoTokenizer.from_pretrained(self.model_name)

self.transformer = AutoModel.from_pretrained(
    self.model_name,
    ...
)
```

Konsequenz

Für sehr viele Modelle musst du **überhaupt keinen Code ändern**.

Du kannst wahrscheinlich einfach

```python
MODEL = "bert-base-uncased"
```

ersetzen durch

```python
MODEL = "answerdotai/ModernBERT-base"
```

oder

```python
MODEL = "FacebookAI/roberta-base"
```

oder

```python
MODEL = "google/electra-base-discriminator"
```

und die Pipeline läuft weiter.

Für Llama ist es etwas komplizierter

`AutoModel` lädt zwar auch Llama, aber es gibt Unterschiede:

- Decoder-only statt Encoder
- anderes Attention-Mask-Verhalten
- andere Tokenisierung
- eventuell andere Hidden-State-Formate

Trotzdem funktioniert `AutoModel` grundsätzlich auch für viele Decoder-Modelle.

Ich würde die Experimente in dieser Reihenfolge machen

1. Reproduktion

```
bert-base-uncased
```

↓

Encoding Scores reproduzieren.

---

2. Einfacher Encoder

```
Nur Text

roberta-base
```

Nahezu identische Architektur.

---

3. Moderner Encoder ⭐

```
Nur Text

answerdotai/ModernBERT-base
```

Das wäre aus meiner Sicht der spannendste Vergleich.

---

4. Optional

Dann

```
Nur Text

Llama-3.2-3B
```

oder

```
Nur Text

Qwen2.5-3B
```

Warum ich ModernBERT bevorzugen würde

Deine ursprüngliche Motivation war:

> BERT verarbeitet Negation und andere semantische Phänomene nur eingeschränkt, wodurch Informationen in den Repräsentationen verloren gehen.

Das passt **perfekt** zu ModernBERT:

- gleiche Modellklasse (Encoder)
- deutlich neuer

- auf modernem Datensatz trainiert
- bessere semantische Repräsentationen in vielen Benchmarks

Wenn ModernBERT tatsächlich die Gehirnaktivität besser vorhersagt, wäre das eine sehr überzeugende Erweiterung der Originalarbeit.

Eine Sache würde ich noch prüfen

Mich interessiert jetzt noch die zweite Zeile:

```python
AutoModel.from_pretrained(...)
```

Du hast nur den Anfang gezeigt.

Kannst du bitte die ersten etwa 40 Zeilen von `transformer_utils.py` zeigen? Insbesondere interessiert mich der Konstruktor von `TransformerRSM`. Dort sehen wir, ob die Autoren `output_hidden_states=True` setzen oder weitere Annahmen über die Modellarchitektur treffen. Das entscheidet darüber, wie einfach der Austausch gegen neuere Modelle tatsächlich wird.

aber im jupyter notebook steht doch gpt-2, das ja auch decoder only ist

Genau – und das ist ein sehr wichtiger Punkt. Das spricht sogar **für** deine Projektidee.

Im Notebook steht:

```python
MODEL = "bert-base-uncased"

# BERT:
# bert-base-uncased
# bert-large-uncased

# GPT:
# gpt2
# gpt2-xl
# gpt-neo-2.7B
```

Und `transformer_utils.py` verwendet:

```python
AutoTokenizer.from_pretrained(...)
AutoModel.from_pretrained(...)
```

Das bedeutet, die Autoren haben die Pipeline **bewusst so geschrieben**, dass sie sowohl

- Encoder (BERT)
- Decoder (GPT-2, GPT-Neo)

unterstützt.

Das ist ein starkes Indiz dafür, dass sie hauptsächlich mit den **Hidden States** arbeiten und nicht mit architekturspezifischen Komponenten.

Warum funktioniert das?

`AutoModel` liefert für alle diese Modelle im Wesentlichen dasselbe Interface:

```python
outputs = model(...)
```

mit z. B.

```python
outputs.last_hidden_state
outputs.hidden_states
```

Egal ob

- BERT
- GPT-2
- GPT-Neo

Die Hidden States haben dieselbe Form:

```
(batch_size,
 sequence_length,
 hidden_size)
```

Für die Encoding-Modelle ist genau das relevant.

Das macht Llama deutlich realistischer

Vorher dachte ich, die Pipeline könnte BERT-spezifisch sein.

Jetzt wissen wir:

- ✅ BERT
- ✅ GPT-2
- ✅ GPT-Neo

laufen bereits durch dieselbe Klasse.

Deshalb würde ich inzwischen sagen, dass auch Modelle wie

- Llama 3.2
- Qwen 2.5
- Gemma 3

prinzipiell gute Kandidaten sind.

Trotzdem würde ich noch etwas prüfen

Mich interessiert noch, **welche Annahmen `TransformerRSM` intern macht**.

Zum Beispiel:

- Nutzt er `last_hidden_state`?
- Nutzt er `hidden_states`?
- Greift er auf `pooler_output` zu? (Dann wäre BERT speziell.)
- Berechnet er Attention-Head-Ausgaben selbst?

Das entscheidet darüber, ob ein modernes Decoder-Modell wirklich ohne Änderungen funktioniert.

Ich würde deine Projektidee inzwischen leicht anpassen

Ursprünglich hattest du geschrieben:

> "Ich nehme ein neueres Modell."

Ich würde sie etwas präziser formulieren:

> **Kann ein modernes Transformer-Sprachmodell (Encoder oder Decoder), dessen Repräsentationen semantisch leistungsfähiger sind als die von BERT und GPT-2, die Gehirnaktivität besser vorhersagen?**

Das passt sehr gut zur ursprünglichen Motivation der Arbeit und nutzt die Tatsache, dass die Pipeline bereits mehrere Modellfamilien unterstützt.

wie messe ich dass ie repräsentationen semantisch leistungsfähiger sind? muss ich eigene tests machen?

Das ist tatsächlich der Knackpunkt deines Projektvorschlags. Es gibt zwei verschiedene Behauptungen, die man leicht verwechselt:

1. **Ein Modell besitzt semantisch bessere Repräsentationen.**
2. **Diese Repräsentationen sagen Gehirnaktivität besser voraus.**

Für dein Projekt musst du streng genommen **nur (2)** zeigen. Die Motivation für (2) kann sein, dass neuere Modelle semantisch stärkere Repräsentationen besitzen.

Möglichkeit 1 (meine Empfehlung): Nur Brain-Encoding vergleichen

Das Paper macht genau eine Aussage:

> Welche Modellrepräsentationen erklären fMRI-Signale?

Dann wäre dein Projekt:

- BERT
- GPT-2
- Llama 3
- evtl. Gemma oder Qwen

alle durch dieselbe Pipeline laufen lassen und vergleichen:

- Encoding Accuracy
- Layer Preference
- Functional Specialization

Dann kannst du in der Diskussion schreiben:

> Neuere Modelle wurden entwickelt, um semantische Informationen besser zu repräsentieren (Negation, Coreference, Long-range Dependencies...). Falls diese Verbesserungen für die menschliche Sprachverarbeitung relevant sind, sollten ihre Repräsentationen auch Gehirnaktivität besser vorhersagen.

Dafür brauchst du **keinen eigenen Semantik-Test**.

Möglichkeit 2: Zusätzlich nachweisen, dass die Repräsentationen semantisch besser sind

Das wäre wissenschaftlich schöner.

Dann könntest du dieselben Layer zusätzlich auf einem bekannten Benchmark testen.

a) SentEval

Sehr beliebt.

Misst, wie gut Embeddings kodieren:

- sentiment
- paraphrase
- semantic similarity
- entailment

Dann könntest du schreiben

Modell	STS	Brain Encoding
BERT	0.78	0.21
GPT2	0.73	0.19
Llama	0.86	0.27

und untersuchen, ob semantisch bessere Modelle auch brain-like sind.

---

b) MTEB

Der moderne Standard.

https://huggingface.co/spaces/mteb/leaderboard ↗

Enthält

- Semantic Similarity
- Retrieval
- Classification
- Clustering

Man könnte einfach die veröffentlichten Scores zitieren.

---

c) Negation Benchmarks

Da du Negation erwähnt hast.

Es gibt Datensätze wie

- NegNLI
- NegBLEURT
- Negated LAMA
- CREPE

oder Papers wie

- *Language Models are Poor at Negation* (Kassner & Schütze)
- *Do Language Models Understand Negation?*

Dann könntest du explizit argumentieren:

> BERT hat bekannte Schwierigkeiten mit Negation. Llama zeigt deutlich bessere Ergebnisse. Führt dies zu besserem Brain Encoding?

Das wäre eine schöne, fokussierte Motivation.

Möglichkeit 3 (meine Lieblingsidee)

Gar keine zusätzlichen Benchmarks.

Stattdessen untersuchst du direkt, **welche linguistischen Eigenschaften** im Gehirn besser vorhergesagt werden.

Das Repository enthält bereits:

```
decode_linguistic_features.py
```

Das decodiert

- POS
- Dependency Labels

aus den Head-Repräsentationen.

Du könntest das erweitern um

- Negation
- Constituents
- Semantic Roles
- Coreference

Dann wäre die Story:

> Neuere Modelle repräsentieren bestimmte linguistische Informationen besser. Werden genau diese Informationen auch stärker im Gehirn repräsentiert?

Das wäre näher am Paper.

Muss man Semantik überhaupt messen?

Eigentlich nein.

In einer guten wissenschaftlichen Arbeit reicht folgende Argumentationskette:

> Frühere Arbeiten zeigen, dass BERT Schwächen bei Negation und anderen semantischen Phänomenen besitzt (z. B. Kassner & Schütze, 2020; Ettinger, 2020). Neuere Decoder-Modelle erzielen auf zahlreichen Sprachbenchmarks deutlich bessere Ergebnisse. Wir untersuchen daher, ob diese verbesserten Repräsentationen auch zu einer besseren Vorhersage menschlicher Hirnaktivität führen.

Das ist eine völlig legitime Motivation.

Was ich für ein 7-CP-Projekt machen würde

Ich würde den Umfang bewusst überschaubar halten:

1. **Reproduktion** der BERT-Ergebnisse.
2. **Llama 3 (oder Gemma/Qwen)** in dieselbe Pipeline einsetzen.
3. Vergleich:
    - Encoding Accuracy
    - Layerwise Encoding
    - Functional Specialization (optional als Bonus)
4. Diskussion im Kontext bekannter Verbesserungen neuer LLMs (Semantik, Negation, Long Context), gestützt auf die Literatur, ohne selbst neue Semantik-Benchmarks durchzuführen.

Das ist methodisch sauber, gut begründbar und vom Umfang realistisch. Wenn danach noch Zeit bleibt, wäre eine kleine Zusatzanalyse – etwa ein Negationsbenchmark oder der Vergleich der Functional Specialization – ein sinnvoller Bonus, aber ich würde ihn nicht als Kern des Projekts einplanen.

gibt es auch einen besseren encoder-decoder, den ich statt bert verwenden kann?

Ja – allerdings muss man zwischen **Encoder-only**, **Encoder-Decoder** und **Decoder-only** unterscheiden. Für dein Projekt ist das wichtig, weil die Pipeline aus dem Paper ursprünglich für **Encoder-Repräsentationen** geschrieben wurde.

1. Moderne Encoder-only Modelle (empfehlenswert)

Das wären die direktesten BERT-Nachfolger.

Modell	Jahr	Vorteile
DeBERTa	2021	Deutlich bessere Sprachrepräsentationen als BERT
RoBERTa	2019	Verbesserte BERT-Trainingsstrategie
ModernBERT	2024	Moderner BERT-Ersatz mit längeren Kontexten
MPNet	2020	Kombiniert BERT- und Permutation-Training
ELECTRA	2020	Effizienteres Pretraining

Von diesen würde ich **DeBERTa** oder **ModernBERT** wählen.

---

2. Encoder-Decoder Modelle

Falls du wirklich Encoder-Decoder meinst:

Modell	Encoder	Decoder
T5	✓	✓
FLAN-T5	✓	✓
BART	✓	✓
mT5	✓	✓

Diese Modelle besitzen **einen vollständigen Encoder** und zusätzlich einen Decoder.

Man könnte also einfach die **Encoder-Ausgaben** verwenden.

---

3. Decoder-only (wie GPT-2)

Hierzu gehören

- Llama 3
- Qwen3
- Gemma 3
- Mistral

Das Paper verwendet bereits GPT-2, also ist grundsätzlich gezeigt, dass Decoder-only Modelle mit der Pipeline funktionieren.

Was wäre wissenschaftlich am interessantesten?

Ich würde **nicht** BERT gegen T5 vergleichen.

Warum?

T5 unterscheidet sich in **zwei Dingen gleichzeitig**:

- andere Architektur
- besseres Training

Wenn T5 besser ist, weißt du nicht, woran es liegt.

Deutlich sauberer wäre:

Encoder-only Vergleich

- BERT
- RoBERTa
- DeBERTa
- ModernBERT

Dann ändert sich hauptsächlich die Qualität der Repräsentationen.

Oder

Decoder-only Vergleich

- GPT-2
- Llama 3
- Gemma 3
- Qwen 3

Dann untersuchst du die Entwicklung moderner autoregressiver Modelle.

Mein Favorit

Ich würde tatsächlich **DeBERTa v3 Base** gegen **BERT Base** vergleichen.

Warum?

- gleiche Modellgröße (~110–180 Mio. Parameter, je nach Variante)
- gleiche Grundidee (Encoder-only)
- gleiche Hidden Size (bei den Base-Varianten 768), wodurch die bestehende Pipeline wahrscheinlich mit minimalen Änderungen funktioniert
- gilt in der NLP-Community als klarer Nachfolger von BERT
- deutlich bessere Ergebnisse auf vielen Sprachverständnis-Benchmarks (GLUE, SuperGLUE usw.)

Damit wäre deine Fragestellung sehr klar:

> **Verbessern modernere Encoder-Repräsentationen die Vorhersage von Gehirnaktivität gegenüber BERT?**

Das ist methodisch sauberer als BERT direkt mit einem sehr großen Llama-Modell zu vergleichen, weil du den Einfluss der Architektur und Modellgröße stärker kontrollierst.

Di., 4. Aug. um 14:33

ok. wie kriege ich etzt die pipeline zum laufen? welche infos

brauchst du?

Jetzt würde ich **systematisch** vorgehen. Das Ziel ist zunächst **nicht**, das ganze Paper nachzurechnen, sondern **einen Notebook-Durchlauf** erfolgreich zu schaffen. Danach können wir auf neuere Modelle umstellen.

Was wir bereits wissen

✅ Repository ist auf dem Cluster.

✅ `bert-brains`-Environment läuft.

✅ `transformers==4.9.1` funktioniert.

✅ PyTorch funktioniert.

✅ Die Daten liegen bereits unter `data/`.

Das sind schon die größten Hürden.

Als Nächstes

Wir müssen herausfinden, **welches Notebook als Einstieg gedacht ist**.

Nach der README ist das wahrscheinlich

```
text_processing_and_transformers/
    transformer-representations.ipynb
```

oder

```
narratives-transcript-processing.ipynb
```

Da eure Daten (`tr_tokens.csv`, `align.csv`) schon vorhanden sind, würde ich **nicht** mit dem Transcript-Processing beginnen, sondern direkt mit

> `transformer-representations.ipynb`

Dafür brauche ich von dir

Am hilfreichsten wären diese Informationen:

1. Die ersten Codezellen

Zeig mir die ersten **20–30 Zellen** oder zumindest

- alle `import`
- alle Konstanten (`MODEL`, `STIMULUS`, `PATH`, ...)
- die erste Stelle, an der ein `TransformerRSM(...)` erzeugt wird.

---

2. Die Definition von `TransformerRSM`

Bitte zeig

```bash
head -n 200 transformer_utils.py
```

oder wenigstens die Klasse

```python
class TransformerRSM
```

insbesondere

- `__init__`
- welche Dateien geladen werden
- welche Parameter erwartet werden.

---

3. Den Aufbau des Repos

Mich interessiert außerdem:

```bash
find . -maxdepth 2 -type f | sort
```

im Repository.

Dann sehe ich sofort, welche Python-Skripte und Daten vorhanden sind.

4. Die Daten

Ich würde gerne wissen, welche Dateien in

```
data/21st_year/
```

liegen.

Insbesondere

```
tr_tokens.csv
ling_features.csv
```

sind wahrscheinlich genau die Inputs für das Notebook.

Dann machen wir Folgendes

Ich würde die Pipeline in dieser Reihenfolge testen:

```
✓ Environment
↓
✓ Imports
↓
✓ Model laden
↓
✓ Einen Stimulus laden (pieman)
↓
✓ Embeddings berechnen
↓
✓ Transformations berechnen
↓
✓ Ergebnisse speichern
```

Erst wenn das läuft, gehen wir zum Encoding-Modell (`analysis/`).

Ich glaube, wir müssen kaum etwas ändern

Da GPT-2 bereits im Notebook vorgesehen ist:

```python
MODEL = "bert-base-uncased"
# GPT:
# gpt2
# gpt2-xl
# gpt-neo-2.7B
```

vermute ich, dass die Autoren die Pipeline bewusst generisch mit `AutoModel` aufgebaut haben. Wenn das stimmt, können wir später relativ einfach auch Modelle wie DeBERTa oder Llama einsetzen.

Schick mir als Nächstes am besten den Inhalt der ersten ~20 Zellen von `transformer-representations.ipynb` (oder alternativ den JSON-Inhalt des Notebooks, falls das einfacher ist). Dann können wir Schritt für Schritt nachvollziehen, welche Teile bereits laufen und wo es eventuell hakt.

nochmal mit readme: wie runne ich die pipeline?

Shared functional specialization in transformer-based language models and the human brain

![alt text](https://github.com/tsumers/bert-brains/blob/master/figure_1_github.png?raw=true)

This repository accompanies the manuscript "Shared functional specialization in transformer-based language models and the human brain". In this manuscript we use the internal computations of a transformer model (BERT) to predict fMRI activity while subjects listen to naturalistic spoken narratives.

Kumar, S.*, Sumers, T. R.*, Yamakoshi, T., Goldstein, A., Hasson, U., Norman, K. A., Griffiths, T. L., Hawkins, R. D., & Nastase, S. A. (2024). Shared functional specialization in transformer-based language models and the human brain. *Nature Communications (accepted)*. https://doi.org/10.1101/2022.06.08.495348

The following list describes the various analysis scripts:

Text Processing and Transformer Models
`narratives-transcript-processing.ipynb`: Used to pre-process transcripts from the `Narratives` dataset. Outputs (1) phonemes and associated nuisance variables for regression analyses; (2) TR-aligned tokens for use with Transformer notebooks.

`transformer-representations.ipynb`: the primary notebook used to generate Transformer representations (embeddings, transformations) for regression analyses.

`transformer-transformation-magnitudes.ipynb`: small script used to produce transformation magnitudes from the transformations themselves.

`transformer-utils.py`: functionality for extracting various Transformer representations, including some experimental metrics that were used in the paper.

`extract_linguistic_features.py`: Extract linguistic features (parts-of-speech and dependency tags) using spaCy.

`decode_linguistic_features.py`: Run the decoding analysis, where we decode linguistic features from the headwise transformation representations. All the necessary functions should be in decode_linguistic_features_utils.py.

Data handling
`create_fmri_dataset.py`: Copy data from narratives dataset into lab project folder.

`ROIs.ipynb`: Handling ROI labeling and visualizing.

Analysis code
`banded_ridge_regression.py`: Run encoding model analyses

(banded ridge regression) with a given representation as main features.

`calculate_boostrap_pvalue.py`: Calculate bootstrap pvalue (with FDR corrections) for individual parcels to determine significance.

`compute_isc.py`: Compute noise ceilings with intersubject correlation.

`headwise_banded_ridge_regression.py`: Headwise version of the encoding analyses (banded ridge regression, but knock out weights for all heads but one when evaluating a head).

`run_encoding_models_banded.py`: Creates a joblist.txt for banded ridge regression jobs to input as a slurm job array.

`run_encoding_models_headwise_banded.py`: Same as above, but for the headwise encoding analyses.

`run_isc_jobs.py`: Run multiple subjects' ISC analyses in parallel (slurm jobs).

`run_jupyter.sh`: Run a jupyter notebook on the cluster.

`run_pvalue_jobs.py`: Run bootstrap p-value analyses in parallel.

`run_save_jobs.py`: Run jobs in parallel that will save collapsed performance across subjects.

`save_mean_volume.py`: Collapse performance across subjects for a specific representation.

Visualizations
`Head_Dep_Brain_Plots.ipynb`: Plots whole brain results for headwise encoding (and dependency encoding).

`Layerwise_Plot.ipynb`: Plots performance across layers.

`Results_Figures.ipynb`: Notebook to plot most main results in the paper.

`View_Encoding_Results.ipynb`: Plots whole-brain glass brain results.

`bert_rdms.ipynb`: Compare RDMs for embeddings and transformations (as well as autocorrelation).

`layer_preference.ipynb`: Create layer preference and layer specificity histograms.

`plot_layer_brain.ipynb`: Process and export layer preferences for visualization on cortical surface.

`headwise_specialization.ipynb`: Run PCA on transformation weights and visualization two-dimensional projections.

Slurm jobs
`avg_job.sh`: Slurm job corresponding to save_mean_volumes.py.

`boot_job.sh`: Slurm job corresponding to calculate_bootstrap_pvalue.py.

`dsq-submit.sh`: Slurm job that submits a job array specified in a .txt file.

`joblist.txt`: Job list for submitting slurm job array.

Miscellaneous
`fmri_conda_env.yml`: yml conda env file for fmri analyses.

`transformer_conda_env.yml`: yml conda env file for transcript pre-processing and Transformer analysis notebooks.

Die README beschreibt die Pipeline leider **nicht als Workflow**, sondern nur als Sammlung von Skripten. Daraus lässt sich aber der beabsichtigte Ablauf ziemlich gut rekonstruieren.

Vollständige Pipeline

```
Nur Text

Narratives-Datensatz
        |
        ▼
1. narratives-transcript-processing.ipynb
        |
        ├── TR-aligned tokens
        └── phoneme/nuisance regressors
        |
        ▼
2. transformer-representations.ipynb
        |
        ├── Embeddings
        ├── Hidden states
        ├── Transformations
        └── Head representations
        |
        ▼
3. transformer-transformation-magnitudes.ipynb   (optional)
        |
        ▼
```

Für dich ist Schritt 1 wahrscheinlich schon erledigt

Du hast bereits

```
data/
    stimuli/
        pieman/
            align.csv
```

und

```
```

```
data/21st_year/
    tr_tokens.csv
    ling_features.csv
```

Das sind genau die Dateien, die laut README durch

```
narratives-transcript-processing.ipynb
```

erzeugt werden.

→ Diesen Schritt musst du vermutlich **nicht** erneut ausführen.

Schritt 2

Jetzt kommt das wichtigste Notebook:

```
transformer-representations.ipynb
```

Das ist laut README

> **the primary notebook**

Hier werden erzeugt

- embeddings
- hidden states
- transformations
- head representations

Das ist genau der Teil, den du später für Llama oder DeBERTa austauschen möchtest.

Schritt 3

Optional:

```
transformer-transformation-magnitudes.ipynb
```

Das berechnet lediglich Beträge (Normen) der Transformationsvektoren.

Nur nötig, wenn spätere Analysen sie verwenden.

Schritt 4

Jetzt beginnt der fMRI-Teil.

```
banded_ridge_regression.py
```

Eingaben:

- Transformer-Repräsentationen
- fMRI-Daten

Ausgabe:

- Encoding Models

Das ist praktisch der Kern des Papers.

Schritt 5

```
calculate_bootstrap_pvalue.py
```

Bestimmt Signifikanz.

Schritt 6

```
save_mean_volume.py
```

Mittelt Ergebnisse über Subjects.

Schritt 7

Danach kommen nur noch

- Layerplots
- Brainplots
- Headplots
- PCA
- Functional specialization

Welche Teile brauchst du für dein Projekt?

Ich würde sie genau so priorisieren:

Phase 1 (Reproduktion)

```
✓ transformer-representations.ipynb
```

↓

```
✓ banded_ridge_regression.py
```

↓

```
✓ Layerwise_Plot.ipynb
```

Damit hast du bereits den Hauptteil des Papers reproduziert.

Phase 2

Dann ersetzt du

```python
MODEL="bert-base-uncased"
```

durch

```python
MODEL="microsoft/deberta-v3-base"
```

oder

```python
MODEL="meta-llama/Llama-3.2-1B"
```

und lässt exakt dieselbe Pipeline erneut laufen.

Phase 3

Optional:

```
headwise_banded_ridge_regression.py
```

und

```
headwise_specialization.ipynb
```

für die Functional Specialization.

Ich würde nicht alles gleichzeitig versuchen

Ich würde die Pipeline in drei Etappen zum Laufen bringen:

✅ Etappe 1

Nur

```
```

```
transformer-representations.ipynb
```

zum Laufen bringen.

✅ Etappe 2

Nur

```
banded_ridge_regression.py
```

mit den erzeugten Repräsentationen ausführen.

✅ Etappe 3

Alle Visualisierungen erzeugen.

Was ich als Nächstes machen würde

Ich würde **nicht** mit Jupyter anfangen, sondern zunächst prüfen, **welche Dateien `transformer-representations.ipynb` erzeugt und wo sie gespeichert werden**. Danach prüfen wir, **welche Dateien `banded_ridge_regression.py` erwartet**. So können wir sicherstellen, dass die Pipeline ohne Lücken ineinandergreift. Sobald diese beiden Schnittstellen klar sind, ist der restliche Ablauf meist deutlich einfacher.

> ok. also erst mal transformer-representations.ipynb rerunnen. welche infos brauchst du?

Genau. Das ist der richtige erste Schritt. Wenn `transformer-representations.ipynb` erfolgreich läuft, hast du den schwierigsten Teil der späteren Modell-Erweiterung bereits geschafft.

Um dir gezielt helfen zu können, brauche ich **nicht das ganze Notebook**, sondern nur die Stellen, an denen konfiguriert wird und Daten geladen werden.

1. Notebook-Struktur (am wichtigsten)

Bitte zeig die ersten Zellen des Notebooks:

- alle `import`-Zellen
- alle Konfigurationszellen (`MODEL`, `STIMULUS`, Pfade, Flags)

- die erste Zelle, in der `TransformerRSM(...)` erzeugt wird.

Falls du das Notebook auf dem Cluster hast, kannst du alternativ die Zellinhalte mit einem kleinen Python-Skript ausgeben oder die relevanten Ausschnitte hier hineinkopieren.

---

2. `transformer_utils.py`

Davon brauche ich insbesondere:

- die Klasse `TransformerRSM`
- den `__init__`
- die Methoden, die
  - den Tokenizer laden,
  - das Modell laden,
  - Dateien speichern.

Du hattest schon gezeigt:

```python
self.tokenizer = AutoTokenizer.from_pretrained(...)
self.transformer = AutoModel.from_pretrained(...)
```

Jetzt interessiert mich der Rest der Klasse.

---

3. Ausgabeverzeichnis

Ich möchte wissen, **wo das Notebook seine Ergebnisse speichert**.

Dazu reicht meist ein:

```bash
grep -R "to_pickle\|np.save\|torch.save\|to_csv\|save" .
```

oder wir sehen es direkt im Notebook.

---

4. Gibt es bereits erzeugte Repräsentationen?

Bitte einmal im Repository suchen:

```bash
find . -type f | grep -E "embedding|transform|hidden|representation|rsm|pkl|npy|pt"
```

Dann sehen wir, ob vielleicht schon Outputs vorhanden sind.

---

5. Falls vorhanden: erste Fehlermeldung

Wenn du versuchst, das Notebook auszuführen und es scheitert, ist die **erste Exception** meist der schnellste Weg zur Lösung.

Mein Plan

Ich würde das Notebook nicht "blind" starten, sondern zuerst verstehen:

1. **Welche Eingabedateien** erwartet es?
2. **Welche Ausgabedateien** erzeugt es?
3. **Welche Modellrepräsentationen** (Embeddings, Hidden States, Head-Outputs, Transformations) werden tatsächlich gespeichert?

Das ist wichtig, weil wir später genau diese Stelle für DeBERTa oder Llama anpassen werden.

Am hilfreichsten wäre jetzt der Anfang von `transformer-representations.ipynb` (Importe + Konfiguration + erste Ausführungszellen) sowie die Definition der Klasse `TransformerRSM` in `transformer_utils.py`. Damit können wir den Ablauf Schritt für Schritt nachvollziehen.

Eingefügter Text(1...
Dokument

```
# Ensure that we re-load changes automagically
%load_ext autoreload
%autoreload 2

import itertools
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pylab as plt

from transformer_utils import TransformerRSM
```

```
STIMULUS = "pieman" # 21st_year, pieman, slumlordreach, black

MODEL = 'bert-base-uncased'
# BERT: bert-base-uncased, bert-large-uncased
# GPT: gpt2, gpt2-xl, gpt-neo-2.7B

foo = TransformerRSM(STIMULUS, model_name=MODEL,
verbose=True)
ACTIVATIONS = True
Z_REPS = True

foo.process_stimulus_activations(save_activations=ACTIVATIONS,
save_z_reps=Z_REPS)

N_LAYERS = len(foo.stimulus_df["activations"][0])

for layer in range(0, N_LAYERS-1):

    if ACTIVATIONS:
        layer_activations =
foo.layer_activations_from_tensor(foo.stimulus_df["activations"],
layer)
        layer_mean_embeddings = np.array([l.mean(0).numpy() for l
in layer_activations])
        np.save("{}_{}_layer_{}_activations.npy".format(STIMULUS,
MODEL, layer), layer_mean_embeddings)
        print("Layer {}: saved mean embeddings {}".format(layer,
layer_mean_embeddings.shape))

    if Z_REPS:
        layer_z_reps =
foo.layer_activations_from_tensor(foo.stimulus_df["z_reps"],
layer)
        layer_mean_z_reps = np.array([l.mean(0).numpy() for l in
layer_z_reps])
        np.save("{}_{}_layer_{}
_z_representations.npy".format(STIMULUS, MODEL, layer),
layer_mean_z_reps)
        print("Layer {}: saved mean z-reps {}".format(layer,
```

layer_mean_z_reps.shape))

layer_glove = foo.layer_activations_from_tensor(foo.stimulus_df["glove"], 0)
np.save("{}_{}_layer_{}_glove.npy".format(STIMULUS, MODEL, 0), layer_glove)

ausschnitt aus dem notebook.

transformer_utils.py :

Perfekt. Das Notebook selbst sieht erstmal völlig normal aus. Um es wirklich zum Laufen zu bringen, brauche ich jetzt **den Inhalt von `transformer_utils.py`**, denn dort passiert praktisch alles:

```python
foo = TransformerRSM(STIMULUS, model_name=MODEL, verbose=True)
foo.process_stimulus_activations(...)
```

Insbesondere interessieren mich:

- `class TransformerRSM`
- `__init__`
- `process_stimulus_activations`
- alle Stellen, wo Dateien eingelesen werden (`pd.read_csv`, `np.load`, `open(...)`)
- alle Stellen mit `AutoTokenizer.from_pretrained` und `AutoModel.from_pretrained`

Am einfachsten

Schick einfach die Datei hoch:

```
text_processing_and_transformers/transformer_utils.py
```

Dann kann ich dir genau sagen

- ob sie auf dem Cluster sofort läuft,
- welche Dateien sie erwartet,
- wo die Outputs landen,

- und wie wir anschließend statt BERT einfach Llama/T5/RoBERTa einsetzen.

Alternativ auf dem Cluster

Falls du nichts hochladen möchtest:

```bash
sed -n '1,220p' transformer_utils.py
```

und danach

```bash
sed -n '220,500p' transformer_utils.py
```

(Dann sehe ich die ganze Datei.)

Ich vermute übrigens schon jetzt den Ablauf:

1. `align.csv` laden (`data/stimuli/pieman/align.csv`)
2. Tokenisierung
3. `AutoModel.from_pretrained("bert-base-uncased")`
4. Hidden States aller Layer berechnen
5. Pro TR mitteln
6. `.npy`-Dateien speichern
7. Diese `.npy` werden anschließend von den Encoding-Modellen verwendet.

Sobald ich `transformer_utils.py` sehe, kann ich dir den kompletten Datenfluss erklären und anschließend zeigen, welche wenigen Zeilen geändert werden müssen, um statt BERT ein aktuelleres Modell zu verwenden.

Eingefügter Text(1...
Dokument

hier transformer_utils.py

Super, das ist genau die Datei, die wir brauchten. Jetzt kann man den gesamten Pipeline-Ablauf nachvollziehen. Die gute Nachricht: **dein geplantes Projekt ("BERT → neueres Modell") ist genau an dieser Stelle umzusetzen.**

Was passiert in `transformer_utils.py`?

1. Das Notebook erzeugt

```python
foo = TransformerRSM(STIMULUS, model_name=MODEL)
```

Dadurch wird zunächst

```python
__init__()
```

aufgerufen.

---

2. Es werden die TR-Tokens geladen

```python
self.stimulus_df = self._load_stimulus(...)
```

Das lädt

```
data/stimuli/pieman/tr_tokens.csv
```

oder

```
data/stimuli/21st_year/tr_tokens.csv
```

je nach

```python
```

```
STIMULUS = "pieman"
```

Diese Datei enthält bereits die Wörter, die zu jedem fMRI-TR gehören. Eingefügter Text

---

3. Tokenizer laden

```python
self.tokenizer = AutoTokenizer.from_pretrained(model_name)
```

also z.B.

```
bert-base-uncased
```

oder

```
gpt2
```

---

4. Modell laden

```python
AutoModel.from_pretrained(
    model_name,
    output_hidden_states=True,
    output_attentions=True
)
```

Das ist sehr praktisch:

Du bekommst automatisch

- Hidden States
- Attention

aller Layer. Eingefügter Text

Danach beginnt die eigentliche Pipeline

Das Notebook ruft

```python
foo.process_stimulus_activations()
```

auf.

Für jeden einzelnen TR

Es wird über

```python
for i, tr in enumerate(tr_chunked_tokens):
```

iteriert. Eingefügter Text

Kontextfenster

Für jeden TR werden

```python
20
```

vorherige TRs angehängt:

```python
context_window_index_start = max(0, i-20)
window_stimulus = ...
```

Das entspricht ungefähr

> 30 Sekunden Kontext

wie im Paper beschrieben. Eingefügter Text

Tokenisierung

Dann

```python
window_token_ids =
tokenizer.encode(...)
```

und

```python
tr_token_ids
```

für den aktuellen TR.

Forward Pass

Dann kommt

```python
embeddings, _ = self.transformer(window_token_ids)[-2:]
```

Das liefert

```
Layer0
Layer1
...
Layer12
```

für BERT.

Eingefügter Text

Das Paper benutzt zwei Repräsentationen

1. embeddings

Das sind einfach

```
hidden_states
```

aller Layer.

Sie werden später gespeichert als

```
activations
```

---

2. z_reps

Hier wird es interessant.

Für BERT

```python
self.transformer.encoder.layer[layer].attention.self(layer)[0]
```

aufgerufen. Eingefügter Text

Das ist **nicht**

```
attention weights
```

sondern der Output

```
Q
K
V
```

```
↓

Attention(Q,K,V)

↓

z
```

also genau die

Transformationen

über die im Paper gesprochen wird.

Danach

Es wird nur noch

```python
tr_activations =
layer[0][...]
```

genommen.

Also nur die Token des aktuellen TRs.

Nicht der ganze Kontext.

Dann

```python
layer_mean_embeddings =
l.mean(0)
```

Das mittelt alle Token eines TRs.

Genau diese Vektoren gehen später ins Encoding Model.

Zusätzlich

Es wird

```python
spaCy
```

geladen

```python
en_core_web_lg
```

und daraus

```python
nlp(tr).vector
```

berechnet. Eingefügter Text Eingefügter Text

Das ist der GloVe-Baseline-Vektor.

Wo werden die Dateien gespeichert?

Im Notebook:

```python
np.save(...)
```

erzeugt Dateien wie

```
pieman_bert-base-uncased_layer_0_activations.npy

pieman_bert-base-uncased_layer_0_z_representations.npy

...
```

Diese `.npy`-Dateien werden anschließend von den Encoding-Modellen eingelesen.

Das ist der wichtigste Punkt für dein Projekt

Die gesamte Pipeline ist **nicht auf BERT fest verdrahtet**.

Sie benutzt

```python
AutoTokenizer
```

und

```python
AutoModel
```

Das bedeutet:

- RoBERTa
- DeBERTa
- ModernBERT
- T5-Encoder
- andere Hugging-Face-Encoder

lassen sich prinzipiell genauso laden.

Aber: Es gibt eine Einschränkung.

Der Code enthält modellabhängige Zweige:

```python
if 'bert' in model_name:
    ...
elif 'gpt' in model_name:
    ...
```

Diese Logik steuert u. a.:

- den Umgang mit Spezialtokens,
- die Berechnung der `z_reps`,

- den Zugriff auf die internen Attention-Module. Eingefügter Text Eingefügter Text

Für **RoBERTa**, **DeBERTa** oder **ModernBERT** müsste dieser Teil angepasst werden, obwohl `AutoModel` das Laden selbst übernimmt.

Mein Vorschlag

Ich würde zunächst genau die Reihenfolge einhalten, die auch die Autoren vermutlich verwendet haben:

1. **`transformer-representations.ipynb`** mit `bert-base-uncased` erfolgreich auf dem Cluster laufen lassen.
2. Prüfen, dass alle `.npy`-Dateien korrekt erzeugt werden.
3. Erst danach das Modell auf z. B. **ModernBERT** oder **DeBERTa-v3** umstellen und die wenigen nötigen Änderungen in `transformer_utils.py` vornehmen.

So hast du zuerst eine vollständig reproduzierte Baseline und kannst anschließend die Modelländerung sauber evaluieren.

wo liegen die erzeugnten .npy daten?

Das hängt davon ab, **von wo aus du das Notebook gestartet hast**. In deinem Notebook wird einfach

```python
np.save("{}_{}_layer_{}_activations.npy".format(STIMULUS, MODEL, layer),
        layer_mean_embeddings)
```

aufgerufen. Es wird **kein Pfad angegeben**, also speichert `np.save()` im **aktuellen Working Directory**.

1. Finde das Working Directory heraus

Führe im Notebook aus:

```python
import os
os.getcwd()
```

Das gibt z.B. zurück:

```
Nur Text
```

```
/nethome/phussein/brains/bert-brains/text_processing_and_transformers
```

Dann liegen die Dateien genau dort.

---

2. Falls du das Notebook auf dem Cluster gestartet hast

Sehr wahrscheinlich befinden sie sich hier:

```
Nur Text

/nethome/phussein/brains/bert-brains/text_processing_and_transformers/
```

Du kannst prüfen:

```
Bash

cd /nethome/phussein/brains/bert-brains/text_processing_and_transformers

ls *.npy
```

oder

```
Bash

find . -name "*.npy"
```

---

3. Welche Dateien sollten entstehen?

Für

```
Python

STIMULUS = "pieman"
MODEL = "bert-base-uncased"
```

erzeugt das Notebook ungefähr

```


pieman_bert-base-uncased_layer_0_activations.npy
pieman_bert-base-uncased_layer_1_activations.npy
```

```
...
pieman_bert-base-uncased_layer_12_activations.npy

pieman_bert-base-uncased_layer_0_z_representations.npy
...
pieman_bert-base-uncased_layer_12_z_representations.npy

pieman_bert-base-uncased_layer_0_glove.npy
```

(bei BERT Base insgesamt 13 Layer inklusive Embedding-Layer).

---

4. Für dein späteres Projekt

Wenn du dieselbe Pipeline mit z.B.

```python
MODEL = "FacebookAI/roberta-base"
```

oder

```python
MODEL = "bert-large-uncased"
```

laufen lässt, entstehen automatisch Dateien wie

```
pieman_FacebookAI_roberta-base_layer_0_activations.npy
```

bzw.

```
pieman_bert-large-uncased_layer_0_activations.npy
```

Damit kannst du die komplette Encoding-Pipeline unverändert weiterverwenden – sie bekommt einfach die neuen Repräsentationen.

Eine Frage noch: Läuft das Notebook bei dir inzwischen komplett durch, oder bist du noch vor `foo.process_stimulus_activations(...)` hängen geblieben?

(base) [phussein@submit brains]$ find . -name "*.npy"
(base) [phussein@submit brains]$

Das bedeutet, dass **keine `.npy`-Dateien erzeugt wurden**. Dafür gibt es im Wesentlichen drei Möglichkeiten:

1. **Das Notebook wurde nie bis zum Ende ausgeführt.**
2. **`process_stimulus_activations()` ist vorher mit einem Fehler abgebrochen.**
3. **Du hast das Notebook noch gar nicht auf dem Cluster ausgeführt** (nur das Environment eingerichtet).

Um herauszufinden, was zutrifft, brauche ich folgende Informationen:

1. Hast du das Notebook bereits ausgeführt?

Oder hast du bisher nur das Conda-Environment erstellt?

---

2. Falls du es ausgeführt hast: Wie startest du es?

- lokal mit Jupyter?
- über `run_jupyter.sh` auf dem Cluster?
- oder hast du den Code in ein Python-Skript kopiert?

---

3. Was passiert bei

```python
foo = TransformerRSM(STIMULUS, model_name=MODEL, verbose=True)

foo.process_stimulus_activations(
    save_activations=True,
    save_z_reps=True
)
```

- Kommt eine Fehlermeldung?
- Oder läuft es minutenlang?
- Oder endet es mit `Done`?

---

4. Zeig mir bitte den Anfang von `transformer_utils.py`, insbesondere:

```python
class TransformerRSM:
```

und die Methoden

```python
__init__
process_stimulus_activations
```

Die sind entscheidend, um zu sehen:

- wo die Daten eingelesen werden,
- ob etwas gespeichert wird,
- und welche Voraussetzungen erfüllt sein müssen.

Ich vermute im Moment, dass wir **noch gar nicht beim eigentlichen Pipeline-Lauf angekommen sind**. Das Environment funktioniert jetzt, aber als Nächstes müssen wir den ersten Notebook-Schritt erfolgreich durchlaufen lassen. Sobald `process_stimulus_activations()` einmal ohne Fehler durchläuft, sollten die `.npy`-Dateien erscheinen.

> ich habe es noch gar nicht ausgeführt. heißt aber die autoren haben die .npy nicht im repo mitgeliefert

Genau. Das ist sogar ziemlich typisch für solche Repositories.

Die `.npy`-Dateien sind **Zwischenergebnisse**, die aus dem Notebook `transformer-representations.ipynb` erzeugt werden. Da sie relativ groß sind (für jede Story × jedes Modell × jeden Layer × Aktivierungen/Z-Repräsentationen), werden sie normalerweise **nicht** mit ins Git-Repository aufgenommen.

Das spricht auch dafür, dass

```python
np.save(...)
```

im Notebook explizit aufgerufen wird – die Autoren erwarten also, dass du diese Dateien selbst erzeugst.

Wahrscheinlicher Workflow der Autoren

1. **Narratives-Daten vorbereiten**
   - `narratives-transcript-processing.ipynb`
   - erzeugt `tr_tokens.csv` usw.
2. **Transformer-Repräsentationen berechnen**
   - `transformer-representations.ipynb`
   - erzeugt viele `.npy`-Dateien
     - Layer-Embeddings
     - Layer-Z-Repräsentationen
     - GloVe
3. **Encoding Models**
   - `analysis/banded_ridge_regression.py`
   - lädt genau diese `.npy`-Dateien wieder ein und verwendet sie als Features.

Bei dir sieht es sogar gut aus

Du hast bereits

```
data/
└── stimuli/
    └── pieman/
        tr_tokens.csv
```

und für `21st_year` ebenfalls `tr_tokens.csv`.

Das heißt, **Schritt 1 ist bereits im Repository enthalten**. Du musst die Transkripte also wahrscheinlich nicht neu preprocessen.

Du kannst direkt mit

```
transformer-representations.ipynb
```

beginnen.

Das würde ich als Nächstes machen

1. Notebook auf dem Cluster starten (oder in VS Code mit dem Cluster-Kernel).
2. Erst einmal nur

```python
STIMULUS = "pieman"
MODEL = "bert-base-uncased"
```

laufen lassen.

3. Prüfen, dass anschließend Dateien wie

```
pieman_bert-base-uncased_layer_0_activations.npy
...
```

existieren.

Erst wenn das funktioniert, ersetzen wir

```python
MODEL = "bert-base-uncased"
```

durch z.B.

```python
MODEL = "FacebookAI/roberta-base"
```

oder später ein moderneres Modell. Dann hast du bereits die komplette Pipeline für dein Projekt.

Eine Sache würde ich vorher noch prüfen: Ich würde mir gern die Implementierung von `TransformerRSM` ansehen, insbesondere `process_stimulus_activations()`. Dann können wir vorhersagen, **wo** genau die `.npy` gespeichert werden und ob für neuere Modelle (RoBERTa, DeBERTa usw.) überhaupt Codeanpassungen nötig sind.

ok. wie mache ich mein brains.sh sodass es das notebook mit bert reproduziert?

Ja. Ich würde das **nicht** über Jupyter laufen lassen, sondern das Notebook einmal in ein Python-Skript konvertieren und dieses per Condor starten. Das ist reproduzierbarer und einfacher zu debuggen.

Option 1 (empfohlen): Notebook als Python-Skript ausführen

Falls das Notebook noch nicht als `.py` existiert:

```bash
jupyter nbconvert --to script transformer-representations.ipynb
```

Dann entsteht

```
transformer-representations.py
```

Dein `brains.sh` könnte dann so aussehen:

```bash
#!/bin/bash

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh
conda activate bert-brains

cd /nethome/phussein/brains/bert-brains/text_processing_and_transformers

echo "Python:"
python --version

echo "Running transformer representations..."

python transformer-representations.py

echo "Done."
```

Option 2: Notebook direkt ausführen (geht auch)

Falls du das Notebook unverändert laufen lassen möchtest:

```bash
#!/bin/bash

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh
conda activate bert-brains

cd /nethome/phussein/brains/bert-brains/text_processing_and_transformers

jupyter nbconvert \
    --to notebook \
    --execute transformer-representations.ipynb \
    --output transformer-representations_rerun.ipynb
```

Das führt alle Zellen aus und speichert ein ausgeführtes Notebook.

Ich würde allerdings vorher noch eine Kleinigkeit ändern

Im Notebook steht momentan

```python
STIMULUS = "pieman"
MODEL = "bert-base-uncased"
```

Das ist in Ordnung.

Ich würde aber zusätzlich **ganz am Anfang** eine Zelle einfügen:

```python
import os

print(os.getcwd())
```

und nach

```python
foo.process_stimulus_activations(...)
```

eine weitere:

```python
import glob

print(glob.glob("*.npy"))
```

Dann siehst du im Condor-Log sofort

- ob das Notebook wirklich durchgelaufen ist,
- in welchem Verzeichnis gespeichert wurde,
- welche `.npy`-Dateien erzeugt wurden.

Eine Frage noch

Kannst du einmal

```bash
ls /nethome/phussein/brains/bert-brains/text_processing_and_transformers
```

ausführen? Mich interessiert, ob dort bereits **`transformer-representations.py`** existiert oder nur das Notebook. Das entscheidet, ob wir direkt ein Python-Skript starten können oder erst `nbconvert` verwenden müssen.

> (base) [phussein@submit brains]$ jupyter nbconvert --to script transformer-representations.ipynb
> -bash: jupyter: command not found
> (base) [phussein@submit brains]$

Das ist normal. Du bist im **base**-Environment, und dort ist Jupyter offenbar nicht installiert.

Da du das Notebook ohnehin im `bert-brains`-Environment ausführen willst, hast du zwei Möglichkeiten.

Möglichkeit 1 (einfach)

Aktiviere zuerst das Environment:

```bash
conda activate bert-brains
```

Dann prüfe:

```bash
which jupyter
```

Falls Jupyter dort installiert ist, kannst du anschließend

```bash
jupyter nbconvert --to script transformer-representations.ipynb
```

ausführen.

Möglichkeit 2 (ohne Aktivierung)

Du kannst Jupyter direkt über Python aufrufen:

```bash
conda run -n bert-brains python -m jupyter nbconvert \
    --to script transformer-representations.ipynb
```

Ich würde aber vorher noch etwas prüfen

Kannst du einmal Folgendes ausführen?

```bash
conda activate bert-brains

which python
which jupyter

python -c "import notebook, jupyterlab; print('Jupyter vorhanden')"
```

Denn im ursprünglichen `transformer_conda_env.yml` waren sowohl

```
jupyter
```

```
jupyterlab
notebook
```

enthalten. Sie **sollten** also vorhanden sein. Wenn `which jupyter` nichts findet, ist bei der Environment-Erstellung etwas schiefgelaufen.

Noch etwas Wichtiges

Mir ist gerade aufgefallen, dass das Repository bereits **mehrere `.ipynb`-Notebooks** enthält. Bevor wir anfangen, sie in `.py` umzuwandeln, würde ich mir ansehen, wie die Autoren sie tatsächlich auf dem Cluster ausgeführt haben. Im README gibt es nämlich:

```
run_jupyter.sh
```

und im Repository liegt auch ein `slurm/`-Ordner.

Es könnte gut sein, dass die Autoren die Notebooks **direkt** auf dem Cluster ausgeführt haben und gar keine `.py`-Dateien verwendet haben.

Kannst du einmal zeigen:

```
</> Bash

cd /nethome/phussein/brains/bert-brains/slurm

ls
cat run_jupyter.sh
```

oder, falls `run_jupyter.sh` nicht dort liegt:

```
</> Bash

find /nethome/phussein/brains/bert-brains -name "run_jupyter.sh"
```

Dann können wir wahrscheinlich genau den vorgesehenen Workflow des Repositories verwenden.

(base) [phussein@submit brains]$ ls /nethome/phussein/brains/bert-brains/text_processing_and_transformers
decode_linguistic_features.py narratives-transcript-processing.ipynb transformer_utils.py
decode_linguistic_features_utils.py transformer-

representations.ipynb
extract_linguistic_features.py transformer-transformation-magnitudes.ipynb
(base) [phussein@submit brains]$

Perfekt. Dann gibt es **keine Python-Version** des Notebooks. Das ist bei Forschungs-Repos nicht ungewöhnlich.

Ich würde das Notebook **nicht** in ein Skript konvertieren, sondern zunächst genau so ausführen, wie es gedacht ist. So reproduzierst du die Ergebnisse der Autoren am zuverlässigsten.

Schritt 1: Jupyter im Environment testen

Du bist noch im `base`-Environment. Wechsle zuerst:

```bash
conda activate bert-brains
```

Dann prüfe:

```bash
which python
which jupyter
jupyter --version
```

Falls `jupyter` vorhanden ist, können wir das Notebook direkt per Batch ausführen.

Schritt 2: Notebook per Batch ausführen

Dein `brains.sh` könnte dann so aussehen:

```bash
#!/bin/bash

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh
conda activate bert-brains

cd /nethome/phussein/brains/bert-brains/text_processing_and_transformers

jupyter nbconvert \
    --to notebook \
```

```
    --execute transformer-representations.ipynb \
    --output transformer-representations_executed.ipynb
```

Das führt jede Zelle nacheinander aus. Falls ein Fehler auftritt, erscheint er im Condor-Log.

Falls `jupyter` nicht gefunden wird

Dann prüfen wir das Environment. Das wäre ungewöhnlich, weil dein `transformer_conda_env.yml` Jupyter eigentlich installiert.

Eine Sache würde ich vorher noch überprüfen

In den ersten Notebook-Zellen steht

```python
STIMULUS = "pieman"
MODEL = "bert-base-uncased"
```

Das ist gut.

Mich interessiert jetzt aber **die nächste Zelle** bzw. der Anfang von `TransformerRSM.__init__()`. Dort wird nämlich festgelegt,

- wo `align.csv` gesucht wird,
- wo `tr_tokens.csv` gesucht wird,
- und in welches Verzeichnis die `.npy` gespeichert werden.

Kannst du den Anfang von `transformer_utils.py` zeigen – ungefähr die ersten **70–100 Zeilen**, also bis einschließlich

```python
class TransformerRSM:
    def __init__(...)
```

Dann können wir sicherstellen, dass das Notebook auf dem Cluster sofort die richtigen Dateien findet und nicht wegen eines Pfadproblems scheitert.

Di., 1. Sept. um 16:28

dass ich etwas am projekt gemacht habe, ist ein paar Wochen her. bring mich auf den letzten Stand. brauchst du infos von mir die ich aus dem repo raussuchen kann ?

(base) [phussein@submit brains]$ ls
CBIG create_env.sh fmri_encoding.sub Schaefer2018_1000Parcels_17Networks_order_FSLMNI152_2mm.nii.gz create_env.sub get_afni.sh
bert-brains datalad.sh get_afni.sub
brains.sh datalad.sub transformer_conda_env_linux.yml
brains.sub

Ja — ich kann dich schon ziemlich gut auf den letzten Stand bringen. Aus unserem bisherigen Verlauf ergibt sich ungefähr dieser Stand:

Wo du mit dem Projekt stehst

Projektbasis: Du reproduzierst Sumers et al. / Kumar et al. (2024), *Shared functional specialization in transformer-based language models and the human brain*.

Deine Projektidee ist:

1. **Zuerst die originale Pipeline mit BERT exakt reproduzieren.**
2. Danach dieselbe Pipeline mit einem **neueren Transformer-Modell** durchführen und untersuchen, ob dessen Repräsentationen bessere Prädiktoren für Gehirnaktivität sind.
3. Als **Bonus**: untersuchen, ob das neuere Modell ebenfalls funktionale Spezialisierung einzelner Heads zeigt.

Die Motivation ist, dass ältere Modelle wie BERT bekannte Schwächen bei semantischen Repräsentationen haben, z.B. bei Negation. Deine Hypothese ist also grob:

> Neuere Modelle könnten semantisch reichhaltigere interne Repräsentationen haben, die stärker mit neuronaler Aktivität zusammenhängen.

Was wir technisch bereits gemacht haben

Dein Cluster läuft auf **Linux**, und du verwendest **HTCondor**, nicht Slurm.

Das Repository liegt hier:

```
Nur Text

/nethome/phussein/brains/bert-brains
```

und enthält u.a.:

```
Nur Text

analysis/
data/
data_handling/
text_processing_and_transformers/
visualizations/
```

sowie die Condor-Dateien.

Du hast außerdem die nötigen Daten bereits im Repository/Projektverzeichnis.

Conda-Environment

Das ursprüngliche `transformer_conda_env.yml` war von einem **Mac** und enthielt deshalb Dinge wie:

```
Nur Text

appnope
libcxx
```

sowie Mac-spezifische Build-Strings.

Wir haben daraus eine Linux-kompatible Version gemacht:

```
Nur Text

transformer_conda_env_linux.yml
```

Die Environment-Erstellung hat schließlich **erfolgreich funktioniert**.

Das Environment heißt:

```
Nur Text
```

```
bert-brains
```

und wir haben überprüft, dass darin ungefähr die ursprüngliche Softwareumgebung vorhanden ist:

```
Nur Text

Python       3.9.23
PyTorch      1.9.1.post3
Transformers 4.9.1
spaCy        2.3.5
```

Das ist wichtig, weil wir zunächst **nicht** auf moderne Package-Versionen upgraden wollen. Für die Reproduktion soll die alte Umgebung möglichst erhalten bleiben.

Dein aktueller Cluster-Aufbau

Du hast momentan:

```
Nur Text

/nethome/phussein/brains/

├── bert-brains/
├── create_env.sh
├── create_env.sub
├── brains.sh
├── brains.sub
├── fmri_encoding.sub
├── transformer_conda_env_linux.yml
├── ...
```

Das Environment wird also über Condor erzeugt.

Dein Testjob für das Environment lief erfolgreich durch:

```
Nur Text

Solving environment: done
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
...
Successfully installed ...
transformers-4.9.1
...
```

```
done
```

Wo wir unmittelbar vor der Pause waren

Wir wollten jetzt **nicht sofort das neue Modell testen**, sondern zuerst die Originalpipeline laufen lassen:

```
Nur Text

transformer-representations.ipynb
```

mit

```
Python

STIMULUS = "pieman"
MODEL = "bert-base-uncased"
```

Das Notebook erzeugt die Transformer-Repräsentationen, die anschließend von der Encoding-Pipeline verwendet werden.

Wichtig: Die `.npy`-Dateien sind **nicht im GitHub-Repo vorhanden**. Das hatten wir gerade festgestellt.

Sie werden durch das Notebook erzeugt.

Was ich jetzt von dir brauche

Da ein paar Wochen vergangen sind, würde ich **nicht raten**, was inzwischen auf dem Cluster passiert ist.

Schick mir bitte diese wenigen Sachen vom Cluster:

1. Deine aktuellen Scripts

```
Bash

cd /nethome/phussein/brains

cat brains.sh
cat brains.sub
```

2. Die aktuelle Repository-Struktur

```bash
cd /nethome/phussein/brains/bert-brains

find text_processing_and_transformers -maxdepth 1 -type f
find analysis -maxdepth 1 -type f
```

3. Aktuellen Status der Environment

```bash
conda activate bert-brains

python --version
python -c "import torch, transformers, spacy; print(torch.__version__); print(transformers.__version__)
```

4. Und besonders wichtig: die aktuelle `transformer_utils.py`

```bash
sed -n '1,180p' text_processing_and_transformers/transformer_utils.py
```

Das reicht zunächst.

Dann kann ich dich **von genau diesem Stand aus Schritt für Schritt weiterführen**, statt wieder alles von vorne aufzubauen.

Unser unmittelbares Ziel wäre dann:

```
Condor
  ↓
bert-brains environment
  ↓
transformer-representations.ipynb
  ↓
BERT-base
  ↓
pieman
```

```
  ↓
.npy Repräsentationen
  ↓
Encoding pipeline
  ↓
fMRI prediction
```

Und **erst wenn diese Reproduktion funktioniert**, gehen wir an dein eigentliches Forschungsprojekt mit dem neueren Modell.

bert-brains/ brains.sh brains.sub
(base) [phussein@submit brains]$ cat brains.sh

```
#!/bin/bash

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh
conda activate bert-brains

cd /nethome/phussein/brains/bert-brains

echo "=== Creating fMRI dataset ==="
python data_handling/create_fmri_dataset.py pieman
# python data_handling/create_fmri_dataset.py slumlordreach
# python data_handling/create_fmri_dataset.py black

# cd /nethome/phussein/brains/bert-brains/
text_processing_and_transformers

# export JUPYTER_PLATFORM_DIRS=1
# export TQDM_POSITION=-1
# export IPYTHON_SUPPRESS_TQDM_WARNING=1

# jupyter nbconvert \
#     --to notebook \
#     --execute narratives-transcript-processing.ipynb \
#     --output narratives-transcript-processing-run.ipynb

# cd /nethome/phussein/brains/bert-brains

# jupyter nbconvert \
#   --to notebook \
#   --execute text_processing_and_transformers/transformer-
```

representations.ipynb \
--output transformer-representations_executed.ipynb(base)
[phussein@submit brains]$ cat brains.sub
##
####################
HTCondor submit description file
Everything with a leading # is a comment

initialdir   = /nethome/phussein/brains
executable   = brains.sh
output   = /scratch/phussein/logs/brains/brains.sh.
$(ClusterId).$(Year)_$(Month)_$(Day)_$(SUBMIT_TIME).out
error   = /scratch/phussein/logs/brains/brains.sh.
$(ClusterId).$(Year)_$(Month)_$(Day)_$(SUBMIT_TIME).err
log   = /scratch/phussein/logs/brains/brains.sh.
$(ClusterId).$(Year)_$(Month)_$(Day)_$(SUBMIT_TIME).log
request_CPUs   = 1
request_memory   = 64G
request_GPUs   = 1
requirements   = (GPUs_Capability
>=8.0)&&(GPUs_GlobalMemoryMb > 60000)
#environment = WANDB_API_KEY=46736878....
getenv = True
queue 1
##
####################
(base) [phussein@submit brains]$

Ja, jetzt ist der Stand klar. Du hast die Pipeline **noch nicht ausgeführt**; dein `brains.sh` macht momentan nur den ersten Schritt:

```bash
python data_handling/create_fmri_dataset.py pieman
```

Die beiden Notebook-Schritte sind auskommentiert.

Was dein aktuelles `brains.sh` macht

Aktuell:

```
Condor Job
   ↓
bert-brains Environment
   ↓
create_fmri_dataset.py pieman
   ↓
fertig
```

Auskommentiert ist momentan:

```
narratives-transcript-processing.ipynb
        ↓
transformer-representations.ipynb
```

Das ist auch ein sinnvoller Zwischenstand: **Das Environment funktioniert und der fMRI-Datensatz wird vorbereitet.**

Was wir jetzt machen sollten

Ich würde **nicht beide Notebooks gleichzeitig** aktivieren. Wir gehen Schritt für Schritt vor.

Schritt 1: Prüfen, ob `create_fmri_dataset.py` schon erfolgreich gelaufen ist

Auf dem Cluster:

```bash
ls -lh /nethome/phussein/brains/bert-brains/data/21st_year
```

und

```bash
ls -lh /nethome/phussein/brains/bert-brains/data/stimuli/pieman
```

Außerdem:

```bash
```

```
find /nethome/phussein/brains/bert-brains/data -name "*.npy"
```

Wir erwarten momentan wahrscheinlich **noch keine** `.npy`, weil das Transformer-Notebook noch nicht ausgeführt wurde.

Schritt 2: Transcript Processing

Bevor wir `transformer-representations.ipynb` ausführen, müssen wir klären, ob

```
Nur Text

narratives-transcript-processing.ipynb
```

für `pieman` überhaupt noch ausgeführt werden muss.

Das ist wichtig, weil das README sagt:

> `narratives-transcript-processing.ipynb`: ... Outputs ... TR-aligned tokens for use with Transformer notebooks.

Du hast zwar bereits `tr_tokens.csv` in deinen Daten gesehen, aber wir sollten **nicht einfach annehmen**, dass es die richtige/aktuelle Datei ist.

Dafür brauche ich jetzt nur einen kleinen Check:

```
Bash

find /nethome/phussein/brains/bert-brains/data -name "tr_tokens.csv" -o -name "align.csv"
```

und:

```
Bash

ls -lh /nethome/phussein/brains/bert-brains/data/stimuli/pieman
```

Danach kommt der eigentliche interessante Teil

Wenn die benötigten Inputdaten vorhanden sind, aktivieren wir zunächst **nur**:

```bash
jupyter nbconvert \
    --to notebook \
    --execute text_processing_and_transformers/transformer-representations.ipynb \
    --output transformer-representations_executed.ipynb
```

mit

```python
MODEL = "bert-base-uncased"
STIMULUS = "pieman"
```

Dann sollten die BERT-Repräsentationen erzeugt werden.

Erst wenn das erfolgreich ist, kümmern wir uns um:

```
.npy
 ↓
banded ridge regression
 ↓
brain prediction
 ↓
layer-wise results
```

Und das ist dann genau die **Reproduktionsbaseline**, die wir später für dein neues Modell brauchen.

Schick mir also als Nächstes die Ausgabe der beiden `find/ls`-Befehle oben. Dann sage ich dir konkret, ob wir `narratives-transcript-processing.ipynb` noch laufen lassen müssen oder direkt zu `transformer-representations.ipynb` gehen können.

(base) [phussein@submit brains]$ ls -lh /nethome/phussein/brains/bert-brains/data/21st_year
total 1.8M
-rw------- 1 phussein phussein 122K Jul 31 17:23 21st_year.csv
-rw------- 1 phussein phussein 298K Jul 31 17:23 21st_year.xlsx
-rw------- 1 phussein phussein 122K Jul 31 17:23 21st_year_edited.csv
-rw------- 1 phussein phussein 122K Jul 31 17:23

21st_year_edited_two.csv
-rw------- 1 phussein phussein 446K Jul 31 17:23 ling_features.csv
-rw------- 1 phussein phussein 120K Jul 31 17:23 tr_tokens.csv
-rw------- 1 phussein phussein 588K Jul 31 17:23 tr_tokens_new.csv
(base) [phussein@submit brains]$ ls -lh /nethome/phussein/brains/bert-brains/data/stimuli/pieman
total 44K
-rw------- 1 phussein phussein 28K Jul 31 17:23 align.csv
-rw------- 1 phussein phussein 15K Jul 31 17:23 tr_tokens.csv
(base) [phussein@submit brains]$ find /nethome/phussein/brains/bert-brains/data -name "*.npy"
(base) [phussein@submit brains]$

Perfekt — damit ist der Stand ziemlich eindeutig.

Aktueller Zustand

Für `pieman` sind die benötigten Input-Daten bereits da:

```
Nur Text

data/stimuli/pieman/
├── align.csv
└── tr_tokens.csv
```

Und es gibt noch **keine `.npy`-Dateien**:

```
Nur Text

find ... -name "*.npy"
→ keine Ausgabe
```

Das heißt: **Wir können jetzt direkt `transformer-representations.ipynb` ausführen.** Das Transcript-Processing müssen wir für `pieman` zunächst nicht erneut machen, weil die benötigten `tr_tokens.csv` bereits vorhanden sind.

Aber: erst einen wichtigen Check

Wir sollten einmal schauen, **welches `tr_tokens.csv` das Transformer-Notebook tatsächlich verwendet**. Es gibt nämlich:

```
data/21st_year/tr_tokens.csv
data/21st_year/tr_tokens_new.csv
data/stimuli/pieman/tr_tokens.csv
```

Da `STIMULUS = "pieman"` gesetzt ist, vermute ich stark, dass `data/stimuli/pieman/tr_tokens.csv` verwendet wird. Ich möchte das aber nicht raten.

Führe bitte aus:

```bash
cd /nethome/phussein/brains/bert-brains/text_processing_and_transformers

grep -n -E "tr_tokens|stimulus_df|STIMULUS|data/" transformer_utils.py
```

und zusätzlich:

```bash
grep -n -E "tr_tokens|STIMULUS|TransformerRSM|process_stimulus" transformer-representations.ipynb | hea
```

Dann sehen wir genau, welche Datei `TransformerRSM` für `pieman` lädt.

Danach

Wenn das passt, würde ich dein `brains.sh` zunächst **nur für die BERT-Repräsentationen** so einstellen:

```bash
#!/bin/bash

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh
conda activate bert-brains

cd /nethome/phussein/brains/bert-brains/text_processing_and_transformers

echo "=== Python ==="
python --version
```

```
echo "=== Transformer representations: BERT ==="

export JUPYTER_PLATFORM_DIRS=1
export TQDM_POSITION=-1
export IPYTHON_SUPPRESS_TQDM_WARNING=1

jupyter nbconvert \
    --to notebook \
    --execute transformer-representations.ipynb \
```

Noch nicht abschicken — erst den `grep`-Check machen. Danach können wir sicherstellen, dass das Notebook die richtigen Daten verwendet und dann den Condor-Job starten.

Das Ziel dieses ersten Laufs ist ganz bewusst nur:

BERT → `pieman` → `.npy`-Repräsentationen

Noch keine neue Modellvariante und noch keine Encoding-Analyse.

das sagt ein anderer chat mir dir:

heute 16:31

fasse mir den aktuellen stand zusammen

Klar. **Aktueller Stand des Kumar-et-al.-Projekts:**

Ziel des Projekts

Du möchtest die Methode aus **Kumar et al.** auf ein **neueres Sprachmodell** übertragen und die Ergebnisse mit einer **BERT-Reproduktion** vergleichen.

Besonders relevant sind:

- **Figure 2:** Vergleich der Sprachmodelle über Hirnareale, relativ zur Noise Ceiling.
- **Figure 3B:** Layer-wise Vergleich von **Embeddings und Transformations**, insbesondere über **Partial Correlations**, um den jeweils einzigartigen Beitrag zur Erklärung der fMRI-Aktivität zu untersuchen.

Was wir bisher gemacht haben

1. Repo und Pipeline

Wir arbeiten mit dem Repo:

```

```

```
/nethome/phussein/brains/bert-brains
```

Die fMRI-Encoding-Pipeline läuft über:

```

```

```
analysis/run_encoding_pieman.py
analysis/banded_ridge_regression.py
```

Du hast dafür einen Wrapper geschrieben, der für Pieman alle BERT-Layer durchläuft.

---

2. Datensatz

Die fMRI-Daten waren **nicht vollständig im Repo enthalten**, deshalb hast du die benötigten Pieman-Daten teilweise selbst heruntergeladen.

Für Pieman gibt es u. a. Subjects:

```

```

```
sub-001
sub-002
sub-003
sub-004
sub-005
sub-006
```

```
sub-008
sub-010
sub-011
sub-012
sub-013
sub-014
sub-015
sub-016
```

→ **14 Subjects**

---

3. BERT-Encoding läuft erfolgreich

Der Job ist inzwischen komplett durchgelaufen:

```

```

```

Finished: sub-016
```

Für jeden Subject wurden

- 12 BERT-Layer
- `activations`
- `z_representations`

berechnet.

Die Ergebnisse liegen unter:

```

```

```

/nethome/phussein/brains/bert-brains/results/
```

z. B.:

```
results/
└── pieman_encoding_layer_0_activations/
    ├── sub-001_parcelwise_results_banded_ridge.npy
    ├── sub-002_parcelwise_results_banded_ridge.npy
    ├── ...
```

---

4. Was wir bisher tatsächlich ausgewertet haben

Wir haben zunächst:

```
pieman_encoding_layer_X_activations/
```

ausgewertet.

Die Datei

```
sub-XXX_parcelwise_results_banded_ridge.npy
```

hat:

```
(1000, 4)
```

also 1000 fMRI-Parcels.

Wir haben für jedes Subject und jeden Layer den Mittelwert der **ersten Spalte** genommen:

```

```

```
x[:, 0].mean()
```

und anschließend über die Subjects gemittelt.

Die vollständigen Ergebnisse über 14 Subjects sind:

```

```

```
Layer 0   0.00769
Layer 1   0.00875
Layer 2   0.01096
Layer 3   0.01039
Layer 4   0.01063
Layer 5   0.01012
Layer 6   0.01203   ← Maximum
Layer 7   0.00965
Layer 8   0.00873
Layer 9   0.00573
Layer 10  0.00716
Layer 11  0.00512
```

Das ist ein **Layer-wise Encoding-Verlauf für BERT-Activations/Embeddings**.

Wichtig: Was wir NOCH NICHT gemacht haben

Die bisherigen Werte sind **noch nicht Figure 3B**.

Wir haben bisher:

gemacht.

Für die eigentliche Figure 3B brauchen wir noch:

also insbesondere die **Transformation-Repräsentationen** und anschließend die **Partial-Correlation-Analyse**.

Auch die `z_representations` haben wir bisher noch nicht inhaltlich ausgewertet.

Wo wir jetzt stehen

Ich würde den Stand momentan so zusammenfassen:

> **Die BERT-Baseline des fMRI-Encoding-Schritts ist erfolgreich reproduziert und für alle 14 Pieman-Subjects berechnet. Erste Layer-wise Encoding-Werte für die BERT-Activations liegen vor. Der nächste Schritt ist, im Repo den Code für die Transformation-Features bzw. den Partial-**

> **Correlation-Schritt von Figure 3B zu identifizieren und anschließend die vollständige Figure-3B-Auswertung für BERT nachzubauen.**

Und **erst danach** sollten wir die gleiche Pipeline auf dein neues LLM übertragen und BERT vs. neues Modell vergleichen.

svg

svg

Ja — das ist ein guter Ausgangspunkt. **Du bist inzwischen deutlich weiter als nur beim Einrichten der Umgebung.**

Ich würde den Stand für dich so ordnen:

1. Technische Umgebung: erledigt ✅

- Das Repository `bert-brains` liegt auf dem Cluster unter: `/nethome/phussein/brains/bert-brains`
- Du hast eine funktionierende Conda-Umgebung `bert-brains` erstellt.
- Wir mussten die ursprüngliche macOS-Umgebung etwas bereinigen (`appnope`, `libcxx`, `llvm-openmp`, alte `jpeg` -/ `libgfortran` -Builds etc.), damit sie auf Linux funktioniert.
- Die entscheidenden alten Versionen wurden installiert:
  - Python 3.9
  - PyTorch 1.9.1
  - Transformers 4.9.1
  - spaCy 2.3.5
- Die Jobs laufen bei dir über **HTCondor**, nicht Slurm.

2. Daten: teilweise vorbereitet ✅

Für `pieman` liegen bereits die Stimulus-/Textdaten vor:

```
Nur Text

data/stimuli/pieman/
├── align.csv
└── tr_tokens.csv
```

Außerdem gibt es bereits die aufbereiteten Daten für `21st_year`.

Wichtig: **`.npy`-Transformer-Repräsentationen sind aktuell nicht vorhanden.**

```bash
find /nethome/phussein/brains/bert-brains/data -name "*.npy"
```

liefert nichts.

Das heißt: Die Transformer-Repräsentationen müssen tatsächlich noch erzeugt werden.

---

3. Transformer-Repräsentationen: hier sollten wir jetzt ansetzen

Das Notebook

```
text_processing_and_transformers/transformer-representations.ipynb
```

ist laut README genau dafür zuständig:

> generate Transformer representations (embeddings, transformations) for regression analyses

Und du hast bereits gesehen, dass dort standardmäßig steht:

```python
MODEL = 'bert-base-uncased'
```

sowie:

```python
foo = TransformerRSM(STIMULUS, model_name=MODEL, verbose=True)

ACTIVATIONS = True
Z_REPS = True

foo.process_stimulus_activations(
    save_activations=ACTIVATIONS,
    save_z_reps=Z_REPS
)
```

Danach werden für jeden Layer `.npy`-Dateien erzeugt.

Das ist momentan der nächste sinnvolle Schritt.

---

4. Danach kommt die eigentliche Reproduktion

Die Pipeline sollte konzeptionell so aussehen:

```
Narratives stimulus
       ↓
TR-aligned tokens
       ↓
BERT
       ↓
┌──────────────────────┐
│ activations          │
│ z_representations    │
└──────────────────────┘
       ↓
fMRI encoding
       ↓
banded ridge regression
       ↓
parcel-wise performance
       ↓
layer-wise results
```

Du hast laut dem anderen Chat bereits einen Encoding-Lauf gemacht. **Aber bevor wir uns darauf verlassen, würde ich jetzt einmal sauber nachvollziehen, welche Dateien tatsächlich vorhanden sind und welche Schritte dafür ausgeführt wurden.** Gerade weil du nach einer Pause wieder einsteigst.

---

5. Dein eigentliches Projekt

Deine Idee ist weiterhin:

Baseline:

> Reproduktion der Kumar-et-al.-Pipeline mit BERT.

Experiment:

> Dieselbe Pipeline mit einem moderneren Transformer-Modell durchführen.

Die Motivation wäre dann:

> BERT/GPT-2 sind vergleichsweise alte Transformer und ihre Repräsentationen weisen bekannte Einschränkungen bei bestimmten semantischen Phänomenen auf. Moderne Sprachmodelle könnten reichhaltigere bzw. semantisch geeignetere Repräsentationen entwickeln. Daher untersuchen wir, ob Repräsentationen eines neueren Modells die menschliche Gehirnaktivität besser vorhersagen.

Das ist für die Kursanforderungen **sehr passend**, weil es direkt unter

> *Investigate whether some ML model … is or isn't brain-like in some respect*

bzw. auch unter

> *Replicate a research paper, varying some aspect that wasn't investigated in the original paper*

fällt.

Der **Bonus mit functional specialization** wäre dann eine zweite, kleinere Forschungsfrage und muss nicht dein Hauptprojekt tragen.

Was ich jetzt von dir brauche

Da du einige Wochen Pause hattest, würde ich **nicht einfach blind weitere Jobs starten**.

Schick mir als Nächstes bitte die Ausgabe von:

```bash
cd /nethome/phussein/brains/bert-brains

find results -maxdepth 2 -type f | head -50
```

und:

```bash
find . -name "*.py" | sort
```

und insbesondere:

```bash
```

```
sed -n '1,240p' text_processing_and_transformers/transformer_utils.py
```

sowie:

```
</> Bash

sed -n '1,240p' analysis/run_encoding_pieman.py
```

Falls `run_encoding_pieman.py` nicht existiert:

```
</> Bash

ls analysis
```

Dann können wir **den tatsächlichen aktuellen Stand des Repos rekonstruieren**, statt aufgrund des alten Chats anzunehmen, was schon gelaufen ist.

Danach würde ich mit dir Schritt für Schritt **zuerst BERT vollständig reproduzieren** und erst wenn diese Baseline steht, das neue Modell einbauen.